Introducción a la Parte III

Traducción al español, no oficial, de la Parte III (introducción y capítulos 20–25) del Handbook of Digital and Computational Research Methods, eds. Anders Koed Madsen y Anders Kristian Munk, para uso personal y docente. Las referencias bibliográficas y los rótulos internos de las figuras se conservan en inglés.

Nosotros, los autores de los capítulos de esta sección, creamos herramientas que se utilizan en las ciencias sociales y las humanidades digitales y computacionales (DCSSH). Algunas de las más conocidas son Gephi[1] (Bastian et al., 2009) para la visualización de redes; CitNetExplorer[2] y VOSviewer[3] (Van Eck and Waltman, 2017) para la bibliometría; DaCy[4] (Enevoldsen et al., 2021) y Talisman[5] (Plique, 2020) para el procesamiento del lenguaje natural; 4CAT[6] (Peeters and Hagen, 2022), DMI-TCAT[7] (Borra and Rieder, 2014), Contropedia[8] (Borra et al., 2015), Hyphe[9] (Jacomy et al., 2016), Minet[10] (Plique et al., 2025), Netvizz[11] (Rieder, 2013) y las YouTube Data Tools[12] (Rieder, 2015) para la investigación sobre redes sociales; y Prompt Compass[13] (Borra, 2023b) para aplicar grandes modelos de lenguaje a las SSH.

Está en la naturaleza de una herramienta estar ya ahí, darse por sentada. Cuando los investigadores de las ciencias sociales o las humanidades toman conocimiento de una herramienta que podría ayudarlos, pueden recibirla bajo una serie de supuestos como los siguientes: esta herramienta ha sido creada por creadores de herramientas profesionales; ha sido diseñada para investigadores cuyas necesidades son similares a las mías; el uso que se promueve de ella es el propósito de su diseño; viene con un método que debería funcionarme como les funciona a otros; su popularidad significa que ha sido validada por la comunidad de investigación; debe estar financiada y mantenida por personas cuyo trabajo es mantenerla; y perdurará. Cada una de estas cosas dista mucho de cumplirse siempre, y comprender por qué puede ayudarlo a relacionarse mejor con su aparato científico, o incluso a contribuir a mejorarlo.

A lo largo de los capítulos de esta sección, nos proponemos cuestionar los supuestos enumerados arriba, porque experimentamos cuán precarias pueden ser estas herramientas. Queremos explicar por qué una herramienta bien puede haber sido creada accidentalmente por personas que sólo querían alcanzar sus propios objetivos de nicho, que no la concebían como una herramienta (y menos aún como una herramienta popular), pero que quedaron atadas a ella cuando circuló entre desconocidos que la readaptaron y se la reapropiaron, incorporándola a sus propios métodos, moldeando la cultura que la rodea mediante artefactos como tutoriales, artículos académicos o plugins, y acentuando una desconexión entre los usuarios y el creador de la herramienta. Alternativamente, una herramienta puede haber sido creada y ser mantenida para un público no académico por desarrolladores profesionales que no son investigadores y que nunca llegan a conocer a los usuarios (p. ej., OpenRefine;[14] Ham, 2013). La creación de herramientas es un panorama diverso, aun si sólo se consideran las aplicaciones a las DCSSH. Incluso una herramienta popular bien puede quedarse sin financiamiento, sin mantenimiento,

abandonada en un estado precario por su creador involuntario, o desconectada de su fuente de datos, hasta que deja de funcionar y es abandonada también por sus usuarios. Reconocer de dónde proviene la tecnología académica nos ayudará a comprender por qué y cómo fue moldeada.

Dicho esto, la mayoría de las herramientas digitales y computacionales para las ciencias sociales y las humanidades (que abreviaremos como «herramientas DC para las SSH») no tienen un destino trágico; simplemente llevan una vida aparentemente apacible, en la que los cambios son poco frecuentes y poco dramáticos. Cada instrumento científico tiene una combinación única de rasgos, y su situación cambia a lo largo de su ciclo de vida a medida que se lo vuelve a desarrollar, se vuelve popular, recibe financiamiento, pasa de un grupo a otro, etc. No obstante, casi cualquier herramienta presenta un rasgo problemático: independientemente del grado de compromiso metodológico incorporado en ella, su diseño no apuntaba a poner en práctica el método del estado del arte. No sólo porque el estado del arte casi nunca se reduce a un único método consensuado, sino ante todo porque la motivación del creador de la herramienta está en otra parte.

Comprender los orígenes, las trayectorias y las limitaciones de las herramientas DC para las SSH no es un mero ejercicio académico: incide directamente en la manera en que los investigadores llevan a cabo su investigación. Las herramientas nunca son estáticas; evolucionan, dejan de funcionar, se discontinúan o se resisten a la adaptación, a veces de maneras que moldean de forma fundamental los tipos de preguntas que los investigadores pueden formular y responder. Relacionarse con los creadores de herramientas y con sus procesos permite a los investigadores sortear mejor estos cambios, ya sea influyendo en el desarrollo de las herramientas, resolviendo problemas o incluso creando soluciones a medida. Es también una invitación a reconocer a los creadores de herramientas como colaboradores vitales en el ecosistema de investigación, o quizá incluso a convertirse uno mismo en creador de herramientas. Además, las herramientas suelen servir como vehículos de impacto académico: al dominar una herramienta o contribuir a su desarrollo, los investigadores pueden encontrarse integrados en nuevas redes de investigación e invitados a proyectos colaborativos. Comprender el papel precario pero poderoso de las herramientas en las DCSSH es clave tanto para orientarse en el futuro de nuestra investigación como para darle forma, lo que proponemos a lo largo de un recorrido de seis capítulos.

AGRADECIMIENTOS

Los capítulos de la Parte III surgen de un panel sobre los creadores de herramientas en las ciencias sociales y las humanidades convocado por el centro MASSHINE de Aalborg University durante el otoño de 2023. El panel incluyó a todos los autores de este texto, además de Antonin Delpeuch, responsable del mantenimiento de OpenRefine, a quien agradecemos calurosamente sus ideas. Dado que nosotros, los autores, provenimos de un conjunto internacional y diverso de instituciones, que abarcan las humanidades, las ciencias sociales, los estudios de medios y la

cienciometría, las observaciones y los comentarios aquí escritos no reflejan las experiencias de un individuo ni de una universidad o un país específicos, sino que pueden considerarse más amplios y endémicos del campo de las humanidades y las ciencias sociales digitales y computacionales. Para reforzar aún más este punto, recopilamos y citamos extensamente trabajos de otros creadores de herramientas de dentro y fuera de esos campos.

Utilizaremos citas clave de una larga entrevista de Jacomy a Geoffrey Rockwell en 2023 como hilo conductor de las numerosas facetas que abordamos. Geoffrey es uno de los creadores de herramientas (junto con el difunto Stéfan Sinclair) detrás de Voyant, una plataforma en línea para explorar textos hermenéuticamente en las humanidades digitales. Recurrimos a sus experiencias con Voyant para ilustrar algunos de nuestros planteos.

NOTAS

1. Gephi es una plataforma de software de código abierto para visualizar y analizar redes complejas y datos de grafos mediante exploración interactiva, algoritmos de disposición espacial y métricas estadísticas.
2. CitNetExplorer (Citation Network Explorer) es una herramienta de software para visualizar y analizar redes de citas de publicaciones científicas, que permite a los investigadores explorar cómo se conectan las obras académicas a través de las referencias a lo largo del tiempo.
3. VOSviewer es una herramienta de software para construir y visualizar redes bibliométricas, especializada en mostrar relaciones entre publicaciones, autores y palabras clave mediante técnicas de mapeo basadas en la distancia.
4. DaCy es un pipeline de procesamiento del lenguaje natural para el danés construido sobre SpaCy, que ofrece capacidades de análisis de texto de alto rendimiento específicamente optimizadas para procesar textos en danés.
5. Talisman es una biblioteca de JavaScript que ofrece una amplia colección de funciones de análisis de texto y métricas de similitud de cadenas, diseñada para tareas de coincidencia aproximada, detección de patrones y comparación de textos.
6. 4CAT (Capture and Analysis Toolkit) es una herramienta de investigación basada en la web que captura, procesa y analiza datos de redes sociales y contenidos web.
7. DMI-TCAT (Digital Methods Initiative - Twitter Capture and Analysis Toolset) es una herramienta de investigación de código abierto diseñada para recopilar, archivar y analizar datos de Twitter mediante funciones avanzadas de consulta y visualización.
8. Contropedia es una herramienta analítica basada en la web que visualiza y analiza la evolución de las controversias en los artículos de Wikipedia mediante el seguimiento de los historiales de edición, las discusiones y las disputas sobre el contenido.
9. Hyphe es una herramienta de rastreo web y construcción de corpus diseñada para las SSH, que permite a los usuarios recopilar, categorizar y analizar datos web de redes personalizadas de sitios web.
10. Minet es una versátil herramienta de línea de comandos para investigaciones digitales, que permite la recopilación, extracción y procesamiento eficientes de datos web de diversas fuentes, incluidos redes sociales, motores de búsqueda y sitios web.
11. Netvizz era una herramienta para extraer datos de Facebook con fines de investigación, que permitía a los usuarios analizar redes, contenidos e interacciones, hasta que fue discontinuada debido a restricciones de la plataforma.
12. YouTube Data Tools es un conjunto de herramientas basadas en la web para la recopilación y el análisis de datos de YouTube, que permite a los investigadores reunir información sobre metadatos de videos, comentarios, canales e interacciones en red.

13. Prompt Compass es una herramienta diseñada para aprovechar los grandes modelos de lenguaje (LLM) en tareas de investigación digital, al brindar acceso a diversos LLM, ofrecer una biblioteca de prompts para la investigación digital y permitir a los usuarios aplicar esos prompts a una serie de entradas.
14. OpenRefine es una potente herramienta de código abierto para limpiar, transformar y organizar grandes conjuntos de datos, que permite a los investigadores explorar, editar y conciliar de manera eficiente datos desordenados para su análisis.

REFERENCIAS

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. Proceedings of the International AAAI Conference on Web and Social Media, 3(1), 361–362.

Borra, E. (2023). Prompt Compass. A Tool for Navigating LLMs and Prompts for Computational Social Science and Digital Humanities Research [Computer software]. Zenodo.

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. Aslib Journal of Information Management, 66(3), 262–278.

Borra, E., Weltevrede, E., Ciuccarelli, P., Kaltenbrunner, A., Laniado, D., Magni, G., Mauri, M., Rogers, R., & Venturini, T. (2015). Societal controversies in Wikipedia articles. Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (pp. 193–196). https://doi.org/10.1145/2702123.2702436

Enevoldsen, K., Hansen, L., & Nielbo, K. (2021). DaCy: A unified framework for Danish NLP. arXiv preprint: arXiv:2107.05295.

Ham, K. (2013). OpenRefine (version 2.5). http://openrefine.org. Free, open-source tool for cleaning and transforming data. Journal of the Medical Library Association, 101(3), 233.

Jacomy, M., Girard, P., Ooghe-Tabanou, B., & Venturini, T. (2016). Hyphe, a curation-oriented approach to web crawling for the social sciences. Proceedings of the International AAAI Conference on Web and Social Media, 10(1), 595–598.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. Computational Communication Research, 4(2), 571–589.

Plique, G. (2020). Talisman: A JavaScript archive of fuzzy matching, information retrieval and record linkage building blocks. Journal of Open Source Software, 5(55), 2405.

Plique, G., Breteau, P., Farjas, J., Théro, H., Descamps, J., Pellé, A., Miguel, L., Pichon, C., Christensen, K,, Pontoire, J., & Mazoyer, B. (2025). Minet, a web mining CLI tool & library for python. (4.1.0). Zenodo. https://doi.org/10.5281/zenodo.15411918

Rieder, B. (2013). Studying Facebook via data extraction: The Netvizz application. Proceedings of the 5th Annual ACM Web Science Conference, 346–355.

Rieder, B. (2015). YouTube Data Tools [software]. Available at: https://tools.digitalmethods.net/netvizz/youtube/ (accessed 09/08/2024).

Van Eck, N. J., & Waltman, L. (2017). Citation-based clustering of publications using CitNetExplorer and VOSviewer. Scientometrics, 111, 1053–1070.

20. Crear herramientas es hacer ciencia por otros medios

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

La creación de herramientas «nunca es meramente técnica» (Rieder et al., 2024). Equipar con herramientas a las SSH nunca se reduce a ahorrar costos y tiempo; también reconfigura las prácticas del investigador. Usar una herramienta modifica su agencia de maneras más profundas que la mera incorporación de una capacidad temporal. Para hacer visibles cosas nuevas, la herramienta oculta otras; al facilitar algunos caminos metodológicos, dificulta comparativamente otros; y, en última instancia, la herramienta cambia sus objetivos. Como dice el conocido refrán, «si la única herramienta que se tiene es un martillo, es tentador tratar todo como si fuera un clavo» (Maslow, 1966, p. 15).

El investigador de las SSH equipado necesita comprender cómo el aparato que utiliza incide en la naturaleza de su actividad académica. Esto requiere reconocer que la producción de herramientas no es puramente técnica, sino que tiene una carga académica, como desarrollaremos en este capítulo. El modo en que tiene esa carga depende de cada herramienta, pero que la tenga no sorprenderá al lector de este manual.

Más sorprendente, sin embargo, puede resultar el hecho de que muchas herramientas DC para las SSH ni siquiera están creadas para usted (como usuario); al menos no inicialmente. Tienen una carga académica porque emanan de prácticas académicas, como extensiones naturales de teorías y métodos («naturales» desde la perspectiva del creador de la herramienta). Este será nuestro punto de partida para comprender, en este capítulo, las circunstancias específicas en las que «las herramientas son portadoras de teoría» (Rockwell, 2023; Rockwell & Sinclair, 2016).

Al presentar de entrada la creación de herramientas en las DCSSH como una forma de ciencia, buscamos ayudar al investigador de las SSH a mirar su aparato DC de otra manera, atendiendo tanto a lo que este necesita de él como a lo que él necesita de este, y a sacarle mejor provecho.

¿QUÉ ES UNA HERRAMIENTA DC PARA LAS SSH?

Las herramientas utilizadas en las DCSSH son instrumentos científicos; pertenecen a una gran familia de artefactos tecnológicos —no necesariamente informatizados— incorporados a la producción de conocimiento —no necesariamente dentro de las SSH— que incluye telescopios, microscopios, balanzas, sismógrafos, cámaras de niebla, equipos de resonancia magnética, imágenes satelitales y muchos más. «Instrumento» añade un matiz pertinente a la palabra «herramienta», aunque a menudo usaremos esta última por brevedad: mientras que el propósito

de una herramienta cotidiana suele ser intervenir sobre la materia (martillo, sierra, soldador…), el propósito de un instrumento científico suele ser, en un sentido amplio, realizar una medición. Así, las herramientas DC para las SSH son un subconjunto de instrumentos científicos que los investigadores de las SSH utilizan para facilitar su producción de conocimiento en un entorno digital y/o computacional, mediante la recolección de datos, su procesamiento y, en última instancia, la producción de información (textos, números, visualizaciones…) derivada de ellos.

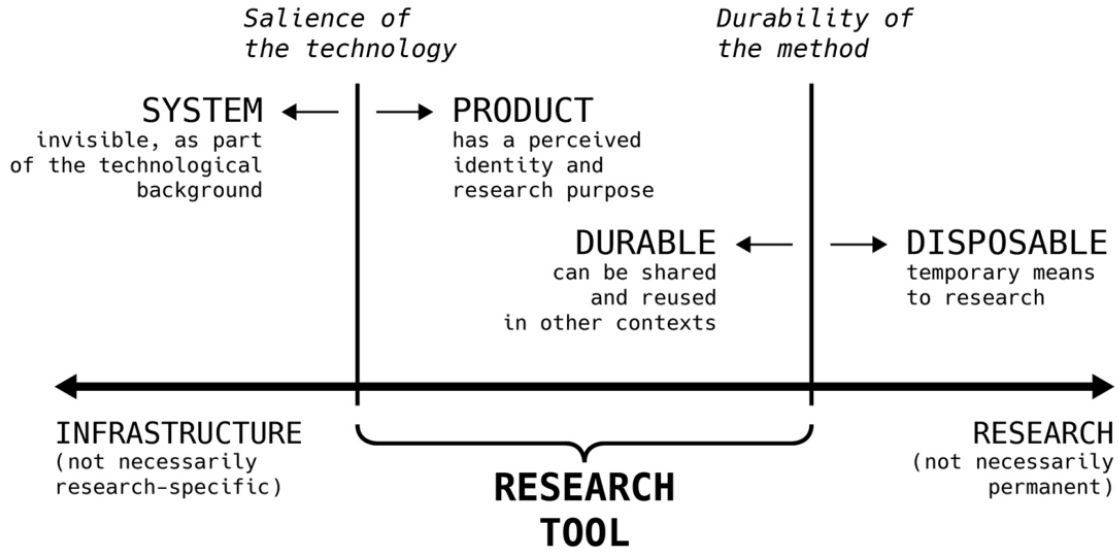


Figura 20.1 Los instrumentos científicos (herramientas de investigación) se relacionan tanto con las infraestructuras (como objetos tecnológicos) como con la investigación (como objetos metodológicos)

Fuente: elaboración propia.

Las herramientas que utilizamos en las SSH tienen, por lo tanto, una doble filiación. Como herramientas, pertenecen a la serie de objetos tecnológicos que nos dan apoyo en nuestras empresas académicas, serie que incluye también muchas infraestructuras (acceso a internet, servicios bibliotecarios, almacenamiento en la nube, mantenimiento informático…). Y como objetos científicos, pertenecen a la serie de medios metodológicos a través de los cuales damos forma a nuestra producción académica. Sin embargo, las infraestructuras y la metodología rara vez se superponen, ya que esperamos una división del trabajo entre los ingenieros que tienen las competencias para mantener las infraestructuras y los académicos a cargo de la dimensión científica de sus métodos. Además, la mayoría de las infraestructuras utilizadas en un entorno académico no son específicas de la ciencia. La doble naturaleza de las herramientas de investigación tiende un puente sobre una división profundamente arraigada en las universidades. Examinaremos esa división con más detenimiento en el próximo capítulo, al presentar la figura del creador de herramientas. Por ahora, limitémonos a señalar que nuestras herramientas de investigación viven en la intersección entre la infraestructura y la investigación, lo que las hace doblemente especiales (figura 20.1). En primer lugar, como objetos tecnológicos, tienen la especificidad de portar un propósito científico, lo que les impide fundirse en el trasfondo de nuestra práctica tecnológica. Necesitamos poder nombrar y discutir nuestros instrumentos

científicos porque tienen una carga metodológica. En segundo lugar, como métodos, tienen la especificidad de ser lo suficientemente duraderos como para reutilizarse tal como vienen. Las SSH son ricas en métodos que necesitan la participación activa del investigador para desplegarse, pero el método implementado por un algoritmo no necesita esa participación activa; puede ser ejecutado por cualquiera (aunque no necesariamente comprendido o interpretado por cualquiera). En ese sentido, las herramientas de investigación son a la vez tecnologías particularmente destacadas, lo que las separa de otras tecnologías útiles para el investigador de las SSH, y métodos particularmente duraderos, lo que las separa de otras partes de la caja de herramientas metodológica de las SSH. Una herramienta es un producto: tiene un nombre, un propósito o una función declarados, a veces un método documentado y revisado por pares, y sus desarrolladores rinden cuentas por ella. A menudo tiene una identidad visual, un sitio web, documentación y artefactos dedicados a la comunidad de usuarios. En el extremo infraestructural del espectro, las herramientas dejan de ser herramientas cuando se convierten en sistemas: máquinas que administrar, servicios que mantener, etc. La creación de herramientas se convierte en mantenimiento, reparación y soporte. En el extremo de la investigación, los artefactos metodológicos dejan de existir como herramientas cuando dejan de perdurar porque no están documentados ni mantenidos, ni siquiera simplemente compartidos. Encontramos el método como pura práctica, por ejemplo, como descubrimiento y exploración. La creación de herramientas se convierte entonces en investigación por medios tradicionales.

Las distinciones producto/sistema y duradero/desechable son direcciones más que fronteras. Distintas personas trazarán la línea en distintos lugares. Por ejemplo, la tecnología de servidores puede parecerle una herramienta a un administrador de sistemas y, al mismo tiempo, parecerle infraestructura a un profesor. La conclusión clave es que los instrumentos científicos existen porque hay una superposición entre la tecnología que es duradera (tarea del soporte) y la que tiene un propósito científico (tarea de los académicos). Esto sugiere dos maneras en que surgen las herramientas de las DCSSH: cuando una pieza de la infraestructura tecnológica se vuelve visible para los investigadores al adquirir una identidad y un propósito científico declarado, o, a la inversa, cuando la tecnología que da soporte a una práctica científica se vuelve duradera y es compartida y reutilizada por otras personas. Ofreceremos un panorama de los distintos tipos de herramientas DC para las SSH y volveremos con más detalle sobre la cuestión de su ciclo de vida en el capítulo 23.

LAS HERRAMIENTAS DC PARA LAS SSH NO SE CREAN PARA SUS USUARIOS

> [Decidimos] que no íbamos a construir una herramienta basada en un análisis de necesidades de otras personas. Íbamos a construir la herramienta que queríamos, la que necesitábamos para hacer los experimentos que queríamos hacer. (Geoffrey Rockwell, en Jacomy, 2023)

Hay muchas razones por las que se crean herramientas DC para las SSH, y satisfacer las necesidades de otras personas rara vez es la principal. Ese tipo de creación de herramientas no tiene que ver con captar un mercado de usuarios, esencialmente porque el mercado de

estudiantes, investigadores, periodistas y activistas sin dinero es demasiado pequeño y demasiado riesgoso como para justificar un trabajo de tiempo completo. Las herramientas DC para las SSH no se crean para enriquecer a sus creadores, sino sobre otras premisas. Veamos algunas razones por las que se crean herramientas DC para las SSH.

Algunas herramientas nacen de la mano de su creador para responder a sus propias preguntas de investigación o satisfacer sus propias necesidades. El creador de la herramienta la crea para sí mismo y luego la comparte con el mundo como efecto secundario. Hay muchas razones para compartirla, entre ellas, que a veces resulta barato y fácil una vez que la herramienta ya existe. Pero aun así, la herramienta no se creó en primer lugar para ser compartida; se compartió porque ya existía. Un buen ejemplo de ello es la herramienta de conveniencia, desarrollada por un investigador-programador para sí mismo con el fin de resolver un problema con el que se encuentra, y compartida posteriormente porque otros tienen el mismo problema. Tales herramientas pueden ser tan modestas como una simple expresión regular envuelta en un formulario HTML, como las herramientas de Borra compare lists, URL harvester o el TLD counter, o pueden añadir un formulario a una cadena de servicios, como Expand Tiny URLs o GeoIP lookup for URLs. Las herramientas de conveniencia no son específicas de las SSH, pero la necesidad de reproducibilidad y el ethos de compartir propio de la academia fomentan su difusión en comparación con otros contextos, como la industria del software.

Algunas herramientas se crean porque implementan los métodos que el autor quiere difundir. Pueden ser vehículos explícitos o implícitos de un método determinado. Por ejemplo, el Lippmannian Device de Borra se compone esencialmente de sólo dos campos de formulario que permiten a los usuarios configurar, consultar y extraer resultados de motores de búsqueda. Sin embargo, al fundamentar las prácticas de la herramienta en las teorías de Walter Lippmann (1922) sobre la opinión pública, la herramienta permite a los investigadores identificar qué actores de la web se alinean con temas específicos y cómo determinados actores abordan los distintos temas (Rogers, 2010). Así, las herramientas pueden difundir el método del creador de la herramienta o el de otra persona, y no tienen por qué ser sólo un vehículo; también pueden tener otros objetivos. Ese tipo de herramientas está intrínsecamente ligado a un conjunto específico de métodos, porque ese es su propósito principal. Por ejemplo, la herramienta de análisis textual IRaMuTeQ de Pierre Ratinaud (Ratinaud, 2009), popular para el análisis del francés y el portugués, está dedicada explícitamente al método ALCESTE (Reinert, 1990), que en este caso fue concebido por otra persona (Ratinaud & Marchand, 2012). Otro ejemplo es la biblioteca Graph-tool de Tiago Peixoto (Peixoto, 2014), que toma posición explícitamente, se centra en los algoritmos del propio creador de la herramienta y refleja sus opiniones acerca de «por qué se deberían evitar los métodos estándar disponibles en otros paquetes de software» (ibid).

Algunas herramientas nacen como subproducto de la práctica académica. Un buen ejemplo en las DCSSH es Netvizz, de Bernard Rieder, una aplicación hoy discontinuada que permitía

descargar y analizar datos de Facebook antes de que la plataforma cerrara su API (Rieder, 2013). La herramienta existió durante más de diez años y atravesó distintas etapas. Comenzó como un simple experimento de software; se convirtió en un medio eficaz para comprender mejor la API de Facebook; luego pasó a ser una forma de que Rieder cultivara sus habilidades de programación, y a partir de allí se convirtió en una responsabilidad molesta para él, porque muchísimas personas la usaban y le pedían asistencia. Netvizz no se creó inicialmente para otras personas, pero pasó a ser mantenida y rediseñada para sus usuarios, aunque, si alguna vez hubiera recibido financiamiento, la situación podría haber sido muy distinta.

Por último, aunque no por ello menos importante, algunas herramientas apuntan a formular un argumento teórico. En las DCSSH en particular, estas herramientas están entrelazadas con una bibliografía de décadas. Como enumera Endres (2017), «los investigadores han propuesto la construcción como modelización teórica (McCarty, 2004), el prototipo como teoría (Manovich en Ramsay & Rockwell, 2012), el prototipo como argumento (Galey & Ruecker, 2010), la API como argumento (Sherratt, 2015), el diseño como argumento (Galey & Ruecker, 2010; Eyman & Ball, 2015), las herramientas de visualización como teoría hermenéutica (Rockwell, 2001), los artefactos digitales como lentes teóricas o "telescopios para la mente" (McCarty, 2012), la construcción como intervención académica (Ramsay & Rockwell, 2012) y la experimentación artesanal como un proceso de ensoñación académica, en el que la teoría y la creación colisionan (Prescott, 2014)». Como ejemplo reciente, Gephisto (Jacomy & Munk, 2024) fue creado para sostener el argumento de que «las herramientas que apoyan el pensamiento crítico no tienen por qué ser difíciles de usar ni hostiles para los principiantes» (ibid, p. 142).

Las herramientas utilizadas en las SSH tienen muchas otras razones para haber sido creadas además de equipar al investigador típico. Por el contrario, el creador de la herramienta suele estar en el centro del propósito de la herramienta cuando esta se crea: porque la herramienta apunta a hacer avanzar su investigación, o a difundir sus métodos, o a sostener sus habilidades de programación, o a formular un argumento. Si la utilidad para otros llega, suele hacerlo en una fase posterior (véase el capítulo 23 sobre el ciclo de vida de las herramientas). Esta situación explica por qué algunas de las necesidades de los investigadores de las DCSSH están cubiertas por múltiples herramientas mientras que otras quedan sin atender: satisfacer esas necesidades no suele ser un incentivo relevante para crear una herramienta. Hay dos razones para ello. En primer lugar, desde el punto de vista del creador de la herramienta, que una herramienta se vuelva popular suele acarrear más cargas que beneficios y, por lo tanto, no es necesariamente algo a lo que aspirar. Las respuestas a la encuesta a creadores de herramientas de las DH de Schreibman y Hanlon (2010) «sí demostraron que las bajas tasas de adopción eran algo que, a juicio de los desarrolladores, perjudicaba el valor de sus herramientas, pero no era su mayor preocupación: el nivel de adopción quedó en cuarto lugar entre cuatro posibles medidas del éxito de una herramienta». En segundo lugar, incluso desde la perspectiva de satisfacer las necesidades de mucha gente, sigue existiendo el riesgo de que la herramienta no encuentre a su público y de que se pierdan el tiempo y la energía invertidos. Al menos en las DCSSH, los

creadores de herramientas rara vez tienen los recursos para hacer estudios de mercado, reorientar su producto, etc. En la economía precaria de la creación de herramientas en las DCSSH, una herramienta sólo se crea cuando su creador obtiene un retorno rápido por su tiempo y esfuerzo: nuevos datos, una oportunidad de publicar un artículo, una colaboración estimulante, una subvención, algo de publicidad… Sólo después de esa etapa surge la cuestión de la popularidad.

> En mi experiencia, cuando hablo con colegas que nunca han usado realmente el análisis de textos asistido por computadora, cuando describen lo que les gustaría tener, es una fantasía. Aunque uno se lo diera, en realidad no lo usarían. Es lo que creen que usarían. Pero no es lo que realmente les sería útil. No saben lo suficiente como para ser una guía buena y confiable. Al menos tuvimos una subvención, un proyecto financiado llamado Just What Do They Do?, en el que sí entrevistamos efectivamente a un montón de personas. Les mostramos distintas pantallas y les preguntamos qué podían hacer con esto y con aquello. … Y, en [líneas] generales, tengo que decir que nos divertimos más y llegamos más lejos simplemente haciendo lo que queríamos; lo que nos parecía interesante. Así que, en ese sentido, no hemos hecho el canónico análisis de necesidades, los estudios de usuarios, etc. No estábamos construyendo cosas para otras personas. Estábamos construyendo cosas para nosotros y para gente como nosotros. ¡Y funcionó! (Geoffrey Rockwell, en Jacomy, 2023)

MUCHAS HERRAMIENTAS DC PARA LAS SSH SON DISPOSITIVOS DE ENSEÑANZA

> Scott Weingart llamó a [Voyant] "la droga de iniciación a las humanidades digitales". Es una gran herramienta de nivel inicial. Y eso hace que se use ampliamente. (Geoffrey Rockwell, en Jacomy, 2023)

La enseñanza es a menudo lo que hace popular una herramienta. Ha sido el caso de Gephi, Netvizz, Voyant y muchas otras. Algunas herramientas se desarrollaron ante todo para la enseñanza, como Netvizz, que «comenzó inicialmente en 2008 como un demostrador puramente técnico para un curso de programación web» (Rieder et al., 2024). Algunas herramientas se desarrollaron con otro propósito, pero pensando en la enseñanza, como Gephisto, concebida principalmente como «una intervención en el campo de los métodos digitales» (Jacomy & Munk, 2024), pero también como punto de partida para enseñar análisis de redes. Algunas no se crearon pensando en la enseñanza, pero llegaron a usarse en ese contexto (Gephi); y, por supuesto, algunas herramientas no tienen ninguna afinidad particular con el aula (DaCy, Minet).

Sin embargo, ninguna herramienta es completamente ajena a la situación de enseñanza, aunque sólo sea porque las personas necesitan aprender a usarla. En cuanto una herramienta tiene una comunidad de usuarios, surge material didáctico: tutoriales, introducciones en video, preguntas frecuentes… Parte o incluso la totalidad de ese material puede provenir de la propia comunidad, no de los creadores de herramientas. Incluso cuando los creadores de herramientas no tienen en mente la enseñanza, algunos docentes pueden usar su herramienta en sus clases, o incluso como eje de una clase magistral o hasta de un curso entero. Esto se vuelve inevitable a medida que una herramienta gana popularidad. Por ejemplo, Gephi ha sido el tema central de al

menos dos libros (Cherven, 2015; Khokhar, 2015), ninguno de los cuales fue escrito por sus creadores de herramientas, ni siquiera está asociado con ellos. Cuanto más vive una herramienta, más probable es que alguien la enseñe.

A la inversa, enseñar una herramienta la hace más popular. Los estudiantes que han aprendido la herramienta la usarán, la llevarán a otros lugares y la presentarán a otros públicos. Esto suele llevar años, lo que explica en parte por qué las herramientas de investigación tienen mucha inercia: pueden tardar en encontrar su público, pero pueden conservarlo durante muchísimo tiempo. Incluso años después de que la herramienta haya sido abandonada, alguien puede necesitar usarla, por ejemplo, para replicar una publicación académica en la que la herramienta se utilizó como parte de la metodología. Cuando los docentes usan herramientas como parte de su plan de estudios, los desarrolladores pueden recibir correcciones urgentes de errores (bugs) de estudiantes o profesores que descubren problemas, incluso después de que el desarrollador haya terminado o dejado el proyecto sin haber sido informado de su uso continuado o en el aula. La enseñanza y la popularidad están en un bucle de refuerzo: enseñar la herramienta la hace más popular, y la popularidad aumenta las probabilidades de que una herramienta se use.

Cuando una herramienta se vuelve popular porque se enseña, su popularidad seguirá la localización de su enseñanza: una universidad determinada, un campo académico, una lengua… La comunidad de usuarios de una herramienta no está necesariamente unificada. Más bien deberíamos usar el plural. Las comunidades de usuarios no son neutrales ni universales, sino situadas. Cada una puede cultivar su perspectiva parcial sobre la herramienta: su propósito, sus fortalezas y debilidades, sus configuraciones preferidas y sus buenas prácticas. Por ejemplo, los científicos de redes consideran que Gephi es muy fácil de usar, pero los humanistas digitales consideran que es difícil de aprender. Los primeros pueden usarlo por sus visualizaciones, pero encontrar demasiado básicos los algoritmos que incorpora, mientras que los segundos pueden usarlo como su plataforma principal para calcular estadísticas de redes. Los primeros pueden entender las redes como modelos estadísticos (Peixoto, 2023), mientras que los segundos pueden verlas como un medio para describir fenómenos (Venturini et al., 2021). Algunas comunidades de usuarios forman su propia «cultura epistémica» o subcultura (Knorr Cetina, 1999; véase también Jacomy & Jokubauskaitė, 2022).

A pesar del impacto de la enseñanza en la popularidad de las herramientas, desde el punto de vista del docente, presentar un único instrumento tiene inconvenientes, porque no está garantizado que los estudiantes puedan usarlo en sus trabajos, típicamente porque la organización está comprometida con una solución alternativa. En consecuencia, el docente puede optar por abordar varias herramientas para un mismo propósito. Esto tiene la ventaja de hacer visibles, por comparación, las elecciones específicas. Este tipo de inquietud constituye la especificidad de la situación de enseñanza. Algunas herramientas están mejor sintonizadas con ella, por ejemplo, porque resultan útiles como formas de introducir otras herramientas, aunque esas otras herramientas puedan estar mejor adaptadas a otros contextos.

En el caso de las DCSSH en particular, bastantes herramientas fueron diseñadas por alguien que, paradójicamente, no se sentía cómodo con la tecnología existente. Algunas herramientas son creadas por investigadores para sí mismos. Netvizz fue desarrollado inicialmente por Rieder (2013) como un experimento de software, una manera de comprender la tecnología y mejorar sus habilidades de programación; Gephi fue prototipado inicialmente por Jacomy para sí mismo y para investigadores de las DCSSH, y sólo posteriormente pasó a manos de los desarrolladores con formación profesional Mathieu Bastian y Eduardo Ramos Ibañez. DMI-TCAT se creó como respuesta a los paneles de datos de las empresas de marketing, que traían incorporadas, p. ej., métricas de vanidad en lugar de métricas propias de las ciencias sociales (Borra and Rieder, 2014). Como ya mencionamos, la incomodidad con la tecnología puede motivar a alguien a crear una herramienta porque busca beneficiarse del proceso de creación de herramientas más que de su resultado. Además, el creador de herramientas también podría estar motivado por el hecho de que las herramientas existentes no son adecuadas para las DCSSH: demasiado complicadas, demasiado caras, sacrifican demasiada calidad en favor del rendimiento… Algunos creadores de herramientas de las DCSSH se parecen más a un estudiante o a un investigador que a un desarrollador profesional empleado en una empresa tecnológica. Por esa misma razón, las herramientas diseñadas por ese tipo de creador de herramientas están naturalmente sintonizadas con el aula. Les hablan a los principiantes porque abren el camino, porque el creador de herramientas actuó como passeur de métodos y algoritmos, o simplemente porque el creador de herramientas es quien sabe programar.

Los propios creadores de herramientas pueden enseñar su herramienta, ya sea a estudiantes o a investigadores —posiblemente como una forma de promoverla, pero, como veremos en el próximo capítulo, el deseo de popularizar la propia herramienta no debe darse por sentado. De manera más pragmática, el creador de herramientas puede estar buscando retroalimentación. A través de ella, uno puede aprender en qué situaciones la herramienta es eficiente o ineficiente, cuándo falla y qué funcionalidades los usuarios entienden o malinterpretan. Con bastante frecuencia, la situación de enseñanza también puede inspirar nuevas funcionalidades cuando los usuarios readaptan la herramienta de maneras inesperadas. Para algunos investigadores, crear herramientas es enseñar por otros medios.

> Cada vez que alguien me pregunta: ¿Podría conectarse por Zoom a mi clase y dar una introducción a Voyant?, digo: Sí. A esta altura, puedo darlas hasta dormido. Las doy con regularidad. Simplemente llego y hablo de Voyant, o dirijo una sesión de una hora. De eso obtengo cierta cantidad de retroalimentación. (Geoffrey Rockwell, en Jacomy, 2023)

LAS HERRAMIENTAS EMPODERAN A LOS USUARIOS … Y A LOS CREADORES

> [Voyant] empodera a las personas. Les permite probar el análisis y la visualización de textos, incluidas cosas como el modelado de tópicos y el análisis de componentes principales. Les permite probarlo, pero hay algo más. Tenemos un montón de herramientas estrafalarias, y las herramientas estrafalarias transmiten un mensaje interesante: se puede jugar. Empodera a las personas que tienen que enseñar en las humanidades digitales. (Geoffrey Rockwell, en Jacomy, 2023)

El empoderamiento que brindan los instrumentos científicos no necesariamente favorece la «buena» ciencia, como han señalado los críticos (Drucker, 2014; Masson, 2017; Tenen, 2016); y esto independientemente de a qué pueda referirse «buena», ya que sencillamente no hay manera de contener el uso de una herramienta dentro de un conjunto predeterminado de prácticas. Cuando toma una herramienta, contribuye a moldear su función y puede ser considerado responsable de ella. Ni siquiera el creador de herramientas tiene un control completo sobre lo que la herramienta permite y no permite. Un desarrollador puede orientar sutilmente a sus usuarios, pero a menudo no puede dictar los usos imaginados o efectivos. En qué consiste el empoderamiento se descubre en parte mediante el uso efectivo por parte de distintas personas y nunca se conoce del todo de antemano. La dirección que toma el ensamblaje herramienta-usuario, es decir, el investigador equipado, surge de un diálogo material entre la perspectiva parcial del investigador y las teorías infraestructuradas en el instrumento. La posibilidad de readaptar una herramienta está en la naturaleza misma del empoderamiento que aporta.

Quizás de manera menos evidente, la herramienta también empodera al creador de herramientas. No sólo porque una herramienta exitosa puede dar poder a su(s) creador(es), sino también, y sobre todo, porque el proceso de crear la herramienta es científicamente fecundo. Además, estos empoderamientos no son distintos, sino que están entrelazados, porque, en muchos sentidos, la herramienta ofrece al usuario final la oportunidad de re-trazar las trazas dejadas por el creador de herramientas durante la creación de la herramienta. Rieder et al. (2024) señalan precisamente esto:

> En un primer nivel, … el software de investigación entra en conversación con el conocimiento y la metodología existentes … [y] depende en gran medida del trabajo y la dedicación que se invierten en crear los miles de componentes básicos que orientan el desarrollo a cada paso. … [En un] segundo nivel, … el proceso de sumergirse en las especificaciones, de toparse con restricciones técnicas y de descubrir "trucos" y resquicios [es] epistémicamente productivo, en el sentido de que puede conducir a nuevas vías que se conviertan en funcionalidades o métodos específicos más adelante. … En un tercer nivel, entran en juego los usuarios finales. … Usuario y autor contribuyen ambos a la evolución de una herramienta de investigación; pero también podría decirse que la herramienta desarrolla al usuario, al presentar y enmarcar enfoques particulares de la investigación con los que quizá aún no esté familiarizado. … En conjunto, estos tres entrelazamientos crean un espacio de acción para los creadores de herramientas que está delimitado y "poblado" por otros actores.

He aquí un ejemplo tomado de la creación de Gephi (Bastian et al., 2009). La herramienta propone al usuario una serie de «algoritmos de disposición espacial», destinados a ordenar los nodos de la red en el plano 2D de la pantalla según cómo están conectados (Venturini et al., 2021). Los algoritmos disponibles al comienzo de la vida de Gephi requerían un saber hacer acerca de qué algoritmo elegir en una situación dada y cómo configurar sus parámetros. Esto, entre otras razones, llevó a los creadores de herramientas a desarrollar su propio algoritmo, ForceAtlas2 (Jacomy et al., 2014). En el proceso experimental de dar con una buena receta, redescubrieron una manera eficiente de equilibrar ese tipo de algoritmo, observada previamente por Andreas Noack (2007) con el nombre de «repulsión de aristas». En pocas palabras, implica

una forma especial de tener en cuenta las aristas en una de las ecuaciones del algoritmo que parece producir una mejor separación visual de los clústeres. Esto los llevó a replantearse si las cualidades estéticas de algoritmos populares como ForceAtlas2 y el LinLog (Noack, 2007) se deben en realidad al «modelo de energía» (ibid), es decir, al perfil general de la ecuación del algoritmo, como se acepta generalmente, o a la propia repulsión de aristas (Jacomy, 2021; Venturini et al., 2021). Crear Gephi fue, para sus creadores de herramientas, una puerta de entrada a una discusión académica sobre el dibujo de grafos y el agrupamiento.

El empoderamiento del creador de herramientas es una externalidad importante de la creación de herramientas, tanto para la comunidad académica como para el propio creador de herramientas. La comunidad académica puede beneficiarse de la emergencia de teorías (infraestructuradas) que quizá no saldrían a la luz por otros medios. Esto aumenta la diversidad epistémica de un campo. También es importante para el propio creador de herramientas, porque el conocimiento científico que obtiene de ello (entre otros beneficios) contribuye a consolidar su identidad como actor de la investigación académica, lo cual es necesario para mantener su estatus de creador científico de herramientas.

> [Voyant] también nos empoderó a Stéfan y a mí para construir una carrera en torno a esto, de una manera que habría sido muy difícil de lograr si no lo hubiéramos hecho. Uno mira hacia atrás en su vida y se dice: podría haber tomado direcciones muy distintas. (Geoffrey Rockwell, en Jacomy, 2023)

LAS HERRAMIENTAS SON PORTADORAS DE TEORÍA

> La tesis de nuestro libro, Hermeneutica, es que, en cierto sentido, las herramientas son portadoras de teoría de un modo diferente al de los textos. Un texto hermenéutico dice: así es como se interpreta un texto, así es como se hace la interpretación. Una herramienta es portadora de una teoría, pero la porta de otro modo. Estábamos jugando con esta idea de que "las herramientas pueden ser teorías", y acto seguido empezamos a decírselo a la gente. Nuestros artículos le decían a la gente que las herramientas son teorías. (Geoffrey Rockwell, en Jacomy, 2023)

La creación de herramientas forma parte de la «vida social de los métodos» (Savage, 2013), inserta en prácticas y relaciones sociales, influida por dinámicas de poder y valores culturales, pero también influyendo en ellos; y no sólo cuando se trata de las grandes herramientas célebres. «Todo CSV es una expresión de poder, toda API es un argumento» (Sherratt en Endres, 2017). El software (de investigación) no se limita a proporcionar funcionalidades técnicas, sino que codifica consideraciones metodológicas y epistemológicas, del mismo modo que los datos utilizados en los sistemas computacionales tienen repercusiones epistemológicas para la investigación (Borra, 2023). Puede, por ejemplo, influir en cómo se seleccionan y operacionalizan las variables, qué análisis pueden llevarse a cabo y cómo se interpretan y presentan los resultados (Uprichard et al., 2008). Una herramienta perfectamente ateórica sería inútil e impotente, porque no podría marcar ninguna diferencia en nada, ya que eso supondría comprometerse con un método frente a otras opciones. De ahí surge el argumento de que las

herramientas son portadoras de teoría. Esto no equivale a afirmar lo que la teoría debería ser. Más bien, constata un hecho básico que debemos aceptar, pues de lo contrario no podríamos analizar adecuadamente cómo los instrumentos científicos afectan la producción de conocimiento. Las herramientas tienen orientaciones epistémicas que afectan la manera en que se lleva a cabo la investigación y el tipo de conocimiento que se produce (Borra & Rieder, 2014). Cuando usamos herramientas de investigación, aceptamos implícitamente ciertas visiones teóricas sobre cómo funciona el mundo.

Podría decirse simplemente que las herramientas tienen agencia científica, la capacidad de moldear los procesos y los resultados de la investigación; o, más precisamente, que el ensamblaje usuario-herramienta puede desplazar los objetivos tanto del usuario como de la herramienta (Latour, 1994). Sin embargo, enmarcar la influencia de las herramientas meramente como «agencia» corre el riesgo de reducirla a un simple impacto en la investigación. Esta perspectiva no logra captar el punto crucial de que, para los creadores de herramientas, la teoría no se aplica simplemente a través de la creación de herramientas, sino que se constituye en el proceso mismo de creación de herramientas. El contenido teórico, por lo tanto, está inextricablemente vinculado al acto práctico del diseño y el desarrollo. En ese sentido, la «teoría» pasa a ser menos un marco preexistente y más una propiedad emergente de la propia práctica de creación de herramientas.

Estamos acostumbrados a encontrar la teoría en forma de texto, como una manera estructurada de explicar fenómenos, que involucra conceptos bien definidos y sus relaciones, como enunciados causales que especifican mecanismos acerca de cómo y por qué ocurren ciertas cosas, quizá con elementos interpretativos. Pero dentro de la herramienta, la teoría tiene un modo de existencia diferente del texto; llamémosla infraestructurada para subrayar que hay una diferencia material (siguiendo ideas similares de Gray et al., 2018). La materialidad de la herramienta trae consigo pérdidas y ganancias respecto de la materialidad del texto. Pierde las propiedades discursivas del texto: las teorías infraestructuradas (teorías incorporadas en sistemas tecnológicos) son menos explícitas, menos discutibles. La carga teórica de una herramienta requiere un trabajo específico para ser criticada, o un enfoque de la crítica diferente, no textual; o bien podría pasar inadvertida y escapar por completo a la crítica. A cambio de esta pérdida, las teorías infraestructuradas ganan affordances (Gibson, 1966), es decir, maneras materiales de guiar al usuario sin requerir ninguna explicación o justificación a priori. En pocas palabras, las teorías infraestructuradas no se leen, sino que se ponen en acto y se experimentan. Muchos pueden convencerse mejor experimentando que leyendo y, al mismo tiempo, el elemento subjetivo de la experiencia puede dificultar la transmisión de ese conocimiento, además de la necesidad de poner la teoría en palabras. Usar una herramienta puede convencerlo de cosas que tal vez no sea capaz de formular; puede transmitirle patrones que no es plenamente consciente de haber interiorizado, información no verbal que solemos etiquetar como saber hacer, como pericia. Esta información constituye, no obstante, conocimiento y es teórica en la medida en que da vigencia a algunas explicaciones en detrimento de otras. «En el contexto de la historia o de

los estudios literarios, la "teoría" no predice, pero sí explica. … Decir que el software es una teoría es decir que las obras digitales transmiten conocimiento como lo hace una teoría, en este sentido más general» (Ramsay & Rockwell, 2012).

El creador de herramientas es muy consciente de que las affordances de su herramienta facilitan algunas agendas de investigación y dificultan otras, en particular cuando moldea la herramienta para una situación dada (su propio uso, un data sprint, un proyecto …). El creador de herramientas se ve inevitablemente arrastrado hacia una posición en cierto modo normativa, le guste o no. Por lo general, moldeará las affordances para mejorar la herramienta, por ejemplo, para imponer mejores prácticas de investigación (Peeters & Hagen, 2022), pero lo que considera bueno podría no funcionar para otra persona, razón por la cual uno siempre debería preguntarse cómo responde la herramienta a sus propias necesidades. Incluso decisiones de diseño aparentemente menores —como las estructuras de las bases de datos, los parámetros por defecto o el orden en que se presentan las opciones— pueden tener efectos significativos en la investigación (Borra & Rieder, 2014). Tales elecciones influyen en la manera en que los investigadores interactúan con los datos y moldean las conclusiones que extraen (Borra, 2023). Como la herramienta será inevitablemente normativa en alguna medida y el creador de herramientas suele ser consciente de ello, rara vez presentará su herramienta como agnóstica. Puede incluso afirmar sin complejos la dirección metodológica que considera mejor mediante «gestos prescriptivos» (Rieder et al., 2024) y mitigar la responsabilidad correspondiente informando de ello al usuario. Alternativamente, algunas herramientas se resisten a tomar decisiones ofreciendo tantas opciones como sea posible, por ejemplo SciKit-Learn (Pedregosa et al., 2011). Como escriben Rieder et al. (2024), «si bien términos como "sesgo" todavía sugieren que la neutralidad epistémica es una posibilidad o, al menos, un desiderátum, sostenemos que es a la vez inevitable y productivo que los compromisos, las idiosincrasias y las inserciones contextuales moldeen la creación de herramientas y, en consecuencia, las propias herramientas. Inevitable porque toda actividad de diseño implica una plétora de elecciones, y productivo porque la creación de herramientas es un espacio de expresión y negociación que puede estimular procesos colectivos de debate metodológico y construcción de sentido».

La capacidad de los instrumentos científicos de ser portadores de teoría suele reconocerse cuando se trata de criticarlos. Esa literatura dentro de las DCSSH presenta recurrentemente las herramientas como caballos de Troya, generalmente del positivismo. Por ejemplo, Eef Masson (2017) escribe: «con la introducción de herramientas digitales de investigación, y de herramientas para la investigación con datos específicamente, los estudios humanísticos parecen estar cada vez más en deuda con las tradiciones positivistas … esas herramientas, la mayoría de las veces, se toman prestadas de disciplinas centradas en el análisis de datos empíricos, por lo general cuantitativos. Inevitablemente, entonces, incorporan las tradiciones epistémicas de las que derivan». O Johanna Drucker (2014): «La mayoría, si no todas, de las visualizaciones adoptadas por los humanistas, como la cartografía SIG, los gráficos y los diagramas, se desarrollaron en otras disciplinas. Estas herramientas gráficas son una especie de caballo de Troya intelectual, un

vehículo a través del cual los supuestos acerca de lo que constituye información irrumpen en tropel con una fuerza poderosa». Nosotros discrepamos de esa perspectiva. Si bien la metáfora del caballo de Troya da cuenta adecuadamente de la capacidad de una herramienta para ofuscar su carga teórica, refutamos que esa carga haya sido ocultada y transmitida intencionalmente por el creador de herramientas con el propósito de influir y engañar. Desarrollamos este argumento de distintas maneras a lo largo de este texto, pero, en pocas palabras, aunque la perspectiva del creador de herramientas es necesariamente parcial, este no tiene pleno control sobre qué teoría termina incorporada en su herramienta. Cuando quiere operacionalizar una teoría, no funciona; y, a la inversa, termina implementando teorías de las que ni siquiera es consciente. Esto último fue lo que ocurrió con el ya mencionado Lippmannian Device, que se originó a partir del uso particular de un scraper de Google que, reconfigurado y cargado de teoría, se convirtió en un medio para mostrar el partidismo de los actores y su compromiso con determinados temas (Rogers, 2010). Rieder et al. (2024) abogan así por una «crítica de las herramientas» no sólo para desvelar supuestos epistemológicos ocultos, sino también para tomar decisiones de diseño informadas y mejorar la transparencia de las herramientas de investigación. La crítica de las herramientas no es meramente reactiva; incide activamente en cómo se diseñan, documentan y comunican las herramientas.

Por lo tanto, es necesario pero no suficiente reconocer que las herramientas tienen una carga teórica o «compromisos epistemológicos» (Rieder & Röhle, 2017). La teoría infraestructurada sigue siendo «ominosa», como señalan Marres y Gerlitz (2016): «Una característica clave de lo ominoso metodológico es que no está necesariamente claro a qué propósitos analíticos pueden servir las herramientas digitales, con qué objetivos de investigación pueden alinearse o qué agendas disciplinarias ponen en acto» (ibid, p. 26). En qué consiste la teoría infraestructurada nunca debería darse por sentado. Incluso cuando el creador de herramientas ha documentado las teorías y los métodos en juego en su herramienta (una gran decisión, dicho sea de paso), no hay que tomarlo al pie de la letra, esencialmente porque la orientación de la herramienta puede pasar inadvertida para sus propios creadores. Esto es motivo de preocupación en lo que respecta a la crítica, pero también una oportunidad para readaptar la tecnología de maneras que beneficien al investigador, lo cual ha sido decisivo para algunas disciplinas como los estudios de medios (digitales). En efecto, «gran parte del debate sobre los métodos digitales en los estudios de redes sociales se ha centrado en la posibilidad de readaptar los dispositivos digitales (Rogers, 2009). Los sociólogos han llamado la atención sobre la inestabilidad y la subdeterminación de los propios métodos digitales de investigación, y han propuesto nociones como métodos plásticos (Lury, 2012) y métodos vivos (Back and Puwar, 2012)» (Marres & Gerlitz, 2016). Las teorías infraestructuradas no se reducen a una influencia oculta, ya que su polivalencia epistémica también le ofrece a usted, el usuario académico, oportunidades para doblegarlas y remodelarlas.

Desde el punto de vista del creador, la práctica de la creación de herramientas apunta a operacionalizar teorías y, al mismo tiempo, a reconocer la libertad del usuario para usar la

herramienta de maneras que desafíen las teorías supuestamente incorporadas en ella. Si una herramienta es portadora de teoría, entonces debemos aceptar que esa teoría bien podría ser descubierta o reinventada (según cómo lo vea) por otros investigadores. Los instrumentos dan forma a la ciencia, aunque de maneras que nadie puede dar por sentadas (incluidos los creadores de herramientas), algo que siempre debería tenerse en cuenta. Parafraseando a uno de nosotros (Borra, 2023):

> [La creación de herramientas puede] considerarse "una contribución a la producción de conocimiento académico" (Rieder et al., 2024, párr. 54). Programar instrumentos de investigación es más que una simple manera de delimitar y reutilizar flujos de datos y actividades en sistemas sociotécnicos; también intenta operacionalizar una técnica de investigación determinada y argumentar a su favor. La combinación de inquietudes teóricamente fundamentadas y construcción de herramientas constituye un género académico que conecta los estudios críticos con la práctica técnica (Agre, 1997) y es una parte vital de la producción de conocimiento, aunque no siempre se la reconozca como tal (Jay et al., 2021; Katz & Stall, 2021). Construir herramientas permite involucrarse con la manera en que los datos se generan, se "infraestructuran" y se usan en la práctica (Gray et al., 2018)

Las teorías estructuradas en una herramienta durante su creación adoptan, por naturaleza, un giro pragmático. Como escribió William James (1907), «las teorías se convierten así en instrumentos, no en respuestas a enigmas en las que podamos descansar». La teoría estructurada remite entonces a los compromisos metodológicos y epistemológicos incorporados en las herramientas, que dan forma a la investigación a través de sus affordances y no mediante una argumentación explícita. Funcionan como medios para dar sentido al mundo, no como explicaciones definitivas sino como vectores de epifanías momentáneas, precisamente porque no consisten en enunciados textuales estables y, por lo tanto, no pueden convertirse en lugares de descanso para nuestro pensamiento.

REFERENCIAS

Agre, P. E. (1997). Toward a critical technical practice: Lessons learned in trying to reform AI. In G. Bowker, S. L. Star, L. Gasser, & W. Turner (Eds.), Social Science, Technical Systems, and Cooperative Work: Beyond the Great Divide (pp. 131–157). Psychology Press.

Back, L., & Puwar, N. (2012). A manifesto for live methods: Provocations and capacities. The Sociological Review, 60(1_suppl), 6–17.

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. Proceedings of the International Aaai Conference on Web and Social Media, 3(1), 361–362.

Borra, E. (2023). Object-activity: Repurposing the dual nature of web data for digital research. [PhD Thesis].

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. Aslib Journal of Information Management, 66(3), 262–278.

Cherven, K. (2015). Mastering Gephi Network Visualization. Packt Publishing Ltd.

Drucker, J. (2014). Graphesis: Visual Forms of Knowledge Production. Harvard University Press.

Endres, B. (2017). A literacy of building: Making in the digital humanities. In J. Sayers (Ed.), Making Things and Drawing Boundaries: Experiments in the Digital Humanities (pp. 44–54). University of Minnesota Press.

Eyman, D., & Ball, C. (2015). Digital humanities scholarship and electronic publication. In J. Ridolfo & W. Hart-Davidson (Eds.), Rhetoric and the Digital Humanities (pp. 65–79). University of Chicago Press.

Galey, A., & Ruecker, S. (2010). How a prototype argues. Literary and Linguistic Computing, 25(4), 405–424. https://doi.org/10.1093/llc/fqq021

Gibson, J. J. (1966). The Senses Considered as Perceptual Systems. Allen and Unwin.

Gray, J., Gerlitz, C., & Bounegru, L. (2018). Data infrastructure literacy. Big Data & Society, 5(2). https://doi.org/10.1177/2053951718786316

Jacomy, M. (2021). Situating visual network analysis. [PhD thesis].

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. Reticular. https://reticular.hypotheses.org/8039

Jacomy, M., & Jokubauskaitė, E. (2022). Unblackboxing Gephi: How a user culture shapes its scientific instrument. Working paper. https://hal.science/hal-03729114/file/Unblackboxing%20Gephi%20Rev%201.pdf (accessed 26/08/2024).

Jacomy, M., & Munk, A. K. (2024). Interfering with the black-box-tradeoff model: Gephisto, a one-click Gephi for critical technical practice. Convergence, 30(1), 142–166. https://doi.org/10.1177/13548565221129053

Jacomy, M., Venturini, T., Heymann, S., & Bastian, M. (2014). ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. PloS One, 9(6), e98679.

James, W. (1907). Lecture II: What pragmatism means. In W. James (Ed.), Pragmatism: A new name for some old ways of thinking (pp. 43–81). Longmans, Green and Co. https://doi.org/10.1037/10851-002

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. International Journal of Digital Curation, 16(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Katz, D. S., & Stall, S. (2021). Software citation. Information Services & Use, 41(1–2), 39–42. https://doi.org/10.3233/ISU-210108

Khokhar, D. (2015). Gephi Cookbook. Packt Publishing.

Knorr Cetina, K. (1999). Epistemic Cultures: How the Sciences Make Knowledge. Harvard University Press.

Latour, B. (1994). On technical mediation. Common Knowledge, 3(2), 29–64.

Lippmann, W. (1922). Public Opinion. MacMillan. https://doi.org/10.1037/14847-000

Lury, C. (2012). Going live: Towards an amphibious sociology. The Sociological Review, 60(1_suppl), 184–197.

Marres, N., & Gerlitz, C. (2016). Interface methods: Renegotiating relations between digital social research, STS and sociology. The Sociological Review, 64(1), 21–46.

Maslow, A. H. (1966). The Psychology of Science. Harper & Row.

Masson, E. (2017). Humanistic data research: An encounter between epistemic traditions. In M. T. Schäfer and K. van Es (Eds.), The Datafied Society: Studying Culture through Data (pp. 25–38). Amsterdam University Press.

McCarty, W. (2004). Modeling: A study in words and meanings. In S. Schreibman, R. Siemens, & J. Unsworth (Eds.), A Companion to Digital Humanities (pp. 254–270). Blackwell Publishing.

McCarty, W. (2012). A telescope for the mind? In M. K. Gold (Ed.), Debates in the Digital Humanities (pp. 113–123). University of Minnesota Press.

Noack, A. (2007). Energy models for graph clustering. Journal of Graph Algorithms and Applications, 11(2), 453–480.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. Computational Communication Research, 4(2), 571–589.

Peixoto, T. (2014). Graph-tool: Efficient network analysis with Python. https://graph-tool.skewed.de(accessed 14/08/2024).

Peixoto, T. P. (2023). Descriptive vs. Inferential Community Detection in Networks: Pitfalls, Myths and Half-truths. Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Prescott, A. (2014). Made in Sheffield: Industrial perspectives on the digital humanities. Studies in the Digital Humanities, (1).

Ramsay, S., & Rockwell, G. (2012). Developing things: Notes toward an epistemology of building in the digital humanities. In M. K. Gold (Ed.), Debates in the Digital Humanities (pp. 75–84). University of Minnesota Press.

Ratinaud, P. (2009). IRaMuTeQ: Interface de R pour les Analyses Multidimensionnelles de TExtes et de Questionnaires. http://www.iramuteq.org (accessed 14/08/2024).

Ratinaud, P., & Marchand, P. (2012). Application de la méthode ALCESTE à de "gros" corpus et stabilité des "mondes lexicaux": analyse du "CableGate" avec IRaMuTeQ. Actes des 11eme Journées internationales d'Analyse statistique des Données Textuelles, 3, 835–844.

Reinert, M. (1990). Alceste une méthodologie d'analyse des données textuelles et une application: Aurelia De Gerard De Nerval. Bulletin of Sociological Methodology/Bulletin de méthodologie sociologique, 26(1), 24–54.

Rieder, B. (2013). Studying Facebook via data extraction: The Netvizz application. In Proceedings of the 5th annual ACM Web Science Conference (pp. 346–355). Association for Computing Machinery.

Rieder, B., & Röhle, T. (2017). Digital methods: From challenges to bildung. In M. T. Schäfer & K. van Es (Eds.), The Datafied Society: Studying Culture Through Data (pp. 109–124). Amsterdam University Press.

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. Convergence, 30(1), 216–235. https://doi.org/10.1177/13548565221127094

Rockwell, G. (2001). The visual concordance: The design of Eye-ConTact. Text Technology, 10(1), 73–86.

Rockwell, G., & Sinclair, S. (2016). A model theory: Thinking through hermeneutical things. In G. Rockwell & S. Sinclair (Eds.), Hermeneutica: Computer-Assisted Interpretation in the Humanities (pp. 149–166). MIT Press.

Rogers, R. (2009). The End of the Virtual: Digital Methods (Vol. 339). Amsterdam University Press.

Rogers, R. (2010). Internet research: The question of method—a keynote address from the YouTube and the 2008 election cycle in the United States Conference. Journal of Information Technology & Politics, 7(2–3), 241–260. https://doi.org/10.1080/19331681003753438

Savage, M. (2013). The 'Social life of methods': A critical introduction. Theory, Culture & Society, 30(4), 3–21.

Schreibman, S., & Hanlon, A. M. (2010). Determining value for digital humanities tools: Report on a survey of tool developers. Digital Humanities Quarterly, 4(2). https://www.proquest.com/docview/2555208512/abstract/8B9E2E7ABEE94C7EPQ/1 (accessed 01/09/2024).

Sherratt, T. (2015). Unremembering the forgotten. Keynote Address, International Conference of the Alliance of Digital Humanities Organisations.

Tenen, D. (2016). Blunt instrumentalism: On tools and methods. Debates in the Digital Humanities, 216, 83–91.

Uprichard, E., Burrows, R., & Byrne, D. (2008). SPSS as an 'Inscription Device': From causality to description? The Sociological Review, 56(4), 606–622. https://doi.org/10.1111/j.1467-954X.2008.00807.x

Venturini, T., Jacomy, M., & Jensen, P. (2021). What do we see when we look at networks: Visual network analysis, relational ambiguity, and force-directed layouts. Big Data & Society, 8(1).

21. Conozca a los creadores de herramientas de las DCSSH

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Los creadores de herramientas DC para las SSH suelen trabajar en contextos institucionales difíciles, a caballo entre funciones de investigación académica y de soporte técnico —algo que rara vez forma parte de la descripción de su puesto. Conocer esta situación es valioso para los investigadores porque les ayuda a comprender mejor por qué existen ciertas herramientas y otras no, por qué algunas herramientas reciben un mantenimiento constante mientras otras languidecen, y cómo relacionarse de manera más eficaz con los creadores de herramientas cuando buscan soporte o solicitan funcionalidades.

Este capítulo examina la organización institucional de las universidades, que suele imponer una separación estricta entre el personal de apoyo y los investigadores académicos, lo que genera tensiones que afectan el desarrollo y el mantenimiento de las herramientas. Esto explica por qué encontrar y mantener herramientas de investigación adecuadas puede resultar difícil en las DCSSH, donde las herramientas especializadas no siempre pueden tomarse prestadas de otras disciplinas o de la industria.

Comprender los contextos profesionales y los desafíos de los creadores de herramientas de las DCSSH permite a los investigadores orientarse mejor en el panorama de las herramientas: saber cómo comunicar eficazmente sus necesidades a través de los canales apropiados, reconocer el valor de construir relaciones con la comunidad de creadores de herramientas y apreciar las complejas soluciones de compromiso que dan forma al desarrollo de herramientas.

¿QUIÉN CREA HERRAMIENTAS PARA LAS DCSSH?

> Mi nombre es Geoffrey Rockwell. Soy profesor de Filosofía y Humanidades Digitales en la University of Alberta. … Una de las cosas que me encanta hacer es enseñar a programar a estudiantes de humanidades. … Ojalá tuviera tiempo para ser mejor programador. (Geoffrey Rockwell, en Jacomy, 2023)

Los instrumentos que los investigadores de las SSH compran en el mercado son construidos y mantenidos por profesionales contratados; en cambio, las herramientas de investigación gratuitas provienen de otro lugar y tienen otra existencia. No se puede suponer sin más que los creadores de herramientas de las DCSSH se ganan la vida creando herramientas. Si observa quiénes son, encontrará ingenieros de investigación que se ganan la vida manteniendo infraestructuras —aunque eso no implica necesariamente el diseño de herramientas—; académicos que crean herramientas como subproducto de su investigación —aunque crear y mantener herramientas queda fuera de la misión de su puesto—; y muchas situaciones

intermedias en las que el creador de herramientas es un científico-programador (Perkel, 2021) con un puesto y/o una misión inusuales, o con un acuerdo extraoficial con sus superiores acerca de su actividad.

La creación de herramientas en las DCSSH conlleva cierta crisis de identidad porque choca con la organización institucional de la ciencia, en particular en el nivel universitario (menos en el nivel de un grupo de investigación particular, como veremos). En efecto, las universidades suelen imponer una separación estricta entre dos tipos de empleados: el personal de apoyo (servicios informáticos, contadores, ingenieros de investigación…) y el personal académico (investigadores, profesores…) (véase la figura 21.1). Reconozcamos que hay sólidos argumentos a favor de esta distinción, en particular como medio para proteger la capacidad de los investigadores de determinar por sí mismos su agenda científica y, al mismo tiempo, conservar dentro de la universidad las competencias técnicas necesarias. Las declaraciones de misión, los indicadores clave de desempeño y los salarios son sensiblemente diferentes a cada lado de esta frontera. La diferencia entre un ingeniero de software de investigación (RSE) y un profesor, por ejemplo, que podrían ambos estar creando herramientas, presupone una división del trabajo entre lo material y lo abstracto, programar y escribir, tecnología y metodología, etc. Lamentablemente, estas dicotomías entran en conflicto con la práctica de la creación de herramientas porque, en pocas palabras, el principio central del diseño de instrumentos científicos es que tecnología y metodología son dos caras de la misma moneda.

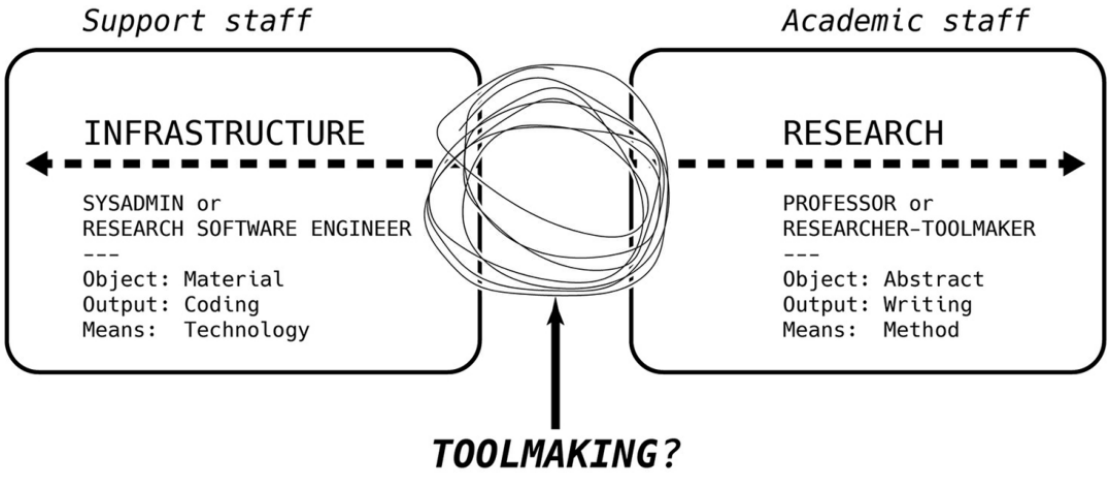


Figura 21.1 La dicotomía infraestructura-investigación tal como la ven las instituciones científicas: como una brecha que sustenta una división del trabajo entre, por ejemplo, el ingeniero de software de investigación y el profesor
Fuente: elaboración propia.

En contra de esta perspectiva, las prácticas de quienes crean herramientas para las DCSSH se distribuyen en un espectro que va de la infraestructura a la investigación. Contrariamente a lo que sugiere la división del trabajo en las universidades, existe un continuo de perfiles de creadores de herramientas entre los puestos de apoyo y los académicos (figura 20.1 del capítulo 20). En el extremo de la infraestructura, los creadores de herramientas se consideran

funcionarios públicos y mantienen el aparato técnico necesario para hacer investigación, pero no desean escribir publicaciones académicas (aunque a menudo terminan haciéndolo en alguna medida). Baxter et al. (2012) acuñaron el término «Research Software Engineer» para identificar ese perfil, alguien que «disfruta [del] desafío no sólo de desarrollar código para resolver un problema, sino de hacerlo bien» y «quier[e] ser reconocido por producir herramientas en las que otros se apoyan para investigar» (ibid, p. 2). En el extremo de la investigación, encontramos académicos que ven la creación de herramientas ya sea como una práctica autoetnográfica y crítica necesaria para su investigación, como una manera de producir demostraciones experimentales y/o como una forma de producción de conocimiento en sí misma. En suma, es parte integral de su propia investigación (mientras que el RSE apoya la investigación de otros). Encuentran maneras de valorizar académicamente sus herramientas, ya sea mediante artículos sustitutos (destinados a ser citados en lugar de una herramienta determinada), libros, praxis (actividades reflexivas que integran teoría y práctica) o haciendo citable la propia herramienta, por ejemplo con un DOI (aunque la falta de DOI no debería impedir citarla; véase Boettiger, 2013). Baxter et al. llaman a este perfil «Researcher-Developer», alguien que «quiere ser juzgado por su producción científica, es un investigador de corazón; sin embargo, desarrolla mucho código debido a la naturaleza de su investigación» (ibid). Fuera de este espectro, los creadores de herramientas pueden provenir del sector privado o de la sociedad civil y apuntar a equipar a otros públicos (ciudadanos; expertos; redacciones; activistas …) y, de manera incidental, ocurre que los investigadores de las DCSSH reutilizan y readaptan sus herramientas.

LA CREACIÓN ACADÉMICA DE HERRAMIENTAS NO ES (TODAVÍA) UNA TRAYECTORIA PROFESIONAL

La creación académica de herramientas carece de una trayectoria profesional clara porque entra en conflicto con la división entre apoyo y ciencia de las instituciones académicas (figura 21.1). Baxter et al. (2012) señalan este problema como una de sus principales razones para proponer la figura del RSE. Inventaron ese nombre en su artículo para llamar la atención sobre el trabajo oculto de los ingenieros de software en la academia, para respaldar mejor la mantenibilidad de la computación científica y para contribuir a abordar otros problemas similares. Como explican, «esta ausencia de una trayectoria profesional se debe a la falta de reconocimiento y valoración de los resultados del trabajo de desarrollo de software en el sistema académico de evaluación, que mide el éxito únicamente a través de publicaciones revisadas por pares ya publicadas. El Science Code Manifesto [(Barnes et al., 2013)], entre cuyos firmantes fundadores figuran Peter Norvig [destacado informático y entonces director de investigación de Google], Joseph Jackson [en representación de la Open Science Alliance] y Victoria Stodden [estadística conocida especialmente por su trabajo sobre la reproducibilidad científica], lo plantea explícitamente: "El software es un producto de investigación esencial, y el esfuerzo por producir, mantener, adaptar y curar código debe ser reconocido. El software se encuentra entre otras contribuciones científicas vitales además de los artículos publicados"». Los debates sobre los problemas de

producir y mantener código en la ciencia no son nuevos, y la falta de una trayectoria profesional bien definida suele considerarse el problema central.

La falta de una trayectoria profesional para los creadores de herramientas pone en riesgo la calidad de las herramientas disponibles para las DCSSH, en particular de las herramientas dedicadas a las necesidades específicas de las SSH, que no pueden tomarse prestadas de ningún otro campo ni de la industria. Las herramientas nacidas dentro de las DCSSH, que pueden ser las más valiosas para el campo, son creadas y mantenidas por empleados de apoyo o académicos que tienen una trayectoria profesional incierta y difícil como creadores de herramientas. La principal amenaza para su actividad de creación de herramientas se presenta de manera distinta a cada lado de la división, pero consiste igualmente en presiones para que dejen de hacer las cosas que corresponden al otro lado. Al RSE típicamente se le dirá que deje de mantener su propia herramienta y que, en cambio, apoye a investigadores acreditados; se le negará una agenda propia. En espejo con esta situación, se presionará al investigador para que deje de programar o de documentar su herramienta y, en cambio, enseñe y publique artículos revisados por pares; su producción tecnológica se considerará carente de valor. La relativa precariedad de las herramientas DC para las SSH tiene su raíz, en gran medida, en esta contradicción entre la práctica de la creación de herramientas y el paradigma profesional de las universidades, lo cual tiene al menos la ventaja de sugerir un camino para mejorar la calidad de nuestros instrumentos científicos: crear una trayectoria profesional identificada para los creadores de herramientas de las DCSSH.

Desde la publicación de Baxter et al. en 2012, los puestos de RSE se han normalizado cada vez más. Sin embargo, la división entre personal de apoyo y personal académico está tan profundamente arraigada en las universidades occidentales (al menos) que esta normalización ha desplazado el problema en lugar de resolverlo. Según nuestra experiencia, desde el punto de vista de un investigador de la Unión Europea en 2024, colaborar con RSE sigue siendo complicado porque suelen pagarse con financiamiento estructural, lo que implica mucha burocracia para determinar quién puede beneficiarse de su ayuda, con qué fin y durante cuánto tiempo. De manera contraproducente, esto puede a su vez llevar a desvincular el desarrollo de software de la investigación propiamente dicha.

En definitiva, la falta de un espacio profesional específico para el creador de herramientas es a la vez un síntoma de los múltiples problemas de la creación de herramientas en la academia y una fuente de problemas en sí misma. Deriva de los compartimentos estancos que separan los puestos de apoyo de los académicos; de la invisibilidad del mantenimiento del código (desarrollamos este punto en el capítulo 23); y del hecho de que la tecnología no se valora como producción académica en las SSH (véase el capítulo 24).

TENSIONES INHERENTES A LA CREACIÓN DE HERRAMIENTAS EN LAS DCSSH

> La primerísima herramienta que creé con John Bradley, TACTWeb, fue el comienzo del camino [de la creación de herramientas]. … La presentamos en ALLC/ACH París en 1994. Y, por si

> sirve de algo, todavía recuerdo a una persona muy importante de la comunidad que básicamente decía que esto no era humanidades digitales, que aquí no había un argumento, que esto [era] sólo una visualización bonita, y que era simpática pero inútil. … Fue una pequeña sorpresa para mí darme cuenta de repente de que mi propia comunidad estaba diciendo algo como: "¡Eh! Esto no es serio". Quiero decir, para ser justos, se trataba de una sola persona. (Geoffrey Rockwell, en Jacomy, 2023)

En la academia, el creador de herramientas se enfrenta a dos formas aparentemente antagónicas de control de acceso (gatekeeping). La primera cuestiona el carácter erudito o científico de la producción de software, mientras que la segunda postula que la herramienta tiene principios metodológicos indeseables.

Este último tipo de control de acceso se presenta en dos variantes. En algunos casos, la crítica simplemente se opone al método incorporado en la herramienta, por ejemplo al presentar un algoritmo implementado en Gephi como «perjudicial» (Peixoto, 2023), mientras que en otros casos culpa a la facilidad de uso de socavar las buenas prácticas metodológicas. Por ejemplo, Tenen (2016) sostiene que «cualquier intento de reempaquetar o simplificar aún más la herramienta sólo puede añadir otra capa de ofuscación. Se sigue, entonces, que sin una discusión crítica sobre los métodos implícitos, el uso de herramientas listas para usar debe tratarse preferentemente con cierta suspicacia». Esta crítica pinta al creador de herramientas (toolmaker) como un creador de necios (foolmaker), alguien que, mediante sus herramientas, habilita a los «perezosos» a hacer cosas que «no deberían», en detrimento del interés general de la comunidad académica.

Estas críticas son legítimas y saludables. El solo hecho de ser utilizada por investigadores no hace que una herramienta sea, en sí misma, académicamente fructífera y metodológicamente sólida. El creador de herramientas suele ser consciente de ello y trata de fundamentar su producción de software, típicamente publicándola de acuerdo con los principios FAIR para que sus metodologías puedan ser examinadas (Lamprecht et al., 2020), haciendo referencia a publicaciones pertinentes revisadas por pares en la documentación y en la interfaz de usuario, etc. Sus esfuerzos pueden considerarse suficientes o no; podemos suponer con seguridad que la mayoría de los creadores de herramientas asumen conscientemente la responsabilidad inherente a la autoría de una herramienta. Sin embargo, esta responsabilidad interfiere con una tensión que se ubica en un lugar completamente distinto.

La herramienta escapa inevitablemente, en cierta medida, a su creador. Permite cosas no previstas; los usuarios la readaptan de maneras inesperadas; tiene vida propia. El creador es sólo parcialmente responsable de lo que la gente hace con su herramienta —¿sigue siendo siquiera «su» herramienta una vez que otros se la reapropian? Cuanto más poderosa es la tecnología, mayor es el potencial de uso indebido. Además, como el espectro de la alfabetización digital es particularmente amplio, una herramienta de software determinada siempre resulta demasiado fácil para alguien y demasiado difícil para otro. Y, por la naturaleza de la comunidad académica, el método que incorpora es siempre el equivocado a los ojos de alguien. Aunque los creadores

de herramientas intenten orientar a los usuarios en la dirección «correcta» de la ciencia abierta o de la privacidad desde el diseño (Peeters & Hagen, 2022), no hay garantía de que la mayoría de los usuarios sigan las intenciones del diseño. Dado que en parte queda fuera del control del creador de herramientas, su responsabilidad está en tensión con la libertad del usuario.

La mayoría de los creadores de herramientas suelen tratar de atender las críticas, pero pueden desestimarlas cuando estas no tienen en cuenta la relativa independencia de la herramienta respecto de su creador. Algunos creadores de herramientas conocen íntimamente esta independencia, aunque resulta difícil dar cuenta de ella. En parte se debe a la necesidad constante de hacer concesiones en el entorno con limitaciones de tiempo y escasez de recursos de un equipo de investigación de las DCSSH, lo cual es fácil de entender. Pero, más importante aún, y menos fácil de interpretar, los datos y los propios algoritmos se resisten al creador de herramientas. Incluso en la mejor situación, el creador de herramientas no tiene otra opción que transigir con la materialidad de la computadora: este algoritmo es demasiado lento o consume demasiada memoria; aquella estructura de datos no admite todos los casos necesarios; esas técnicas algorítmicas aparentemente complementarias resultan ser irreconciliables; aquella técnica de visualización genera antipatrones que confunden a la mitad de sus usuarios; o, más prosaicamente, otra plataforma de redes sociales más cierra su API. El creador de herramientas a menudo hace lo que puede y no lo que desea, y parte de las críticas que recibe apuntan en realidad a cosas que están fuera de su control o que no puede permitirse en términos de tiempo o de recursos.

¿Quién asume la responsabilidad por la manera en que se usa la herramienta? El creador de herramientas también puede quedar expuesto a riesgos vinculados con aplicaciones imprevistas o no deseadas de su herramienta. Cuando la herramienta permite al usuario hacer algo ilegal o causar daño, ¿se responsabilizará al creador de herramientas? Por ejemplo, la empresa de redes sociales Meta (Facebook, Instagram) ha venido presentando demandas contra el web scraping (Hutchinson, 2024), al igual que X (ACLU, 2024). Muchos investigadores de las DCSSH que se apoyaban en web scrapers para su investigación pueden estar preocupados por ello, y su investigación puede ya verse afectada por esas demandas, independientemente de su resultado. ¿Quién afronta el riesgo, la herramienta o el investigador? ¿Qué clase de tecnología es la herramienta DC para las SSH? ¿Se parece más a un cuchillo, principalmente una mercancía aunque un arma en potencia, cuya agencia reside en la mano que lo empuña? ¿O es como una sustancia química altamente tóxica, principalmente una amenaza aunque útil en situaciones excepcionales, dañina por la sola virtud de su existencia? ¿Son algunas herramientas DC para las SSH tan peligrosas que deberían prohibirse, por razones legales, metodológicas o éticas? «¡Qué plaga han desatado sobre nosotros!», le dijo una vez Johanna Drucker (una destacada investigadora de las DH) a uno de nosotros (Jacomy) por haber cocreado la herramienta Gephi. La responsabilidad se distribuye entre el usuario, la herramienta misma y su(s) creador(es), y esa distribución nunca puede darse por sentada (Latour, 1994). Cuando el uso de una herramienta en la investigación implica un riesgo, múltiples actores pueden quedar expuestos: del

investigador al creador de herramientas, del financiador a la universidad. ¿Contará el creador de herramientas con el respaldo de su institución si alguien usa su herramienta para infringir los términos de servicio de una plataforma —en particular porque normalmente no está permitido hacer obras derivadas— o incluso las políticas o leyes de los países? Es el caso, por ejemplo, del Censorship Explorer del DMI. Debido a ese tipo de riesgo, la herramienta coloca a su creador en tensión con otros actores. Los creadores de herramientas pueden responder a ello de maneras muy diferentes: desde ignorarlo por completo, pasando por limitar el impacto tratando de mantener su herramienta relativamente confidencial, hasta mitigar el riesgo mismo, ya sea siguiendo pautas como los principios FAIR (Lamprecht et al., 2020) o simplemente absteniéndose de implementar ciertas funcionalidades.

LOS CREADORES DE HERRAMIENTAS OBTIENEN APOYO DE REDES COLABORATIVAS

> Al principio, daba soporte a aplicaciones de texto y procesamiento de textos, pero luego pasé a la tecnología educativa. … Tuve mucha suerte, ¿sabe?, dado que no tenía formación en informática ni formación técnica, de estar integrado en una unidad así. ¡Uno de esos grupos gestionaba la Internet académica de Canadá! … Era un grupo muy generativo. Tuve mucha suerte. (Geoffrey Rockwell, en Jacomy, 2023)

La mayoría de los creadores de herramientas corren el riesgo de quedar aislados, aunque sólo sea porque las necesidades del cuidado de la tecnología moldean su vida profesional de maneras inusuales. La colaboración con otros creadores de herramientas o tecnólogos es una necesidad, porque carecen de algunas de las competencias necesarias para gestionar toda la pila de software de su herramienta, pero también una fuente de alegría y realización, porque les ayuda a avanzar hacia los objetivos intelectuales que han elegido.

No es raro que los creadores de herramientas establezcan alianzas entre sí y con investigadores. Las herramientas no son simplemente productos; también son proyectos, y tienen demasiadas facetas como para que una sola persona las abarque cómodamente: desarrollo e implementación de algoritmos, mantenimiento técnico, administración de sistemas, diseño de la experiencia de usuario, gestión de la comunidad, recaudación de fondos, valorización académica … Muchos creadores de herramientas de las DCSSH deben encontrar la manera de ser lo bastante buenos en tareas que en la industria se consideran oficios completamente distintos. Esa situación es similar a la de un artesano. Por razones similares, los creadores de herramientas pueden desarrollar una forma de compañerismo. Construyen una red de socios a través de la cual pueden acceder a las competencias de las que carecen, encontrar mentoría, obtener apoyo emocional de otros creadores de herramientas y desarrollar su oficio. Esta red puede ser completamente invisible para los investigadores que los rodean (cuando trabajan en un laboratorio o grupo de investigación), porque sigue una infraestructura diferente, propia de la cultura de los creadores de herramientas (p. ej. GitHub, canales de IRC).

La inserción de algunos creadores de herramientas en ese compañerismo importa para las respuestas institucionales a la falta de una trayectoria profesional específica, pero también para las relaciones entre los creadores de herramientas y los investigadores que usan sus herramientas. En una situación en la que un investigador le plantea una demanda a un creador de herramientas, por ejemplo, corregir un error o agregar una funcionalidad, el éxito en obtener una respuesta depende en gran medida de cómo se le presente la demanda al creador de herramientas. Las demandas que emanan de la red de compañerismo suelen atenderse rápidamente, ya que el creador de herramientas se encuentra en una situación de dependencia mutua. Si quien hace la demanda goza de la confianza de otro creador de herramientas, este podría actuar como intermediario. De lo contrario, simplemente adoptar las normas de la comunidad será apreciado, registrando la demanda como un «issue» en GitHub (donde se aloja el código de la mayoría de los proyectos de herramientas de las DCSSH) en lugar de enviar un correo electrónico.

> Para Stéfan y para mí era muy importante que, año tras año, desarrolláramos una alianza. No era una alianza puntual. Fue artículo por artículo, herramienta por herramienta, retoque de interfaz por retoque de interfaz. No creo que el tipo de trabajo que hacíamos pueda hacerlo una sola persona. Yo podía ser un programador bastante bueno, pero Stéfan era mejor, y los dos juntos podíamos hacer más que por separado. Si yo hubiera tenido que hacerlo solo, o Stéfan hubiera tenido que hacerlo solo, no habríamos tenido tanta capacidad de respuesta. No habría existido el diálogo, la discusión, entre nosotros dos. Fue una bendición. Es simplemente una de las suertes de la vida, que él y yo estuviéramos juntos en la misma universidad en el momento justo, ambos con nuestras distintas trayectorias, construyendo cosas juntos. … Creo que hay que prestar mucha más atención a los elementos humanos del cuidado. ¿Cómo se disfruta trabajar en un proyecto con alguien a lo largo del tiempo? (Geoffrey Rockwell, en Jacomy, 2023)

REFERENCIAS

ACLU (2024). Federal judge dismisses Elon Musk's X lawsuit against nonprofit researchers [Press release]. Retrieved March 25, 2025, from https://www.aclu.org/press-releases/federal-judge-dismisses-elon-musks-x-lawsuit-against-nonprofit-researchers

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. Digital Research 2012 (pp. 1–3). The University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf

Barnes, N., Jones, D., Norvig, P., Neylon, C., Pollock, R., Jackson, J, Stodden, V., & Suber, P., (2013) Science Code Manifesto. http://web.archive.org/web/20130513204815/http://sciencecodemanifesto.org/ (archived 13/05/2013 and accessed 16/10/2025).

Boettiger, C. (2013).Doi != Citable. Lab Notebook, June 3. https://www.carlboettiger.info/2013/06/03/DOI-citable.html (accessed 19/06/2024).

Hutchinson, A. (2024). Meta abandons legal case over data scraping after losing key judgment. Social Media Today. https://www.socialmediatoday.com/news/meta-abandons-legal-case-data-scraping-losing-key-judgment/708538/ (accessed 13/08/2024).

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. Reticular. https://reticular.hypotheses.org/8039

Lamprecht, A.-L., Garcia, L., Kuzak, M., Martinez, C., Arcila, R., Martin Del Pico, E., Dominguez Del Angel, V., van de Sandt, S., Ison, J., Martinez, P. A., McQuilton, P., Valencia, A., Harrow, J., Psomopoulos, F.,

Gelpi, J. L., Chue Hong, N., Goble, C., & Capella-Gutierrez, S. (2020). Towards FAIR principles for research software. Data Science, 3(1), 37–59. https://doi.org/10.3233/DS-190026

Latour, B. (1994). On technical mediation. Common Knowledge, 3(2), 29–64.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. Computational Communication Research, 4(2), 571–589.

Peixoto, T. P. (2023). Descriptive vs. inferential community detection in networks: Pitfalls, myths and half-truths. Cambridge University Press.

Perkel, J. M. (2021). Ten computer codes that transformed science. Nature, 589(7842), 344–348. https://doi.org/10.1038/d41586-021-00075-2

Tenen, D. (2016). Blunt instrumentalism: On tools and methods. Debates in the digital humanities, 216, 83–91.

22. ¿Por qué los creadores de herramientas de las DCSSH crean herramientas?

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Sostenemos que los creadores de herramientas de las DCSSH intentan diseñar sus herramientas de modo que se mantengan alineadas con sus perspectivas intelectuales, porque las herramientas expresan su agenda científica o incluso la encarnan cuando el método se ha descubierto a través del proceso de creación de la herramienta.

La necesaria apropiación de la herramienta obliga a los creadores de herramientas de las DCSSH a priorizar intenciones de diseño específicas que reflejan las restricciones y los incentivos de su situación. En lugar de crear soluciones universales, elaboran instrumentos con consideraciones particulares en mente: hacer que una tecnología sea fácil de usar, favorecer el aprendizaje mediante la exploración, integrarse con otras herramientas existentes y seguir respondiendo a las necesidades de los usuarios sin dejar de resistir la proliferación de funcionalidades.

El capítulo explora en detalle estas intenciones de diseño y muestra cómo las perspectivas situadas específicas de los creadores de herramientas moldean el aparato de las DCSSH. Subrayamos que el proceso de diseño de herramientas no es unidireccional. Si bien los creadores de herramientas guían el desarrollo de las herramientas, los investigadores también pueden remodelarlas activamente mediante su readaptación, contribuyendo a las comunidades de usuarios, colaborando con los creadores de herramientas o incluso convirtiéndose ellos mismos en creadores de herramientas.

LAS HERRAMIENTAS DC PARA LAS SSH ENCARNAN LA AGENDA CIENTÍFICA DEL CREADOR DE HERRAMIENTAS

Todo instrumento científico media y moldea la producción de conocimiento de maneras específicas y, por lo tanto, posee su propia agencia científica. Esa agencia emana de la agenda científica del creador de herramientas. Desde la perspectiva de los creadores de herramientas, la creación de herramientas es un acto de difusión de su perspectiva científica. Esa perspectiva no necesita estar tan precisamente definida y formulada como en un artículo académico, pero a veces lo está. A menudo es puramente metodológica, pero a veces es más sustantiva. Las herramientas DC para las SSH son portadoras de teoría (infraestructurada), y sus creadores son conscientes de ello. Para ellos, la libertad de diseñar la herramienta que imaginan es lo mismo que la libertad académica para los investigadores: una manera de expresar su propia agenda científica.

Como sostuvimos en el capítulo 20 (figura 20.1), una infraestructura se convierte en herramienta cuando deja de ser un sistema invisible y empieza a tener una identidad y un propósito: cuando se convierte en un producto. Producto sólo significa que todos piensan y se comunican acerca de las herramientas nombrándolas, asociándolas con un conjunto de funcionalidades, un propósito y, en general, concibiéndolas como totalidades. Aquí debemos subrayar que, aunque el concepto de producto está fuertemente arraigado en la gestión industrial de proyectos, los instrumentos científicos provienen mayormente de otro lugar. Aunque ocasionalmente puedan convertirse en el núcleo de un negocio, siguiendo los principios FAIR, las herramientas DC para las SSH suelen desarrollarse como software de código abierto o como parte de una infraestructura de investigación más grande. El hecho de que las herramientas DC para las SSH sean productos crea la necesidad de recursos específicos (volveremos sobre este tema en el capítulo 24) y tiene otras consecuencias para los creadores de herramientas. Los productos están destinados a un determinado público; los productos son diseñados por una figura con autoridad (persona o grupo); y los productos son objeto de curaduría.

El creador de herramientas es, en última instancia, quien responde por la herramienta, con una excepción importante: las herramientas lo bastante grandes como para contar con el apoyo de un colectivo o una institución (es decir, con un público amplio, una identidad clara y un financiamiento relativamente estable) suelen diseñarse en comité. Esta es la norma en los grandes proyectos de software de código abierto, por ejemplo, OpenRefine (Ham, 2013), cuya hoja de ruta de desarrollo deciden múltiples actores de diferentes maneras, entre ellas un foro en línea, el liderazgo del principal responsable del mantenimiento y un comité científico. En este caso, el colectivo está claramente estructurado y su responsabilidad está distribuida. Sin embargo, incluso OpenRefine comenzó como «una herramienta de limpieza de datos de código abierto llamada Freebase Gridworks… , concebida por David Huynh» en 2008 (OpenRefine, 2024). Como la mayoría de las herramientas DC para las SSH son bastante pequeñas, tienen uno solo o apenas unos pocos creadores, que deben asumir toda la responsabilidad.

El creador de herramientas a menudo tiene la capacidad de crear la herramienta de otra persona, pero rara vez lo hará. Algunos de nosotros (los autores) hemos sido abordados en múltiples ocasiones por investigadores que tienen una idea de herramienta pero carecen de las competencias para materializarla; la mayoría de las veces, esos investigadores subestiman lo que implica crear una herramienta. Esos investigadores parecen creer que, una vez que se poseen las habilidades, programar una herramienta sencilla sólo cuesta una cantidad moderada de tiempo, lo cual suena plausible pero resulta sumamente poco realista. Esto se debe esencialmente a que los costos de diseño, mantenimiento y responsabilidad permanecen ocultos para ellos. Sin embargo, en un grupo de investigación en el que hay un creador de herramientas, esos costos pueden volverse visibles para los colegas investigadores, lo que allana el camino para la colaboración (que no siempre viene acompañada de atribución académica).

Aquí radica una diferencia importante entre un ingeniero de software de investigación (RSE) que no crea herramientas y uno que sí lo hace: sólo este último se ve llevado por las

circunstancias a asumir el rol de «propietario del producto» (product owner) (para tomar prestada una expresión de la industria del software). En ese sentido, muchos creadores de herramientas están más cerca del perfil de «Investigador-Desarrollador» caracterizado por Baxter et al. (2012); aunque, como explicamos antes, hay creadores de herramientas de distintos tipos (véase capítulo 20, figura 20.1). Preferimos llamar a ese perfil investigador-creador de herramientas, para mantener el énfasis en la cuestión de las herramientas. El punto clave aquí es que un investigador-creador de herramientas no está tan disponible para prestar servicio como un RSE, ya que sólo colaborará con alguien que no crea herramientas en la medida en que la colaboración sea de baja fricción y mutuamente beneficiosa. Por otro lado, para el investigador, colaborar con un creador de herramientas idealmente se asemeja mucho a colaborar con otro investigador: en pie de igualdad. En cambio, colaborar con un RSE no sólo es más asimétrico (porque el investigador dirige la agenda), sino que con demasiada frecuencia está plagado de sobrecargas organizativas (porque el RSE se reparte entre muchos proyectos). La disponibilidad para prestar servicio del RSE genera costos de gestión de proyectos precisamente porque se le impide elegir su propia agenda, mientras que el investigador-creador de herramientas colaborará como le parezca conveniente (en la medida en que pueda).

Es fácil imaginar que un investigador-creador de herramientas pueda desear crear una herramienta para encarnar y difundir sus puntos de vista metodológicos. Es menos fácil ver que la herramienta también hace al creador de herramientas, en particular empujándolo al rol de propietario del producto, es decir, el autor y diseñador de una pieza del proceso de producción de conocimiento científico, lo que conlleva responsabilidad. Estos dos movimientos se refuerzan mutuamente y nos incitan (tanto a creadores de herramientas como a investigadores) a identificar al creador de herramientas con su herramienta; es decir, a reducir al creador de herramientas a un servidor de la herramienta, a la vez que se reduce la herramienta a una expresión de su creador. Aunque en general hay algo de verdad en esta identificación, también es engañosa.

En efecto, la mayoría de los creadores de herramientas tienen otros objetivos además de atender las necesidades de su herramienta (y de sus usuarios); y las herramientas nunca son implementaciones transparentes de métodos bien definidos que existen en la mente del creador de herramientas. Por el contrario, muchas herramientas pueden verse como subproductos de exploraciones científicas en las que implementación y método son coconstitutivos, inventados juntos en un mismo movimiento. La agenda científica que el creador de herramientas afirma a través de su herramienta no necesariamente preexiste a la herramienta (véase capítulo 23). Algunos métodos son inventados por investigadores e implementados por creadores de herramientas, mientras que otros son descubiertos por el propio creador de herramientas mediante una experimentación algorítmica que puede dejar como resultado una pieza de software reutilizable que constituye de facto un prototipo de herramienta.

Incluso cuando el creador de herramientas es «sólo» el descubridor de un método, incluso cuando su agenda científica no es anterior a la creación de la herramienta y deriva enteramente

de ella, la existencia de la herramienta y su difusión afirmarán una agenda científica que se le atribuirá, en la medida en que se lo hace responsable de la agencia de su herramienta. Al creador de herramientas le resulta muy difícil distanciarse de su herramienta; por ejemplo, explicar que lo que defiende como investigador puede diferir de lo que su herramienta hace en ciertos contextos. Por ejemplo, los creadores de Gephi han criticado en vano el uso de la detección de comunidades en algunas situaciones, aunque la comunidad de usuarios de Gephi la presenta, en algunos tutoriales y discusiones de ayuda mutua, como parte integral del método de la herramienta (Jacomy & Jokubauskaitė, 2022). La casi inevitable identificación de la herramienta con su creador lleva a este último a mantener la herramienta lo más alineada posible con sus propios puntos de vista científicos y, así, a afirmar su autoría sobre ella.

LAS HERRAMIENTAS SON EL ORGULLO Y LA MOTIVACIÓN DEL CREADOR DE HERRAMIENTAS

> No creo que nadie vaya a leer nunca, en algún momento del futuro, los libros y los artículos [que escribí]. … Pero, sin duda, se me conoce por Voyant. … Hay gente que se me ha acercado y me ha dicho: ¡Oh, enseño con Voyant! ¿Sabe?, nadie se me acerca y me dice: ¡Oh, leí un artículo que usted escribió! (Geoffrey Rockwell, en Jacomy, 2023)

Una recompensa clave por asumir el rol de creador de herramientas es ser valorado y reconocido por los demás. Sin embargo, sorprendentemente, esa motivación rara vez está detrás del acto inicial de crear una herramienta; está detrás de la aceptación del propósito de sostener una o más herramientas. Uno no se convierte en creador de herramientas por crear una pieza de tecnología, sino por publicarla y darle soporte: una herramienta necesita perdurar. La economía de la creación de herramientas está dominada por el hecho de que retirarse del sostenimiento de la herramienta es fácil y a veces deseable, mientras que seguir con el problema de hacer que la herramienta perdure es costoso. Los creadores de herramientas aceptan la carga del mantenimiento en particular porque obtienen distintas cosas a cambio.

La perspectiva de quedar asociado a una pieza de software puede venir acompañada de temores, como el de ser objeto de burla por escribir «código chapucero que a nadie le gusta», quedar expuesto por los errores o romper la misteriosa etiqueta del desarrollo de código abierto (Kuzak, 2021; véase también Jiménez et al., 2017). Algunos investigadores que escriben código simplemente prefieren ocultarlo porque lo consideran perjudicial para su carrera (Yehudi, 2021). «Las principales razones que dan los investigadores para ello son la vergüenza debida a la percepción de que el código es de mala calidad, la falta de confianza [en que] el software sea robusto para otros usuarios y usos, y el tiempo necesario para prepararlo para su publicación, incluida la provisión de licencias y documentación adecuadas» (Jay et al., 2021; véase también Jay et al., 2016). Programar una pieza de software no lo convierte a usted en creador de herramientas, en la medida en que simplemente puede negarlo. Asumir la autoría de la herramienta es una premisa de la figura del creador de herramientas que no puede darse por

sentada. Volveremos en el capítulo 24 al problema de valorar la creación de herramientas en la academia.

En el otro extremo, abrazar el camino del creador de herramientas puede traer beneficios. Si la herramienta es popular, puede aportar notoriedad y reconocimiento. Si se publican artículos sustitutos (que se citan en lugar de la herramienta) o si las versiones de la herramienta obtienen un DOI, por ejemplo en el Journal of Open Source Software (Smith et al., 2018), puede aportar una gran cantidad de citas académicas. Puede tener valor en otros ámbitos, desde el periodismo de datos hasta la industria. Puede abrir puertas, aunque no necesariamente las que usted desea. Y las herramientas pueden tener un impacto mayor, una relevancia mayor que los artículos que pueda permitirse escribir. Sus herramientas pueden sobrevivir a sus escritos y a sus ideas.

Sin embargo, incluso esos beneficios tienen sus contrapartidas. Nosotros, autores de este texto, conocemos el riesgo de convertirnos en «el tipo de esa herramienta», de quedar encerrados en el propósito de cuidar de una herramienta teniendo ambiciones más amplias y otras habilidades. Inevitablemente, algunas personas nos ven así. Esto es un problema específicamente para los creadores de herramientas situados en el lado académico del espectro (figura 21.1, capítulo 21), que ven sus herramientas como sólo una parte de su producción. Con demasiada frecuencia, crear herramientas para la investigación no se considera tan importante como escribir un argumento en un texto. Hay tantísimos proyectos de investigación en los que autores han invertido cantidades significativas de tiempo en investigar y programar, sólo para recibir un agradecimiento en una nota al pie. A la inversa, en el lado de la infraestructura, un RSE que crea herramientas podría considerar deseable que su creación de herramientas esté protegida por la declaración de misión de su puesto. Enfrentar la instrumentalización por parte de los usuarios (investigadores, pero no sólo ellos) y elegir si resistirla o no es parte integral de la condición del creador de herramientas.

El trabajo oculto es otra parte de la condición del creador de herramientas. Reparar la herramienta cuando se rompe es un acontecimiento, pero el mantenimiento y la administración de las máquinas no lo son, y permanecen invisibles (Denis & Pontille, 2022). En consecuencia, el creador de herramientas a menudo tiene que encontrar maneras de justificar el tiempo que dedica a tareas que son invisibles para otros investigadores, a menos que simplemente lo haga en su tiempo libre. Esto no es específico de las DCSSH. De la astronomía a la zoología, prácticamente todos los grandes hallazgos científicos de la era moderna son posibles gracias a computadoras potentes cuyo software es moldeado por «científicos-programadores» específicamente capaces de abordar las preguntas de investigación adecuadas (Perkel, 2021); sin embargo, las instituciones científicas siguen vendiendo la narrativa heroica del supercientífico cuyas capacidades de teorización permiten alcanzar un avance decisivo. Una gran parte del esfuerzo de creación de herramientas científicas sigue sin contabilizarse en las instituciones académicas actuales. Desarrollaremos este punto en el capítulo 24.

La mayoría de los creadores de herramientas están orgullosos de las herramientas que crean y sienten cierto amor por ellas. Sus herramientas son una extensión, una encarnación de su

agenda científica. El orgullo es un componente necesario de la exigente economía del cuidado de la tecnología, sin el cual el creador de herramientas abandona su creación (véase también Rieder et al., 2024). Y, para ser claros, el amor y el orgullo a menudo coexisten con cierta dosis de odio y vergüenza. Los creadores de herramientas a veces desearían liberarse de las cadenas del cuidado y la responsabilidad, pero lo aceptan como parte de sí mismos. Uno se convierte en creador de herramientas al aceptar quedarse con los lados buenos y malos de ese extraño apego a una pieza de tecnología.

La creación de herramientas de las DCSSH es situada, y moldea el diseño de las herramientas. Porque los creadores de herramientas de las DCSSH son quienes son, porque su situación es la que es, diseñan sus herramientas de ciertas maneras y no de otras. A menudo las diseñarán para que sean fáciles de usar y para aprender haciendo. Evitarán implementar funcionalidades que se superpongan con lo que otras herramientas similares ya hacen y, en general, se resistirán a ampliar demasiado el alcance de su herramienta, pese a la presión de algunos otros usuarios. En el resto de este capítulo, detallaremos de dónde provienen estas distintas intenciones de diseño y cómo moldean las herramientas DC para las SSH.

DISEÑAN HERRAMIENTAS FÁCILES DE USAR

> Los humanistas no investigan todos los días de la semana ni todas las semanas del año. Están dando clases y, de repente, hacen un montón de investigación. Necesitan herramientas que puedan adoptar rápidamente, usar intensamente, dejar de lado y luego no usar durante seis meses. (Geoffrey Rockwell, en Jacomy, 2023)

Los creadores de herramientas son los primeros usuarios de sus herramientas y les desagrada perder tiempo en pasos innecesarios. Automatizan y optimizan; cajanegrizan la tecnología. Adaptan las herramientas a sí mismos, aunque no exclusivamente. Cuando enseñan, también optimizan para sus estudiantes, porque eso les facilita la vida. Cuando colaboran con investigadores que usan su herramienta, la adaptan para que resulte más accesible a ese público. Crean herramientas para que las usen ellos mismos, las personas de su entorno y desconocidos. La facilidad de uso es la expresión de la convicción del creador de herramientas de que ser accesible es algo bueno.

Los creadores de herramientas son conscientes de que la facilidad de uso implica compromisos que, llevados al extremo, podrían desvirtuar el propósito de la herramienta. Ponen un límite a cuán simples pueden ser, y deben ser, las herramientas, y ese límite depende de las circunstancias. Pero ninguno de nosotros, los creadores de herramientas que escribimos este texto, hizo jamás una herramienta intencionalmente complicada para disuadir a los usuarios de usarla. Las herramientas están pensadas para ser tan simples como puedan permitírselo o, en otras palabras, tan complicadas como sea necesario, pero no más. Lamentablemente, ese objetivo no necesariamente se cumple.

Una herramienta puede ser difícil de usar por muchas razones. En primer lugar, puede tener errores, comportarse de manera incoherente, colgarse o ser lenta o tener retrasos. Los creadores

de herramientas generalmente procuran evitarlo si pueden, siempre que cuenten con los recursos y las competencias necesarios. En segundo lugar, puede resultar difícil de interpretar visualmente, por falta de contraste o de principios de organización, o por exigir demasiada navegación por menús, demasiados clics o demasiado desplazamiento para llegar a sus funcionalidades principales. Estos problemas se deben a un diseño gráfico y/o de interacción deficiente y, aunque los creadores de herramientas quieren mitigarlos, bien pueden evaluar mal la situación porque carecen de habilidades de diseño o de comentarios de los usuarios. En tercer lugar, una herramienta puede resultar confusa porque ofrece demasiadas posibilidades; el usuario se siente abrumado y no ve dónde realizar las acciones necesarias para cumplir su tarea. Este desajuste entre el creador y el usuario puede ser un problema de diseño, pero a menudo también es un simple desajuste de público. Algunas herramientas están pensadas para expertos que desean las múltiples funcionalidades que dominan, en detrimento del principiante que necesita adquirir ese dominio. Para los creadores de herramientas, esta situación plantea la cuestión de cómo las herramientas ayudan a los principiantes a progresar, si es que lo hacen (Jacomy & Munk, 2024). En cuarto lugar, una herramienta puede ser difícil de usar porque las elecciones que el usuario debe hacer son difíciles, ya que tomar esas decisiones implica un esfuerzo que la herramienta no alivia. Este tipo de dificultad es real, pero relativa a la expectativa del usuario de que la herramienta lo asista más de lo que efectivamente lo hace. El usuario puede sentirse desatendido por la herramienta, pero quien la crea puede tener excelentes razones para diseñarla de ese modo. La razón principal es que el método de la herramienta requiere juicio humano, porque algunas decisiones dependen de factores que exceden el alcance de la herramienta o porque el creador de herramientas considera que ningún algoritmo puede hacer un trabajo lo suficientemente bueno.

Las herramientas DC para las SSH están pensadas para aportar comprensiones interpretativas, no sólo para producir resultados. La facilidad de uso puede ser contraproducente cuando permite al usuario obtener resultados que dependen de decisiones que no puede tomar por sí mismo, porque no ve lo que implican las elecciones o, peor aún, porque ni siquiera es consciente de que esas decisiones existen. En una situación así, el usuario no domina el método que la herramienta respalda. Los creadores de herramientas a veces limitan la facilidad de uso cuando consideran necesario exponer ciertas decisiones al usuario, ya sea para que entienda lo que está ocurriendo o porque esos puntos de decisión son compromisos metodológicos. En cierto sentido, los creadores de herramientas protegen a los usuarios de sí mismos, de su deseo de obtener resultados sin esfuerzo; pero, en otro sentido, paradójicamente, los creadores de herramientas también desean que los usuarios simplemente la pasen bien con las herramientas.

> No creemos que nuestros usuarios tengan mucha paciencia. Sin duda hay personas que dedicarán una hora a instalar Anaconda, y luego una cosa u otra, y luego se pondrán a jugar con las herramientas y a leer los libros … Y una semana después obtienen algo. Pero nuestra herramienta está pensada como un nivel de entrada. Se puede usar un texto que ya hayamos subido, o pegar algún texto, o pegar una URL, o cargar un texto, lo que se pueda, y nosotros

> intentaremos arreglárnoslas con eso. … Intentamos que sea lo más fácil posible empezar y jugar. (Geoffrey Rockwell, Jacomy, 2023)

DISEÑAN PARA EL JUEGO Y PARA APRENDER HACIENDO

> Sin duda coincidiría en que las personas deberían entender [una visualización que hicieron con una herramienta]. Pero creo que hay una manera intuitiva en la que los humanistas pueden entenderla en parte poniendo a prueba los resultados. … La visión [de Stéfan y mía] es que queremos que las personas empiecen a jugar, y empezarán a entender jugando. Esa es nuestra filosofía para abordar esto, en lugar de ponerlas en una situación de asistente en la que hay que tomar una serie de decisiones, hacer instalaciones, y solo si uno se portó muy bien y aprobó el examen llega a ver algo. (Geoffrey Rockwell, Jacomy, 2023)

Algunas herramientas están diseñadas para que el usuario pueda aprender usando la herramienta: aprender a usar la herramienta, por supuesto, pero también aprender otras cosas: que hay distintas maneras de hacer una misma cosa, distintos algoritmos y métodos; cómo funciona un algoritmo determinado; cuáles son, o podrían ser, los distintos pasos del método; que distintos procesos dan resultados diferentes, cuáles son y qué implican; y más.

Para el investigador con poco tiempo, la herramienta es a menudo un medio para un fin. Pero la investigación tiene otros momentos en los que la herramienta es más que eso, en los que el investigador puede reflexionar sobre cómo usa la herramienta, qué implica el método y qué significa para su investigación. En ese sentido, la academia es profundamente distinta de otros contextos, como el marketing o el trabajo de consultoría. La herramienta no puede seguir siendo una asistencia puramente utilitaria; las prácticas instrumentadas deben ser exploradas, examinadas, cuestionadas y reconfiguradas.

Aprender haciendo es, por supuesto, más desordenado que el ordenado aprender leyendo que practica la mayoría de los docentes. Con la lectura, uno puede al menos fingir que el conocimiento es alguna cualidad que el texto transmite a la mente. Se puede evaluar a los estudiantes midiendo la fidelidad de su conocimiento a la lección. Pero no ocurre lo mismo con el uso de una herramienta, donde el conocimiento es implícito. El conocimiento es simplemente más que eso, como todos sabemos. Si miramos más de cerca la práctica de la lectura, debemos enfrentar el hecho de que, incluso con las enciclopedias, no leemos de principio a fin ni por entradas pertinentes. Deambulamos, hojeamos, saltamos y volvemos atrás. Curioseamos. Stephen Ramsay (2010) lo llama «screwing around» [andar perdiendo el tiempo] y lo opone a la «search» [búsqueda], como en una biblioteca y como en Google. Curiosear es «una actividad completamente distinta. Porque, una vez que se tiene acceso programático al contenido de la biblioteca, andar perdiendo el tiempo se vuelve de pronto una actividad mucho más esclarecedora y útil» (ibid, p. 6). Ramsay defiende el curioseo como un método de investigación tan válido como la búsqueda. «No se trata de reemplazar uno por el otro, como le dirá cualquier bibliotecario. Se trata más bien de preguntarse si estamos dispuestos a aceptar navegar y tropezar —andar perdiendo el tiempo, en sentido amplio— como una metodología de investigación» (ibid, p. 7). La «hermenéutica de andar perdiendo el tiempo» (ibid) con

herramientas trae consigo no solo el descubrimiento de la herramienta en sí, sino también la readaptación y la invención. Puede sacar a la luz cosas de las que ni siquiera el creador de herramientas es consciente.

Los creadores de herramientas diseñan para aprender haciendo y para el juego por dos razones. En primer lugar, porque la herramienta es su medio de expresión. Prefieren transmitir conocimiento a través de las affordances de la herramienta antes que a través de la documentación porque, si no fuera así, para empezar escribirían texto y no código. Los creadores de herramientas a menudo piensan, o más bien experimentan, el método a través de su herramienta. En segundo lugar, los creadores de herramientas generalmente encuentran placer en usar y crear herramientas (lo cual no excluye el sufrimiento, para que quede claro). La creación de herramientas implica juego porque trae sorpresas, giros inesperados y exige soluciones creativas, que también pueden encontrarse en la investigación en las DCSSH. Los creadores de herramientas de las DCSSH tienden a apreciar el aspecto creativo de la creación de herramientas y de la investigación, y naturalmente lo respaldan en sus herramientas. Muchas herramientas DC para las SSH son lúdicas por esa razón. Permanecen intencionalmente abiertas a usos no utilitarios como prácticas epistémicas valiosas.

> No creo que hubiéramos llegado adonde llegamos si no lo hubiéramos disfrutado. Y parte del placer consistía, de hecho, en pensar en el juego. En inglés hay un uso pertinente de la palabra play [juego]. Digamos que uno tiene una perilla: puede toquetearla hasta que se rompa. Ese tipo de juego, jugar con cosas que no funcionan del todo bien. Hay un juego entre la teoría y la implementación que era realmente divertido. Es como una montaña rusa. Uno sube muy alto en la teoría; construíamos este tipo de teoría, y la mayoría de mis colegas no tuvieron la suerte de poder luego descender en picada, hacer que funcionara y después hacer que funcionara para otras personas. Esa montaña rusa, para mí, tenía que ver con el vértigo. Roger Caillois tiene esa teoría del juego, y el vértigo es uno de sus elementos. (Geoffrey Rockwell, en Jacomy, 2023)

DISEÑAN HERRAMIENTAS COMO PARTES DE UN ECOSISTEMA

Las herramientas de código abierto, incluidas las que se usan en las DCSSH, se desarrollan para coexistir con otras herramientas. Puede resultar sorprendente para quienes piensan en las herramientas como competidoras en un mercado, pero los creadores de herramientas de código abierto similares a menudo cooperan. Nosotros, los autores de este texto, sin duda sentimos que estamos todos en el mismo barco. Ese barco se llama Mantenimiento Sin Financiamiento y no tiene velas: todos estamos remando.

La necesidad de minimizar la carga del mantenimiento genera una división natural del trabajo entre las herramientas. Algunos creadores de herramientas piensan: si la herramienta de otra persona tiene una funcionalidad que me gustaría que tuviera mi herramienta, podría derivar a los usuarios a esa otra herramienta y tener menos cosas que mantener. Esta mentalidad implica tender puentes entre herramientas y fomentar la intercompatibilidad. Los creadores de herramientas de las DCSSH diseñan sus herramientas para la coexistencia, no para apoderarse

del mercado. Para el creador de herramientas de código abierto, que su herramienta se convierta en la única no es el sueño, sino la pesadilla.

Esta inclinación por la coexistencia contribuye a dar forma a las herramientas DC para las SSH. Por ejemplo, a diferencia de los productos comerciales, las herramientas libres evitan tener su propio formato de archivo específico. Por lo general, utilizan entradas y salidas en formatos universales como CSV, JSON o XML. Tienden a reutilizar estándares abiertos tanto como sea posible.

Toda herramienta de código abierto depende de una miríada de otras y, a veces, otras dependen de ella. Inserta al creador de herramientas en una amplia red cooperativa, en gran medida paralela a la comunidad de investigación: una red de desarrolladores de software con ideas afines. Los creadores de herramientas de las DCSSH muestran respeto por sus colegas creadores de herramientas. Por lo general, evitan invadir el ámbito de otra herramienta o piden permiso (p. ej. para reutilizar un algoritmo). El diseño de las herramientas de las DCSSH está moldeado en gran medida por esta cooperación, en particular mediante una división del trabajo en la que los creadores de herramientas buscan crear herramientas que se complementen entre sí. Por eso, como usuario, rara vez puede permitirse aprender una sola herramienta para una necesidad determinada.

> El proyecto TAPoR inicial, el portal de análisis de textos, era un proyecto imperial; y eso era parte del problema. Intentaba decir: ¡podemos hacerlo todo para todos! Eso era evidentemente insostenible. O al menos no logramos descubrir cómo sostenerlo. En cambio, decir: miren, tenemos esto, ¡es gratuito, es de código abierto! Pueden hacer lo que quieran con esto; pueden tomar el código; publicamos artículos con nuestras ideas: pueden tomar las ideas o discrepar con ellas. Creo que, en términos generales, evitamos cualquier tipo de conflicto imperial. Desde entonces, Voyant ha recibido algunos premios, como el premio Zampolli, que es realmente el máximo galardón para un proyecto en las humanidades digitales. Creo que eso refleja el hecho de que es apreciado en la comunidad. Espero que no se lo vea como algo que silencia a alguien. (Geoffrey Rockwell, Jacomy, 2023)

DISEÑAN HERRAMIENTAS QUE LIDIAN CON LAS DEMANDAS DE LOS USUARIOS

> Siempre hay algún detalle que quieren. Y esa es la belleza de haber agregado el entorno de programación de cuadernos (notebooks). Ahora puedo decir: de acuerdo, Voyant hace lo que hace, pero ustedes tienen un entorno de programación de cuadernos. Consigan una subvención, contraten a un programador de JavaScript y pueden agregar lo suyo. Y si es lo suficientemente bueno, lo tomaremos y lo incorporaremos a la herramienta principal. (Geoffrey Rockwell, Jacomy, 2023)

Los creadores de herramientas y los usuarios a menudo quieren cosas distintas, pero el desacuerdo rara vez se refiere a cómo debería funcionar una funcionalidad. Más bien, se refiere a qué funcionalidades deberían o no formar parte de la herramienta. Es una cuestión de alcance. Por lo general, un usuario demanda una funcionalidad que es razonable en sí misma, pero que resulta problemática para el creador de herramientas por otras razones. La cajanegrización de la herramienta oculta esa complejidad a los usuarios, lo cual generalmente es algo bueno, ya que

simplifica el uso, pero también da la falsa impresión de que construir una herramienta consiste en apilar funcionalidades como si fueran ladrillos. Los usuarios pueden sentirse frustrados cuando se les niegan funcionalidades que consideran simples o necesarias. Por ejemplo, los usuarios llevan más de una década pidiendo una funcionalidad para deshacer en Gephi, y los creadores de herramientas llevan años trabajando en ella; sin embargo, todavía no está lista. El hecho de que la funcionalidad para deshacer sea omnipresente puede llevar a quienes no son desarrolladores a engañarse sobre la dificultad de implementarla.

Cuando una funcionalidad útil no se implementa, generalmente es porque resulta demasiado costosa de implementar o de mantener, o porque no puede implementarse por otras razones prácticas. Por ejemplo, los usuarios de software para redes sociales como 4CAT o YouTube Data Tools a veces demandan funcionalidades que dependen de datos que no están disponibles o que van en contra de las condiciones de servicio de las plataformas. Más raramente, el diseñador considera que una funcionalidad es contraproducente y se niega intencionalmente a implementarla. Los distintos creadores de herramientas tienen perspectivas diferentes al respecto. Algunas herramientas toman posición de manera muy marcada (Gephisto es un buen ejemplo; Jacomy and Munk, 2024), mientras que otras son lo más inclusivas posible desde el punto de vista metodológico (p. ej. SciKit Learn; Pedregosa et al., 2011). Pero los creadores de herramientas coincidirán al menos en un aspecto de esto: la proliferación de funcionalidades (feature creep), también conocida como «featuritis», es una patología de las herramientas que envejecen en la que la acumulación de funcionalidades se vuelve perjudicial para el mantenimiento y/o el uso. La featuritis es difícil de evitar. O bien el creador de herramientas abandona su herramienta, o bien la sostiene el tiempo suficiente como para enfrentarse al problema. Algunos creadores de herramientas optan por seleccionar rigurosamente las funcionalidades de sus herramientas y mantenerlas al mínimo para limitar la complejidad del diseño y la carga de mantenimiento.

> Hay una enorme cantidad de funcionalidades involucradas [en Voyant]. Cada vez que alguien viene y me dice: ¿puedo hacer X?, le digo: sí, simplemente vaya aquí, haga eso … ¡Vaya! ¡Ni siquiera sabía que eso estaba ahí! Y, en cierta medida, este es el problema de la featuritis y de la proliferación de funcionalidades del software en general. Voyant existe desde hace bastante tiempo. Seguimos agregando funcionalidades. (Geoffrey Rockwell, Jacomy, 2023)

LAS HERRAMIENTAS SON RECONFIGURADAS POR SUS USUARIOS

> [Voyant] está hecho para humanistas. … Dicho esto, … recibo correos electrónicos de abogados para usarlo. Un correo del hijo de un médico cuyo padre lo usa. Lo usan muchas personas diferentes. (Geoffrey Rockwell, Jacomy, 2023)

Las herramientas circulan de manera distinta que los artículos. Contribuyen a construir comunidades de un modo diferente. Algunas herramientas tienen comunidades de usuarios poco ortodoxas, organizadas en torno a una readaptación específica. Gephi, por ejemplo, ha sido reapropiado por una parte de la comunidad de consultores que se dedican a la optimización para motores de búsqueda (SEO). Tienen sus propios objetivos (no científicos) y han

desarrollado sus propias prácticas; constituyen una subcultura epistémica (Knorr Cetina, 1999; véase también Jacomy & Jokubauskaitė, 2022). Los usuarios no siempre son quienes los creadores de herramientas piensan. A medida que una herramienta viaja a lugares lejanos, se libera de la esfera de personas que los creadores de herramientas conocen y para las que diseñan. La variedad de usuarios es a menudo una sorpresa y una fuente de asombro para los creadores de herramientas, que entonces se preguntan: ¿qué hacen realmente los usuarios con mi herramienta?

Los usuarios no están completamente libres del diseño de la herramienta, en la medida en que la herramienta los restringe, pero pueden liberarse de la intención de diseño. Los investigadores (y otros usuarios) pueden readaptar las herramientas de las DCSSH, lo harán y deberían hacerlo, porque estas siempre son más de lo que sus creadores pretendían.

La idea de la readaptación está en el centro de las herramientas desarrolladas en la Digital Methods Initiative (DMI) de Ámsterdam, como 4CAT (Peeters and Hagen, 2022), DMI-TCAT (Borra and Rieder, 2014), Netvizz (Rieder, 2013) o YouTube Data Tools (Rieder, 2015). Todas surgen de la constatación de que los distintos medios necesitan sus propios métodos de investigación, lo que llevó a los creadores de herramientas de la DMI a readaptar plataformas y motores en línea. El director de la DMI, Richard Rogers, «posiciona los métodos digitales como una investigación sensible a los métodos incorporados en el medio, en contraposición a la aplicación de métodos existentes de las ciencias sociales y las humanidades» (Weltevrede, 2016). Esta reapropiación del espacio público digital condujo, en última instancia, a la creación de muchas herramientas. Las prácticas de estos creadores de herramientas muestran que la tecnología, como las API de las redes sociales, nunca está cautiva de sus intenciones de diseño. Las API son herramientas para algunos y material para la creación de herramientas para otros. Las herramientas de la DMI se construyen sobre otras herramientas readaptadas para posibilitar la investigación.

Cuando se readapta una herramienta, se practica en esencia la creación de herramientas, incluso en ausencia de cualquier intervención material sobre la herramienta misma. Se practica la innovación metodológica, una forma de experimentación científica entre otras. Si la comunidad de usuarios es proactiva en la publicación de conocimiento sobre una herramienta, entonces contribuye a darle forma. Los creadores de herramientas codiseñan las herramientas con los usuarios porque estos tienen el potencial de reinventarlas por completo. Dicho de otro modo, las herramientas de las DCSSH no residen solo en su código fuente, sino también en los artefactos culturales que las contextualizan, desde los debates en línea, los tutoriales y la documentación hasta los materiales didácticos, los artículos científicos y los data sprints.

Como hemos visto, los creadores de herramientas tienen sus propios objetivos y motivaciones para crear las herramientas que usamos en las DCSSH. Sus perspectivas son parciales. No necesariamente apuntan al bien común, ante todo porque dista mucho de haber una única versión del «bien común» para las DCSSH, pero también porque crean herramientas por razones específicas (p. ej. compartir sus visiones científicas, comprender las API, ayudar a

otros) mientras están sujetos a muchas restricciones (p. ej. la rendición de cuentas, las limitaciones técnicas, la falta de recursos para el mantenimiento). No existe una autoridad superior que decida qué herramientas deberían crearse o sostenerse para atender de la mejor manera las necesidades de las DCSSH, y no hay razón para suponer que esas necesidades estén adecuadamente cubiertas por la caja de herramientas DC actualmente disponible para los investigadores de las SSH. El aparato disponible para los investigadores en las ciencias humanas está moldeado por fuerzas distintas de una lógica de satisfacción de las necesidades académicas.

Dicho esto, los investigadores tienen más poder del que tal vez suponen sobre el ecosistema de herramientas de las DCSSH. Pueden readaptar herramientas y contribuir a reconfigurar sus métodos mediante tutoriales y materiales didácticos; pueden presupuestar las funcionalidades que desean en las solicitudes de subvención; pueden cooperar con los creadores de herramientas para reconfigurar su aparato; y pueden involucrarse con el software y, eventualmente, convertirse ellos mismos en creadores de herramientas. De esas distintas maneras, tienen la posibilidad de aportar nuevas intenciones de diseño y orientar el desarrollo de herramientas de las DCSSH en otras direcciones.

REFERENCIAS

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. Digital Research 2012 (pp. 1–3). The University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. Aslib Journal of Information Management, 66(3), 262–278.

Denis, J., & Pontille, D. (2022). Le soin des choses: Politiques de la maintenance. La Découverte.

Ham, K. (2013). OpenRefine (version 2.5). http://openrefine.org. Free, open-source tool for cleaning and transforming data. Journal of the Medical Library Association, 101(3), 233.

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. Reticular. https://reticular.hypotheses.org/8039

Jacomy, M., & Jokubauskaitė, E. (2022). Unblackboxing Gephi: How a user culture shapes its scientific instrument. Working paper. https://hal.science/hal-03729114/file/Unblackboxing%20Gephi%20Rev%201.pdf (accessed 26/08/2024).

Jacomy, M., & Munk, A. K. (2024). Interfering with the black-box-tradeoff model: Gephisto, a one-click Gephi for critical technical practice. Convergence, 30(1), 142–166. https://doi.org/10.1177/13548565221129053

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. International Journal of Digital Curation, 16(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Jay, C., Sanyour, R., & Haines, R. (2016). "Not everyone can use Git": Research Software Engineers' recommendations for scientist-centred software support (and what researchers really think of them). Journal of Open Research Software.

Knorr Cetina, K. (1999). Epistemic cultures: How the sciences make knowledge. Harvard University Press.

Kuzak, M. (2021). On the road to sustainable research software. FOSDEM 2021, communication in the Open Research devroom (video). https://archive.fosdem.org/2020/schedule/event/open_research_sustainable_soft/ (accessed 07/05/2024).

Jiménez, R. C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., ... & Crouch, S. (2017). Four simple recommendations to encourage best practices in research software. F1000Research, 6.

OpenRefine (2024). OpenRefine Project History. https://openrefine.org/openrefine_history (accessed 20/08/2024).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12(85), 2825–2830.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. Computational Communication Research, 4(2), 571–589.

Perkel, J. M. (2021). Ten computer codes that transformed science. Nature, 589(7842), 344–348. https://doi.org/10.1038/d41586-021-00075-2

Ramsay, S. (2010). The hermeneutics of screwing around; or what you do with a million books. In K. Kee (Ed.), Pastplay: Teaching and learning history with technology (pp. 111–120). The University of Michigan Press.

Rieder, B. (2013). Studying Facebook via data extraction: the Netvizz application. In Proceedings of the 5th annual ACM web science conference (pp. 346–355). Association for Computing Machinery.

Rieder, B. (2015). YouTube Data Tools [software]. Available at: https://tools.digitalmethods.net/netvizz/youtube/ (accessed 09/08/2024).

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. Convergence, 30(1), 216–235. https://doi.org/10.1177/13548565221127094

Smith, A. M., Niemeyer, K. E., Katz, D. S., Barba, L. A., Githinji, G., Gymrek, M., Huff, K. D., Madan, C. R., Mayes, A. C., Moerman, K. M., Prins, P., Ram, K., Rokem, A., Teal, T. K., Guimera, R. V., & Vanderplas, J. T. (2018). Journal of Open Source Software (JOSS): Design and first-year review. PeerJ Computer Science, 4, e147.

Weltevrede, E. J. T. (2016). Repurposing digital methods: The research affordances of platforms and engines [Doctoral dissertation, Universiteit van Amsterdam].

Yehudi, Y. (2021). Open research in life science: Funding foundational tools, trust, and talent. FOSDEM 2021, communication in the Open Research devroom (video). https://archive.fosdem.org/2021/schedule/event/open_research_life_science/ (accessed 07/05/2024).

23. El ciclo de vida de las herramientas DC para las SSH

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Sostenemos que las herramientas DC para las SSH nunca son productos acabados con una vida tranquila que el creador de herramientas pueda anticipar. Katz y Stall resumen el ciclo de vida de las herramientas científicas de la siguiente manera: «El software de investigación suele ser desarrollado por los propios investigadores, a menudo inicialmente para resolver un único problema, y luego generalizado para resolver otros problemas e, idealmente, compartido para que otros investigadores también puedan beneficiarse y evitar la duplicación del trabajo que requieren el desarrollo y el mantenimiento. Cuando este proceso funciona, el software puede convertirse en software comunitario, desarrollado, respaldado y mantenido por una comunidad de usuarios-desarrolladores, y ampliarse a medida que se aplica a nuevos problemas. Que esto ocurra o no depende en gran medida tanto de la cultura de la comunidad como de los incentivos asociados» (Katz & Stall, 2021).

Desarrollamos esas ideas considerando distintos tipos de herramientas en distintas situaciones. Los temas que abordamos siguen los ciclos de vida de las herramientas de principio a fin: cómo llegan a existir las herramientas, cómo reciben su nombre, cómo evoluciona su conjunto de funcionalidades, cómo se mantienen y cómo dejan de funcionar y mueren. Comprender estas distintas etapas de la existencia de una herramienta ayuda a pensar críticamente sobre los métodos incorporados en ella, a evaluar cuán probados y comprobados están y cómo podría usted involucrarse en readaptar la herramienta y remodelar su propósito científico. De hecho, muchas piezas de software que no son aplicaciones, como los scripts y los servicios en línea, también son herramientas DC para las SSH. Según su nivel de compromiso con esos objetos, puede que advierta que ya está recorriendo el camino de la creación de herramientas, o que podría contribuir a mejorar o incluso simplemente a sostener las herramientas que necesita para su investigación. En particular, nos tomaremos el tiempo de discutir la necesidad de mantener las herramientas DC y lo que ello implica para la comunidad de investigación, una cuestión crucial para los creadores de herramientas de las DCSSH, pero invisible para la mayoría de los investigadores, que no son desarrolladores.

LAS HERRAMIENTAS VIENEN EN DISTINTAS FORMAS Y TAMAÑOS

Algunos investigadores han inventariado herramientas DC para las SSH. La Comisión Europea financió el proyecto SSHSOC, que tenía como objetivo «proporcionar una Nube Abierta de Ciencias Sociales y Humanidades (SSH) completa, donde los datos, las herramientas y la formación estén disponibles y sean accesibles para los usuarios de datos de las SSH» (Veršić &

Ausserhofer, 2019). En particular, produjo un «mercado abierto» en línea (Social Sciences and Humanities Open Marketplace, 2023) en el que hay registradas 2165 herramientas (en el momento en que escribimos, en agosto de 2024). Esas herramientas están categorizadas por función. Cada herramienta puede tener múltiples funciones, y las 10 más representadas son «analizar» (548 herramientas); «visualización de datos» (344); «análisis visual» (283); «análisis de contenido» (243); «descubrir» (194); «anotar» (180); «capturar» (158); «difundir» (151); «publicar» (148); y «colaborar» (144).

Más interesante para nosotros, la plataforma también clasifica algunas de las herramientas por «tipo de invocación» (SSH vocabulary service: Invocation type, 2023), es decir, según cómo se presentan materialmente al usuario. Su ontología tiene sólo las siguientes nueve categorías (en orden alfabético): «Command line»; «Command processor»; «Instruction page»; «Library»; «Local application»; «RESTful webservice»; «Script»; «Source code»; y «Web application» (ibid). Lamentablemente, sólo unas pocas de las entradas parecen tener este campo completado, y no es posible buscar por él en la plataforma en línea, lo que nos impide contar cuáles de esas categorías son las más representadas. Con todo, podemos usar esta ontología como una representación de la diversidad material de las herramientas DC para las SSH.

Podemos organizar esos tipos planteando algunas preguntas. La primera pregunta que hay que plantear es: ¿la herramienta es computacionalmente pasiva o activa? Las herramientas computacionalmente pasivas consisten en inscripciones puras (conocimiento, datos), destinadas a ser leídas o compartidas, pero no ejecutadas. La página de instrucciones es de ese tipo. Por extensión, podríamos considerar los artículos y los libros como herramientas destinadas a ser leídas y puestas en acto por medios no computacionales. El tipo código fuente es un caso límite, ya que representa el modo de existencia computacionalmente pasivo de un dispositivo que, por lo demás, es activo. Ausentes de la ontología de SSHSOC (aunque muy reales), algunas herramientas son materialmente activas pero no computacionales, como el bolígrafo y el cuaderno del etnógrafo, o los dispositivos de grabación como micrófonos y cámaras, cuyo tamaño y apariencia tienen un impacto bien conocido en las entrevistas, por ejemplo. Las herramientas no computacionales no son el foco de este texto, pero vale la pena mencionar que muchos de nuestros planteos sobre la tecnología no son específicos de lo digital, como la necesidad de mantener un instrumento o el hecho de que este tiene agencia científica.

Con las herramientas computacionalmente activas para las SSH, hay una segunda pregunta que plantear: ¿la herramienta es un componente o es autónoma? Los componentes son herramientas para quienes experimentan artesanalmente y para ingenieros de software, pero pueden parecer infraestructuras a quienes no programan. Esos tipos son la biblioteca (p. ej. DaCy, SciKit-Learn) y el servicio web RESTful (p. ej. Textgrid; Neuroth et al., 2011). Los investigadores suelen acceder a esas herramientas dentro de Jupyter Notebooks (Kluyver et al., 2016) o desde otras herramientas como Voyant u OpenRefine. Por el contrario, las herramientas autónomas son independientes y autosuficientes. Podemos diferenciarlas aún más preguntando si ofrecen una interfaz gráfica de usuario (GUI). El script y la herramienta de línea de comandos

son puramente textuales y no tienen GUI; la única diferencia entre ambos es que el script está pensado para ser leído y editado, mientras que la línea de comandos está cajanegrizada (p. ej. Minet); pero cualquier script puede convertirse en una línea de comandos si es necesario. La aplicación local (p. ej. VOSviewer, Gephi) y la aplicación web (p. ej. DMI-TCAT, Hyphe) se benefician de una GUI, razón por la cual las llamamos aplicaciones. Además, y de manera transversal a esas distinciones, también podemos diferenciar las herramientas según se ejecuten localmente (en su propia computadora) o sólo se pueda acceder a ellas en línea (en un servidor, en una infraestructura en la nube o mediante una GUI web). Resumimos este panorama en la figura 23.1. Con esta descripción, buscamos mostrar la diversidad de lo que «herramienta» puede significar en las DCSSH. Aunque tendemos a pensar en las herramientas como aplicaciones (autónomas y con una GUI), muchas herramientas DC para las SSH son, de hecho, diferentes.

Según su dominio de lo digital, el mismo dispositivo puede parecerle una infraestructura o una herramienta. «La infraestructura es a la vez relacional y ecológica: significa cosas diferentes para grupos diferentes y forma parte del equilibrio de la acción, las herramientas y el entorno construido, inseparable de ellos» (Star, 1999). Aquí definimos las infraestructuras como sistemas de soporte fundamentales en los que uno puede permitirse no pensar. Como ya mencionamos, un dispositivo puede ser considerado una herramienta por alguien y una infraestructura por otra persona. Jupyter Notebooks puede ser al mismo tiempo una infraestructura para ejecutar métodos computacionales y una herramienta para introducir a los estudiantes en la exploración de datos. Del mismo modo, una biblioteca de código como SpaCy (Honnibal & Montani, 2017) podría verse ya sea como una dependencia de otra herramienta como DaCy (como parte de su infraestructura), ya sea como una herramienta en sí misma, a la que recurrir cuando surgen ciertas necesidades. Por razones similares, algunos públicos encuentran incómodo el uso de herramientas de interfaz de línea de comandos (CLI) como Minet, mientras que otros las consideran superiores a las aplicaciones equivalentes basadas en GUI: más directas, rápidas de usar y flexibles.

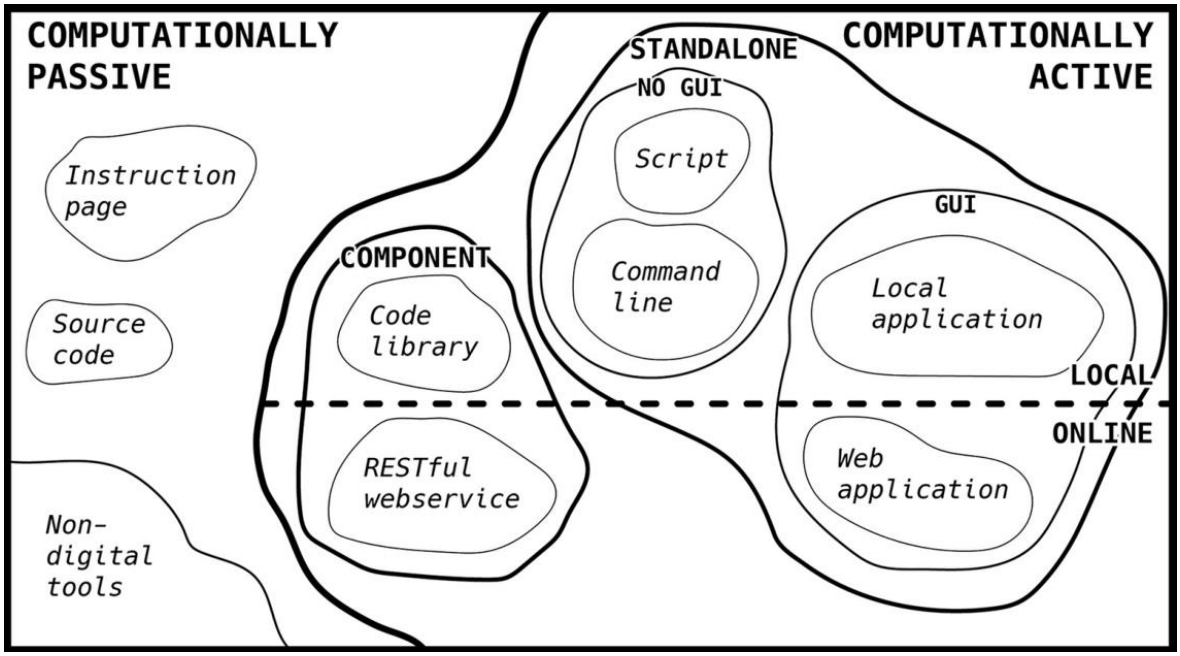


Figura 23.1 Panorama de los diferentes tipos de herramientas DC para las SSH, desde el punto de vista de su materialidad

Notas: «Local» se refiere al software que puede ejecutarse en su propia computadora, mientras que «en línea» se refiere a sistemas que funcionan sólo en la nube. «GUI» significa «interfaz gráfica de usuario».

Fuente: elaboración propia.

Las herramientas rara vez se construyen desde cero. La mayoría de las veces, el acto principal de la creación de una nueva herramienta consiste en integrar componentes existentes en una forma más mercantilizada, que los vuelve accesibles a nuevos públicos. Las herramientas están hechas de otras herramientas que están hechas de otras herramientas, y así sucesivamente; en cada paso, la diferencia de contexto readapta los componentes y los convierte en algo nuevo. A partir de un artículo existente que describe un algoritmo, un creador de herramientas escribe un script que implementa ese algoritmo para poder usarlo con sus datos. A partir de un script existente, un creador de herramientas escribe una biblioteca para reutilizar fácilmente ese método en distintos contextos. A partir de bibliotecas existentes, un creador de herramientas crea una herramienta CLI para que repetir el método que siempre usa no requiera esfuerzo. A partir de bibliotecas existentes, un creador de herramientas construye una aplicación basada en GUI para que a él y a otros les resulte más fácil monitorear el comportamiento de un algoritmo y tomar decisiones metodológicas. El acto aparentemente mínimo de tomar una pieza de tecnología y trasladarla a un contexto diferente, con un público diferente, basta para crear una herramienta. A los ojos del nuevo público, el dispositivo es una herramienta porque, para ellos, marca una diferencia en la forma en que acceden a la tecnología.

A medida que envejecen, las herramientas se vuelven más o menos famosas, más o menos populares. Algunas herramientas son de nicho y sólo les hablan a especialistas, mientras que otras son para todos. Aun así, no es fácil adivinar si una herramienta se volverá popular. La popularidad escapa en gran medida al control del creador de la herramienta, quien puede desear

que su herramienta fuera más popular, pero también, a veces, menos. En efecto, un público que crece en órdenes de magnitud cambia las condiciones de existencia de una herramienta y conduce a la transformación de esta, de la organización social que la rodea y, a veces, de la carrera y la vida personal del creador de la herramienta. Desde su concepción hasta el olvido, el ciclo de vida de una herramienta puede tener muchos giros. A continuación lo examinaremos.

LA MAYORÍA DE LAS HERRAMIENTAS DC PARA LAS SSH NACEN DE LA PRÁCTICA

> Stéfan y yo empezamos una praxis ágil en la que los dos nos reuníamos en una sala e intentábamos hacer un proyecto en un día. … El objetivo era hacer muchos proyectos y ver qué herramientas eran útiles y cuáles no. Simplemente improvisábamos con lo que teníamos y, cuando conseguíamos algo que funcionaba, lo implementábamos para Voyant. (Geoffrey Rockwell, Jacomy, 2023)

Una de las especificidades del software como entorno tecnológico es la flexibilidad que ofrece la posibilidad de duplicar código prácticamente sin costo. Como el proceso de prototipado de una herramienta implica iterar sobre implementaciones, el costo de implementación determinará el costo del proceso de diseño. Las herramientas físicas son costosas de implementar porque cada prototipo es costoso; las herramientas de software no lo son. El diseño de la herramienta sigue teniendo un costo, porque requiere tiempo y esfuerzo, pero iterar una vez más nunca es un problema. Mientras que la economía del diseño de herramientas materiales está condicionada por la necesidad de minimizar el número de prototipos construidos, el diseño de software está impulsado por la oportunidad de iterar sin costo. En consecuencia, las iteraciones pueden ser tan pequeñas y frecuentes que el proceso de diseño parece continuo. Algunos proyectos de software pueden incluso adoptar la llamada integración continua, en la que hasta la más pequeña actualización del código (validada por pruebas) se compila automáticamente en una nueva versión publicada. Esto tiene, a su vez, dos consecuencias específicamente para los instrumentos de software: todo prototipo es una herramienta; y toda herramienta es un prototipo.

Todo prototipo es una herramienta en el sentido de que cualquier prototipo que funcione, es decir, que marque una diferencia en un contexto científico, puede potencialmente reutilizarse como herramienta sin agregarle nada. Esto se debe, en particular, al hecho de que los laboratorios (en sentido amplio) son lugares de experimentación donde el aparato no está necesariamente cajanegrizado y pulido, sino semiabierto y sometido a ajustes artesanales. En otros términos, los investigadores son particularmente propensos a sacrificar mucha comodidad si una herramienta resuelve un problema que está bloqueando su investigación. Por supuesto, a uno suele atraerle la tecnología porque es poderosa. La mayoría de los investigadores de las SSH, que no se definen como «digitales» o «computacionales», han tenido o tendrán que afrontar la incomodidad de un software desconocido porque su investigación lo exige. Típicamente, desde el punto de vista del investigador, si el Jupyter notebook a medio hacer de un colega hace algo que realmente hace avanzar su propia investigación, intentará usarlo. Muchas piezas de software inacabadas empezaron a circular porque eran útiles. Como observan

van Es et al. (2021), las herramientas DC para las SSH llegan a existir primero como «prototipos con errores [y] soluciones tentativas».

Toda herramienta es un prototipo en el sentido de que nunca está terminada. Por ejemplo, Gephi tiene más de una década y, sin embargo, todavía no ha alcanzado la versión 1.0, el hito que sus creadores reservan como forma de indicar que estaría, a sus ojos, terminada. Esto no es específico de la ciencia, sino que está generalizado en la industria del software. Cualquiera que use una computadora para trabajar probablemente esté familiarizado con el estado en perpetua evolución del software de oficina, que a veces impone un grado sorprendente y frustrante de inacabamiento. Pero, de todos modos, ninguna pieza de software está nunca verdaderamente terminada, ya que el contexto tecnológico cambia y exige que todo código sea mantenido para que siga funcionando.

Poder actualizar sus herramientas es parte integral de la condición de los creadores de herramientas. En efecto, la responsabilidad que asumen con sus herramientas necesita ser atenuada o contrarrestada por otros factores, en particular la posibilidad de corregir sus errores. Como mencionamos antes, algunos investigadores tienen miedo de compartir su código (Jiménez et al., 2017; Kuzak, 2021), hasta el punto de que algunos incluso ocultan el hecho mismo de que programan (Yehudi, 2021). Plataformas como GitHub permiten a los creadores de herramientas sostener una relación viva con sus usuarios: actualizar su código y su documentación, recibir y responder comentarios, etc. Como las herramientas nunca están realmente terminadas, los creadores de herramientas necesitan un espacio para contextualizarlas y negociar su responsabilidad. Esto puede significar, por ejemplo, reconocer que una herramienta tiene un error o un problema, pero que sus creadores no tienen el tiempo o los recursos para resolverlo.

Que todo prototipo sea una herramienta y que toda herramienta sea un prototipo se refuerzan mutuamente. El hecho de que los prototipos tengan éxito no incentiva a los creadores de herramientas a invertir en cajanegrizarlos para convertirlos en productos terminados. Además, que el inacabamiento se vuelva la norma facilita dar a conocer un prototipo desde muy temprano.

La mayoría de las herramientas DC para las SSH nacen de la práctica, no porque las herramientas de otros orígenes tengan menos probabilidades de éxito, sino porque todo proyecto de software crece de manera continua a partir de una pieza mínima. En el contexto de las DCSSH, esa pieza mínima suele ser una experimentación artesanal para un experimento científico. Esa experimentación artesanal se convierte en una herramienta porque encuentra un público, lo que obliga a los creadores de herramientas a emerger como autores y a gestionar su relación con ese público. Esto obliga a las herramientas a adquirir una identidad, un propósito, límites más claros, y a empezar a convertirse en productos (nunca terminados).

DENOMINACIÓN Y GESTIÓN DE LA IDENTIDAD EN LA CREACIÓN DE HERRAMIENTAS

> En ese momento todavía no se llamaba Voyant, sino Voyeur. En cierto momento nos dimos cuenta de que la palabra "voyeur" en inglés no tiene ninguna connotación positiva. En francés, está bien. Un poco mejor, en todo caso. Pero en inglés … Así que cambiamos a Voyant. De hecho, algunos buenos amigos nuestros nos dijeron amablemente: esta no es la palabra adecuada para lo que están haciendo. Nos encanta, pero no es la palabra adecuada. (Geoffrey Rockwell, Jacomy, 2023)

Crear una herramienta implica comprometerse con un nombre, una identidad visual, un sitio web y muchas otras cosas que tejen la relación con el público. Esto a veces toma por sorpresa al creador de la herramienta cuando surge la necesidad de comunicar acerca de una pieza de software que hasta entonces no tenía nombre, por ejemplo, cuando decide escribir un artículo sobre ella. La sorpresa puede llegar de otra manera, quizá más extendida, cuando la herramienta tiene un nombre provisorio y el creador de la herramienta se da cuenta, demasiado tarde, de que el público emergente ya se refiere a la herramienta de ese modo. El creador de la herramienta debe entonces sopesar si cambiar el nombre o no.

El nombre de la herramienta importa por múltiples razones. Por supuesto, el nombre cumple el propósito de la comunicación, y puede ser más o menos apto para transmitir el mensaje correcto. Pero un nombre puede ser inadecuado por razones más prácticas. El problema más importante podría ser que el nombre ya se refiera a otra herramienta, a veces similar. Esto es más común de lo que se podría pensar en la ciencia, porque la cultura académica suele incentivar al creador de la herramienta a mantenerse descriptivo y fáctico, y alguien más tuvo exactamente el mismo razonamiento en otro lugar. De manera similar pero distinta, si el nombre es una palabra común, impedirá que las personas la encuentren fácilmente en un motor de búsqueda web, como ocurre, por ejemplo, con GUESS, de Eytan Adar (Adar, 2006). Otros problemas menores incluyen la internacionalización, los términos culturalmente sensibles, etc. Elegir un nombre óptimo no es muy importante para una herramienta de las DCSSH, pero evitar complicaciones innecesarias es valioso para el creador de la herramienta; una de esas complicaciones es cambiar el nombre demasiado tarde, lo que puede implicar reescribir una serie de documentos, migrar sitios web enteros y hacer que los artículos académicos que citan el nombre antiguo parezcan obsoletos. Por ejemplo, Jupyter Notebooks (Kluyver et al., 2016) se llamaba inicialmente IPython Notebooks, pero «se escindió de IPython en 2014 por obra de Fernando Pérez y Brian Granger» (Wikipedia contributors, 2024; véase también Pérez & Granger, 2007).

Gephi fue prototipado inicialmente por Jacomy como «Graphiltre», la contracción de «graph» y «filter», ya que se trataba de visualizar grafos y filtrarlos al mismo tiempo. Posteriormente pasó a manos de Bastian, quien lo redesarrolló como una pieza de software más robusta. En ese momento, empezó a llegar a un público amplio, organizar la comunicación del proyecto se volvió una necesidad y se decidió buscar un nombre mejor y más definitivo. Ese paso fue también una forma de que el equipo se reapropiara colectivamente del proyecto. Se

decidió conservar el concepto, pero simplificarlo: «graph» quedó resumido en la letra G, y «filter» en la letra griega Phi, que también constituye el logo de la herramienta hasta el día de hoy. Como el equipo era francés, pronunciaban esas dos letras «jé» y «fee» y, por lo tanto, las escribían «Géphi». El acento se eliminaría más tarde para no confundir a un público internacional. Los angloparlantes que descubrían la herramienta no sabían cómo pronunciarla, porque su documentación estaba entonces esencialmente en forma de texto, y surgiría una pronunciación diferente en la que la G se pronuncia como en «graph». Hasta el día de hoy, Gephi se pronuncia de manera diferente en Francia que en otros países.

La identidad no es necesariamente fácil de decidir, pero la pregunta más difícil podría ser quién decide la identidad, cuándo y cómo. Fijar la identidad es una forma, quizá la forma principal, de reclamar la autoría. Esto se debe a la manera en que las piezas de software llegan a existir. Como el proceso es gradual, no hay un punto de partida claro. Y como siempre implica código existente, el comienzo siempre parece un acto banal de reunir cosas existentes. Los creadores de herramientas no tienen un momento claro de alumbramiento; pero sí tienen un momento de partida de nacimiento, y es el de fijar (o restablecer) la identidad de la herramienta. Cambiar la identidad de una herramienta realiza un restablecimiento de la autoría, porque es el punto de su desarrollo que será relevante para distinguir la nueva herramienta de la antigua, por ejemplo, para diferenciar Jupyter de IPython. Fijar la identidad de una herramienta es un instrumento de poder para los múltiples actores en juego.

El creador de la herramienta puede tener que lidiar con al menos otro actor potencial: su institución académica o su empresa. Si la herramienta se construyó durante el horario de trabajo, ¿pertenece a la universidad? Si fue financiada por un proyecto, ¿pertenece a quien lo financió? A medida que una herramienta se vuelve popular, surgen nuevas preguntas: cómo financiar el proyecto, cómo mantenerlo y en qué dirección llevarlo. Si el creador de la herramienta no está alineado con su institución anfitriona, pueden surgir problemas dolorosos. A la inversa, el creador de la herramienta puede aprovechar proactivamente la identidad de su herramienta para enrolar a otros actores. Por ejemplo, el «DMI» de DMI-TCAT remite a la Digital Methods Initiative, el grupo de investigación donde se desarrolló. El nombre OpenRefine tiene una historia similar. Creada inicialmente con el nombre de Freebase Gridworks, fue «adquirida» por Google (OpenRefine, 2024) y «la herramienta pasó a llamarse "Google Refine"» (ibid). Luego, «Google transfirió el liderazgo del proyecto a la comunidad de código abierto, que la renombró "OpenRefine" y la trasladó a GitHub» (ibid). La presencia de «Google» en el nombre era una marca de propiedad que tuvo que eliminarse posteriormente.

Si algunos creadores de herramientas tienen dificultades para cambiar el nombre de la herramienta, otros las tienen para conservarlo. Esto ocurre, en particular, cuando una herramienta cambia y desborda su alcance o su concepto iniciales. Por ejemplo, 4CAT (Peeters and Hagen, 2022) se creó para analizar el foro web 4chan, pero desde entonces se ha convertido en una navaja suiza de fuentes de datos y análisis. 4CAT solía significar «4chan Capture and Analysis Toolkit» (p. ej. como se cita en Rogers, 2023), siguiendo el esquema de denominación

de DMI-TCAT, donde la primera T corresponde a Twitter. Posteriormente, 4CAT se abrió a varias otras plataformas en línea, y el nombre pasó a ser potencialmente engañoso. Los creadores de la herramienta decidieron conservar el nombre, pero reinterpretar su significado. El título que muestra actualmente la herramienta es «4CAT: Capture and Analysis Toolkit» (4CAT, 2018) y su logo son cuatro gatos, una forma de ofrecer una nueva justificación para el 4 del nombre.

La cuestión de la denominación muestra que los creadores de herramientas suelen verse llevados a convertirse en gestores de producto, a veces en contra de su voluntad. Sin embargo, las políticas de identidad no se detienen en la vida simbólica de la herramienta. También impregnan sus funcionalidades y su contenido científico, ya que deben tomarse decisiones sobre su alcance y su conjunto de funcionalidades.

AGREGAR Y QUITAR FUNCIONALIDADES

> Voyant no hace algunas cosas que me gustaría que hiciera. Es un blanco móvil, sobre todo cuando se intenta hacer algo que sea, en cierto sentido, actual. (Geoffrey Rockwell, Jacomy, 2023)

La vida de una herramienta está pautada, en particular, por la incorporación y, a veces, la eliminación de funcionalidades. Lo que la herramienta puede hacer decide en gran medida cómo debe construirse y quién la usará. Es una elección editorial que hacen los creadores de herramientas y, en el caso de las herramientas DC para las SSH, también una serie de compromisos metodológicos. La herramienta debe cambiar bajo la doble influencia de la evolución del conocimiento científico y de las propias visiones científicas de los creadores de herramientas.

Las elecciones editoriales sobre el conjunto de funcionalidades están condicionadas por influencias contrapuestas. El creador de herramientas puede querer seguir un nuevo estado del arte, pero carecer del tiempo o de las habilidades. El creador de herramientas también puede desatender intencionadamente el consenso científico en beneficio de su propio método, si ve en su herramienta un vehículo para su propia influencia académica. Y el creador de herramientas puede verse obligado a tomar decisiones por razones prácticas, con independencia del valor científico de algunas funcionalidades: porque algunas funcionalidades dependen de otras funcionalidades, o son incompatibles con otras funcionalidades; o porque facilitan el mantenimiento, o lo dificultan. Por ejemplo, los autores de Gephi eliminaron varias funcionalidades, como la compatibilidad con grafos jerárquicos, de la versión 0.9.0 en 2015 (Gephi team, 2022), porque encarecían demasiado el mantenimiento. Otra situación habitual es la incorporación de una funcionalidad por la única razón de que se la prometió en una subvención académica. Buena parte de la vida de la herramienta se decidirá por la negociación del conjunto de funcionalidades que lleve a cabo el creador de herramientas, regida tanto por la necesidad de sostener el proyecto mediante el financiamiento y el mantenimiento como por el deseo de dar a la herramienta una forma determinada.

He aquí otro ejemplo del proyecto Gephi: la funcionalidad de deshacer. Los usuarios cometen errores y, en prácticamente cualquier software corriente, la funcionalidad de deshacer les permite volver al estado en que estaba su trabajo antes del error. Esa funcionalidad es estándar y omnipresente; sin embargo, está ausente en Gephi. Los usuarios la han pedido desde que existe. Los creadores de herramientas lo saben, pero la funcionalidad es sencillamente demasiado difícil de implementar como para llegar a lo más alto de la lista de prioridades, dado el tiempo que pueden dedicar al proyecto. La situación cambió cuando un grupo de investigadores de los Países Bajos trabajó con un desarrollador para desarrollar el Fieldnotes Plugin (Wieringa et al., 2019), una extensión de Gephi que permite al usuario registrar automáticamente su actividad dentro de la herramienta en notas de campo que documentan su proceso de análisis de redes. Resulta que construir una funcionalidad de deshacer exige, de manera similar, llevar un registro de cada acción del usuario. Algunas herramientas incluso fusionan el registro de actividad y la funcionalidad de deshacer, por ejemplo, OpenRefine. Durante el encuentro anual de programación de Gephi en 2021, los creadores de herramientas trabajaron en esa funcionalidad con el responsable del mantenimiento de OpenRefine, Antonin Delpeuch. En su informe, escriben: «Aprendimos cómo se hace actualmente [la funcionalidad de deshacer/rehacer] en OpenRefine, los problemas que genera y cómo podría hacerse mejor. La funcionalidad es factible en Gephi, y esbozamos una arquitectura» (Gephi team, 2021). El proyecto Gephi se benefició de una retroalimentación interesante de otro proyecto similar. Mientras escribimos (verano de 2024), la funcionalidad de deshacer sigue en desarrollo. La sesión de trabajo de 2021 ayudó a descomponer la funcionalidad en subproyectos más pequeños que pueden implementarse uno tras otro.

LA CARGA DEL MANTENIMIENTO

> En cierto momento, nos dimos cuenta de que [nuestro modelo actual] era demasiado difícil de manejar. Tendríamos que conseguir $1,000,000 al año para mantener el desarrollo, y eso no iba a ser sostenible con el tipo de financiamiento que se obtiene en las humanidades. Así que … lo dividimos en partes más pequeñas que pudiéramos sostener. … Y en cierto momento nos dimos cuenta: ¡esto es muy popular! Ya no tenemos el problema de que nadie lo use. Ahora tenemos el problema de cómo mantenerlo. (Geoffrey Rockwell, Jacomy, 2023)

Toda tecnología se degrada sin mantenimiento, incluso la digital. «El software se pudre con el tiempo. No porque el código en sí se deteriore, sino porque el entorno en el que se ejecuta cambia inevitablemente» (Rieder et al., 2024). Sin embargo, como el mantenimiento es invisible, se puede pensar en él como una rutina diaria en la que el responsable del mantenimiento comprueba que se preserve el statu quo. Lamentablemente, casi nunca es así. El mantenimiento adopta muchas formas, y el responsable del mantenimiento casi no tiene voz sobre lo que exigirá, cuánto costará y cuándo requerirá intervención. El mantenimiento da forma a la herramienta al obligar al creador de herramientas a tomar decisiones radicales o a permitir que la herramienta se rompa y se degrade.

El mantenimiento es cuidado bajo múltiples formas, tanto en el sentido de preocuparse por algo como en el de ocuparse de algo. «Como han observado muchos comentaristas, el cuidado … se refiere tanto a una disposición mental de preocupación como a las prácticas concretas en las que nos involucramos como resultado de esas preocupaciones» (Tronto, 1998). Siguiendo a Joan Tronto, Rieder et al. (2024) proponen mirar al creador de herramientas desde el ángulo de la «ética del cuidado». Siguiendo a Annemarie Mol (2008), entendemos el cuidado como un conjunto de prácticas sujetas a una lógica de interdependencia, en la que el creador de herramientas a veces debe hacer lo que se necesita más que lo que se desea (por él mismo, por los usuarios o por las autoridades científicas). El diseño y el mantenimiento de herramientas implican un cuidado dirigido en primer lugar a la herramienta, pero también, a través de ella, a los métodos y a otras diversas actividades científicas, a la comunidad de investigación, a otros usuarios, a los propios creadores de herramientas y a sus seres queridos (laboratorio, personal informático, colegas creadores de herramientas …). El mantenimiento se orienta a atender sus necesidades en diversos grados.

El mantenimiento se diferencia de la reparación en que no constituye un acontecimiento. La reparación puede dramatizarse como un relato heroico, pero el mantenimiento es invisible (Denis & Pontille, 2022). La película de 1985 de Terry Gilliam, Brazil, tiene como personaje heroico central a Archibald «Harry» Tuttle, un técnico de calefacción independiente que repara ilegalmente los sistemas de aire acondicionado de la gente en lugar de los incompetentes trabajadores de Central Service, lo que lleva a que se lo acuse de terrorismo. Tuttle es heroico porque su intervención marca una diferencia visible; devuelve el sistema de un estado averiado a uno funcional. Por el contrario, el mantenimiento evita que el sistema se rompa, lo que no tiene ningún efecto aparente. Como Tuttle, el reparador viene de afuera, arregla y se va; constituye una asistencia externa. El responsable del mantenimiento vive en contacto con la cosa mantenida y se lo da por sentado como si formara parte de ella. El mantenimiento no tiene héroes.

El mantenimiento no puede tener héroes porque no se deja dramatizar de este modo. Aunque esto trae aparejados problemas, no es necesariamente un inconveniente para los creadores de herramientas. En efecto, los creadores de herramientas no son forasteros que irrumpen para arreglar las cosas con sus poderes superiores. Más bien, si efectivamente son investigadores-programadores, viven en estrecha proximidad con la situación en la que opera su herramienta, y por eso pudieron crearla en primer lugar.

La invisibilidad del mantenimiento no es un problema en sí misma, y los creadores de herramientas no desean necesariamente ser héroes, pero aun así conlleva una preocupación importante: el mantenimiento es muy difícil de financiar. Las instituciones académicas aceptan financiar una nueva funcionalidad o la corrección de un problema, pero no el trabajo invisible de evitar, ante todo, que las herramientas se rompan. Desarrollaremos este punto en el capítulo 24. Dejar que una herramienta se rompa antes de arreglarla puede ser productivo para el creador de herramientas, ya que le recuerda al público la necesidad de cuidar la herramienta. Esto puede

permitir que se lo dramatice heroicamente, pero al mismo tiempo la rotura también pone de manifiesto un fallo del mantenimiento y puede llevar al público de la herramienta a desconfiar de ella. Desde el punto de vista del creador de herramientas, la solución sostenible consiste en convencer a los organismos de financiamiento de que financien el mantenimiento de las herramientas pese a su falta de visibilidad, algo muy difícil de lograr.

Gephi sufrió una falla masiva y prolongada de 2013 a 2015. El problema surgió porque versiones (entonces) recientes de macOS exigían una actualización de Java que era incompatible con el núcleo de Gephi, una parte crítica de la herramienta. Gephi simplemente se negaba a ejecutarse. El núcleo no podía arreglarse fácilmente por una razón específica: la energía de los desarrolladores estaba concentrada en reescribir el núcleo por completo. Como explican los creadores de herramientas, «en marzo de 2013 había llegado el momento de pensar cómo sería Gephi 1.0 y de darse cuenta de que necesitaba un nuevo núcleo. Este fue, con mucho, el proyecto más complejo que el equipo tuvo que superar, pero los desarrolladores tenían una visión a largo plazo y [sabían] que los desarrollos futuros se apoyarían ahora en un núcleo robusto y extensible, con un rendimiento de primer nivel mundial» (Gephi team, 2015). Arreglar el núcleo antiguo no solo llevaría tiempo y esfuerzo, sino que también perjudicaría el avance de la herramienta y postergaría el lanzamiento de la nueva versión. Por lo tanto, el equipo decidió concentrarse en lanzar la nueva versión lo antes posible. Lamentablemente, lanzar el nuevo motor llevó unos dos años, durante los cuales la situación empeoró. A comienzos de 2013, solo unos pocos estudiantes con la versión más reciente de macOS tenían problemas con Gephi, y aquello podía pasar por una excepción, un error temporal. Pero a finales de 2015, la mayoría de los estudiantes con una Mac se encontraban con el problema de incompatibilidad. Esto desató el rumor de que el desarrollo de Gephi había sido abandonado, y algunos docentes que usaban esa herramienta empezaron a pasar sus cursos a alternativas. El lanzamiento de la versión 0.9 en diciembre de 2015 corrigió el problema y trajo otras mejoras, lo que permitió que el público volviera a confiar en el proyecto, aunque quizá con mayor conciencia de su precariedad. También llevó a los creadores de herramientas a repensar las condiciones de su propia resiliencia. En 2018 escribieron: «no podemos simplemente producir una versión de Gephi que funcione bien y dejarla así. Dejaría de funcionar porque la tecnología cambia. … Lamentablemente, cuando surgen incompatibilidades, por lo general les corresponde a los desarrolladores del núcleo ocuparse de ellas. … No estamos seguros de que podamos seguir el ritmo de estos cambios de manera eficiente debido a nuestro liderazgo técnico limitado» (Gephi team, 2018a). Desde entonces, el equipo ha estado trabajando para estabilizar a los responsables del mantenimiento del proyecto.

Como señalan Denis y Pontille (2022), lo mantenido no es aquello que no se rompe, sino aquello que se deja mantener. La mantenibilidad de una herramienta depende de la claridad de su código, de la disponibilidad de los desarrolladores familiarizados con ese código y de la flexibilidad de la infraestructura de desarrollo de software. También depende de la capacidad de los responsables del mantenimiento para atender las necesidades de la herramienta, es decir, para

detectar problemas antes de que se conviertan en errores que la rompan y afecten a los usuarios. La más típica de esas necesidades es tener las dependencias de software actualizadas; pero también, a veces, no actualizar a la última versión si no es lo bastante estable. Los creadores de herramientas suelen trabajar en una versión de la herramienta más reciente que la última publicada, pero solo la publican cuando han validado su estabilidad. Con bastante frecuencia, usan un sistema de pruebas automatizadas para comprobar que las intervenciones en el código no rompan las funcionalidades preexistentes; tienen más conocimiento del código base que un usuario corriente. Si usan un sistema de notificación de fallos, también reciben un aviso cuando surge un nuevo problema en las máquinas de los usuarios. Esto les permite atender continuamente la fragilidad de la herramienta.

Muchas herramientas DC para las SSH son precarias, en particular porque a los creadores de herramientas no se les proporcionan los recursos para evitar que surjan problemas. En consecuencia, el mantenimiento «también es "entrecortado" y requiere intervenciones intermitentes y difíciles de anticipar, a menudo con un alto grado de urgencia» (Rieder et al., 2024). Las averías y las reparaciones no son simplemente una parte molesta del mantenimiento de las herramientas de las DCSSH; constituyen oportunidades de cambio que marcan el ritmo de la evolución de la herramienta. La naturaleza del mantenimiento tiene un impacto en los métodos disponibles para los investigadores de las SSH.

En efecto, la reparación y el mantenimiento siempre tienen un aspecto ontológico (Denis and Pontille, 2022). La herramienta mantenida nunca queda inalterada. Esto va en contra de la noción, extendida entre los investigadores de las DCSSH, de que su aparato consiste en cierto modo en productos terminados, publicados con esmero porque sus conjuntos de funcionalidades reflejan las intenciones de los creadores de herramientas. Esta concepción ignora la resistencia material del entorno informático, tanto del hardware como del software. No solo el diseño de la herramienta está condicionado, sino que cada acto de mantenimiento puede imponer una transformación. Reparar y mantener nunca consiste realmente en volver al estado previo a la rotura. Para los creadores de herramientas, no hay statu quo; no hay criterios estables.

He aquí otro ejemplo más del proyecto Gephi. Su motor gráfico depende de una biblioteca de código (JOGL) que dejó gradualmente de mantenerse hacia 2021. Empezó a introducir errores para algunos usuarios que los creadores de herramientas de Gephi no podían resolver porque eran internos a esa biblioteca. Era imposible reemplazar esa biblioteca sin reescribir el motor gráfico, pero no había otra solución. Los creadores de herramientas empezaron a reescribir el motor, lo que ofreció dos oportunidades: usar técnicas más recientes para mejorar el rendimiento y estructurar el código de manera que fuera más fácil cambiar de biblioteca en el futuro. Reescribir el motor llevó unos años, y se volvió utilizable en 2024. En un extraño giro del destino, la biblioteca problemática volvió a tener mantenimiento durante ese tiempo, y el equipo de Gephi decidió finalmente seguir usándola en el nuevo motor. Toda esta empresa puede verse como una operación de mantenimiento, destinada a evitar que Gephi se rompiera

por una dependencia de código rota. Pero no se limitó a preservar el statu quo de la herramienta; lo cambió con un impacto notable tanto para los creadores de herramientas como para los usuarios, ojalá para mejor.

LAS HERRAMIENTAS NO DURAN PARA SIEMPRE

> Tengo la sospecha de que los días de Voyant pueden estar llegando a su fin, de que podría ser el regreso de la línea de comandos o de la línea de prompt. Ese se convertirá en el nuevo paradigma para el análisis de datos. Y no será solo Voyant, será Tableau, serán todo tipo de herramientas que serán reemplazadas por estas nuevas herramientas [que usan IA generativa]. (Geoffrey Rockwell, en Jacomy, 2023)

Las herramientas no duran para siempre, aunque solo sea porque requieren mantenimiento. El código puede permanecer inalterado, pero el resto del ecosistema sigue evolucionando; así, «el software se pudre con el tiempo» (Rieder et al., 2024). La infraestructura digital cambia lentamente y, sin cuidado, el software se degrada.

Algunos tipos de herramientas pueden ser más resilientes que otros porque algunas partes de la infraestructura son más estables que otras. Desde el punto de vista de los creadores de herramientas, usar muchas bibliotecas es preferible a reinventar la rueda, pero también los coloca en una posición difícil. Las dependencias son vulnerabilidades, como hemos visto. Las bibliotecas también dificultan el proceso de instalación, en particular cuando se requieren versiones específicas de esas bibliotecas (y cada biblioteca depende de otras bibliotecas, y así sucesivamente). Algunas herramientas se construyen sobre plataformas que atenúan los problemas de incompatibilidad de diversas maneras. Java (usado por Gephi, VOSviewer y OpenRefine) promete un despliegue multiplataforma (macOS, Windows, Linux …). Docker (usado por Hyphe) promete un control total sobre el entorno de paquetes. La web como plataforma de aplicaciones (4CAT, DMI-TCAT, YouTube Data Tools, The Lippmannian Device …) promete un acceso universal desde el navegador (una promesa que no necesariamente se cumple). Esas estrategias de resiliencia, y otras, siempre tienen un costo en complejidad del código. Aliviar la carga del mantenimiento haciendo que la herramienta sea más resiliente a las averías también hace que el código sea más difícil de reapropiar por parte de otros desarrolladores. La industria del software ha desarrollado estándares y buenas prácticas para garantizar que el código pueda reapropiarse, pero solo los desarrolladores con formación profesional los dominan. Es más probable que un ingeniero de software de investigación (RSE) posea esas habilidades que un investigador-creador de herramientas autodidacta. La idiosincrasia del código base de muchas herramientas DC crea una dependencia respecto del creador de herramientas. Por ello, aunque el código fuente sea abierto, el creador de herramientas puede no ser capaz de transmitir su herramienta a otras personas, o al menos no sin un esfuerzo considerable.

De hecho, las herramientas rara vez sobreviven a la pérdida de sus creadores. Transmitir una herramienta a otros es difícil porque la creación de herramientas es, sobre todo, un deber. A

menos que el creador de herramientas logre sumar a otros para que se reapropien del proyecto y lo hagan suyo después de él, cuando abandona la herramienta, esta se degrada y se vuelve inutilizable; muere.

> Para ser sincero, ahora mismo estoy cerca de jubilarme. Lo que siento ahora mismo es que quiero encontrar una manera de traspasar esto con elegancia a otra persona. … He puesto mucho trabajo y he conseguido algo de financiamiento, así que probablemente tengo dinero suficiente para cinco años, y voy a usar esos cinco años para traspasarlo; o dejarlo morir. (Geoffrey Rockwell, Jacomy, 2023)

EL CÓDIGO ABIERTO NO GARANTIZA LA SOSTENIBILIDAD

La cuestión de la degradación del software atraviesa todo el movimiento del software de código abierto. Aunque la infraestructura de código abierto permite compartir y transmitir el código, no garantiza que alguien retome el proyecto a partir de ahí y lo lleve más lejos. No hay ahí afuera una multitud de responsables del mantenimiento esperando código gratuito que mantener.

Los investigadores que no están familiarizados con la comunidad del código abierto pueden tener expectativas erróneas. La infraestructura de código abierto —por ejemplo, GitHub— es una necesidad para compartir un proyecto, pero no es suficiente. La mayoría de las veces, los programadores la usan para compartir su trabajo dentro de una comunidad ya establecida. Aunque contar con esa infraestructura abierta ayuda a encontrar nuevos colaboradores, la mayor parte del trabajo de reclutamiento tiene que hacerse en otros lugares, como congresos y talleres.

> A veces se sugiere "abrir el código" del software como solución [al problema del mantenimiento], partiendo del supuesto de que otros contribuirán o continuarán el trabajo siempre que el código fuente y la documentación del proyecto estén disponibles. Esto pasa por alto el conocimiento y la experiencia sustanciales acumulados durante el trabajo en una herramienta, que alimentan directamente el desarrollo posterior. También pasa por alto que los proyectos de código abierto que funcionan bien suelen tener un liderazgo o una gestión comunitaria fuertes y proactivos. (Rieder et al., 2024)

Para proyectos tan de nicho como las herramientas DC para las SSH, la aparente promesa de conseguir nuevos colaboradores mediante la infraestructura de código abierto es en gran medida una ilusión. Sin embargo, en los proyectos más populares a veces ocurre, como un acontecimiento raro y valioso. El equipo de Gephi ha intentado atraer desarrolladores desde el comienzo del proyecto, y de manera más proactiva desde 2021. Ningún desarrollador se ha sumado al núcleo del proyecto en años, por razones que se han identificado en gran medida (Jacomy & Bastian, 2023): Java es una tecnología antigua que no resulta atractiva para los desarrolladores jóvenes; los desarrolladores sénior de Java de la comunidad del código abierto esperan unos ingresos que el proyecto Gephi no puede permitirse (por ahora); los investigadores-creadores de herramientas que tienen el tiempo y la buena voluntad para contribuir tienen las competencias para crear y mantener un plugin, pero no el núcleo del software. La estrategia actual del proyecto es retener a los desarrolladores de plugins e introducirlos en el código base del núcleo, de modo que la mayor cantidad posible de personas

tenga el conocimiento necesario para intervenir si hace falta. Dicho esto, hay una excepción importante a esta situación. El actual desarrollador principal de Gephi, Eduardo Ramos Ibáñez, llegó al equipo al postularse en 2010 al Google Summer of Code, una infraestructura creada por Google para patrocinar una selección de proyectos de código abierto financiando a estudiantes para que trabajen en el código durante las vacaciones de verano. Eduardo no fue el estudiante seleccionado para el proyecto, pero siguió involucrado y continuó contribuyendo, incluso sin financiamiento. Diseñó partes importantes de la herramienta a lo largo de los años, en particular el «data laboratory» (una vista de hoja de cálculo de los nodos y las aristas). Llegó a dominar el código base de Gephi y se convirtió en el desarrollador principal en 2018 (Gephi team, 2018b). Sin embargo, el caso de Gephi es atípico. Muchos proyectos de código abierto valiosos no logran atraer ni a un solo desarrollador.

Al escribir que «el código abierto no garantiza la sostenibilidad», queremos subrayar dos cosas. En primer lugar, aunque la sostenibilidad es deseable, hay mejores maneras de lograrla que abrir el código. Esas maneras son compatibles con el código abierto, pero en muchos aspectos independientes de él. Volveremos sobre esto en el próximo capítulo. En segundo lugar, el propósito de abrir las fuentes de una herramienta no debería ser garantizar la sostenibilidad. Debería ser permitir el escrutinio metodológico, fomentar la reproducibilidad y defender el código como bien común. El carácter abierto del código de muchas herramientas DC para las SSH debería derivarse del movimiento de la ciencia abierta. Abrir el código con el propósito de atraer trabajo de mantenimiento gratuito no solo es ingenuo; también está condenado al fracaso y es contraproducente. Como saben todos los colaboradores del código abierto, incorporarse a un proyecto requiere mucho más que la capacidad de descargar el código. Es ante todo un proceso social. Abrir el código fuente tiene muchos beneficios por sí mismo (permite el escrutinio, facilita el trabajo en equipo, incentiva las buenas prácticas), pero no basta para que una herramienta sea sostenible.

> Sólo sé de un equipo que haya entrado y hecho algo con el código, que quería cambiar algo y luego volvió a nosotros. (Geoffrey Rockwell, en Jacomy, 2023)

REFERENCIAS

4CAT (2018). OILab. https://4cat.oilab.nl/ (accessed 29/08/2024).

Adar, E. (2006). GUESS: a language and interface for graph exploration. In Proceedings of the SIGCHI conference on Human Factors in computing systems (pp. 791–800).

Denis, J., & Pontille, D. (2022). Le soin des choses: Politiques de la maintenance. La Découverte.

Gephi team (2015). Gephi 0.9 released: Play with network data again. Gephi Blog. https://gephi.wordpress.com/2015/12/21/gephi-0-9-released-play-with-network-data-again/ (accessed 30/08/2024).

Gephi team (2018a). Is Gephi obsolete? Situation and perspectives. Gephi Blog. https://gephi.wordpress.com/2018/11/01/is-gephi-obsolete-situation-and-perspectives/ (accessed 30/08/2024).

Gephi team (2018b). Meet Eduardo, our new lead developer. Gephi Blog. https://gephi.wordpress.com/2018/11/01/meet-eduardo-our-new-lead-developer/ (accessed 30/08/2024).

Gephi team (2021). Gephi code sustainability retreat 2021: Debriefing. Gephi Blog. https://gephi.wordpress.com/2021/12/06/gephi-code-sustainability-retreat-2021-debriefing/(accessed30/08/2024).

Gephi team (2022). Releases. GitHub. https://github.com/gephi/gephi/wiki/Releases#removed-features(accessed 29/08/2024).

Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing.

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. Reticular. https://reticular.hypotheses.org/8039

Jacomy, M., & Bastian, M. (2023). Gephi towards v1.0: The codebase, and the rest. FOSDEM 2023. https://archive.fosdem.org/2023/schedule/event/graph_gephi_future/ (accessed 29/08/2024).

Jiménez, R. C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., Capella-Gutierrez, S., Hong, N. C., Cook, M., Corpas, M., Flannery, M., Garcia, L., L. I. Gelpí, J., Gladman, S., Goble, C., Ferreiro, M. G., Gonzalez-Beltran, A., Griffin, P. C., Grüning, B., ... & Crouch, S. (2017). Four simple recommendations to encourage best practices in research software. F1000Research, 6.

Katz, D. S., & Stall, S. (2021). Software citation. Information Services & Use, 41(1–2), 39–42. https://doi.org/10.3233/ISU-210108

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter Notebooks–a publishing format for reproducible computational workflows. In B. Schmidt & F. Loizides (Eds.), Positioning and power in academic publishing: Players, agents and agendas (pp. 87–90). IOS Press.

Kuzak, M. (2021). On the road to sustainable research software. FOSDEM 2021, communication in the Open Research devroom (video). https://archive.fosdem.org/2020/schedule/event/open_research_sustainable_soft/ (accessed 07/05/2024).

Mol, A. (2008). The logic of care: Health and the problem of patient choice. Routledge.

Neuroth, H., Lohmeier, F., & Smith, K. M. (2011). Textgrid–virtual research environment for the humanities. International Journal of Digital Curation, 6(2), 222–231.

OpenRefine (2024). OpenRefine Project History. https://openrefine.org/openrefine_history (accessed 20/08/2024).

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. Computational Communication Research, 4(2), 571–589.

Pérez, F., & Granger, B. E. (2007). IPython: A system for interactive scientific computing. Computing in science & engineering, 9(3), 21–29.

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. Convergence, 30(1), 216–235. https://doi.org/10.1177/13548565221127094

Rogers, R. (2023). The propagation of misinformation in social media: A cross-platform analysis. Amsterdam University Press.

Social Sciences and Humanities Open Marketplace (2023). Webpage. https://marketplace.sshopencloud.eu/ (accessed 27/08/2024).

SSH vocabulary service: Invocation type (2023). https://vocabs.sshopencloud.eu/vocabularies/invocation-type/invocationTypeScheme (accessed 2024-08-27).

Star, S. L. (1999). The ethnography of infrastructure. American Behavioral Scientist, 43(3), 377–391.

Tronto, J. C. (1998). An ethic of care. Generations: Journal of the American society on Aging, 22(3), 15–20.

van Es, K., Schäfer, M. T., Wieringa, M. (2021). Tool criticism and the computational turn. A "methodological moment" in media and communication studies. Medien & Kommunikationswissenschaft, 69(1), 46–64.

Veršić, I. I., & Ausserhofer, J. (2019). Social sciences, humanities and their interoperability with the European Open Science Cloud: What is SSHOC? Mitteilungen Der Vereinigung Österreichischer Bibliothekarinnen Und Bibliothekare, 72(2), 383–391.

Wieringa, M., van Geenen, D., van Es, K., & van Nuss, J. (2019). The field notes plugin: Making network visualization in Gephi accountable. In A. Daly, K. Devitt, & M. Mann (Eds.), Good Data. INC Theory on Demand. Available at: http://networkcultures.org/blog/publication/tod-29-good-data/(accessed 30/08/2024).

Wikipedia contributors (2024). Project Jupyter. Wikipedia. https://en.wikipedia.org/wiki/Project_Jupyter (accessed 29/08/2024).

Yehudi, Y. (2021). Open research in life science: Funding foundational tools, trust, and talent. FOSDEM 2021, communication in the Open Research devroom (video). https://archive.fosdem.org/2021/schedule/event/open_research_life_science/ (accessed 07/05/2024).

24. Valorar la creación de herramientas en la academia

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Este capítulo aborda la compleja relación entre la creación de herramientas y la academia en las DCSSH. Examinamos cómo el sistema académico tiene dificultades para reconocer y recompensar adecuadamente el trabajo de desarrollar y mantener herramientas de investigación, pese a su creciente importancia para el trabajo académico. El desafío fundamental reside en que el trabajo con herramientas a menudo no logra ser reconocido como un logro académico, lo que genera un problema de trabajo oculto que afecta tanto a la creación como al mantenimiento de las herramientas de investigación. Nos proponemos ofrecer consejos a los aspirantes a creadores de herramientas que deben sortear estos desafíos y, al mismo tiempo, concientizar a los usuarios de herramientas acerca del trabajo, a menudo invisible, que sostiene nuestra infraestructura de investigación. Esperamos que, al sensibilizar a los usuarios de herramientas respecto de los desafíos sistémicos inherentes al desarrollo y el mantenimiento de herramientas en la academia, podamos cultivar colectivamente un mejor entorno para la investigación en las DCSSH.

La resistencia del entorno académico a la creación de herramientas ha dado lugar a diversas estrategias de adaptación. Exploramos cómo los creadores de herramientas readaptan mecanismos académicos tradicionales, como la publicación y el financiamiento mediante subvenciones, para legitimar su trabajo, y mostramos tanto las posibilidades como las limitaciones de estos enfoques. Sin embargo, la situación no es estática: en los últimos años se han producido cambios significativos en la manera en que las instituciones valoran y apoyan el desarrollo de software de investigación, y también analizaremos estas tendencias en evolución y su posible impacto en el futuro de la creación de herramientas en la academia.

LAS OPERACIONES NO PASAN POR TRABAJO ACADÉMICO

> Stéfan [Sinclair] y yo tuvimos que pasar a un enfoque más profesional, orientado a la producción. Y, por supuesto, la academia no recompensa la producción. No puedo sentarme allí y decir: acabamos de lanzar Voyant 2.3, y puede haber sido tanto trabajo como tres artículos. Obtuve reconocimiento por Voyant y Hermeneutica y eso fue todo. No se lo obtiene una y otra vez a menos que se haga algo espectacular. … Este es uno de los problemas que tenemos los académicos: no se nos recompensa por los sistemas de producción. (Geoffrey Rockwell, en Jacomy, 2023)

Schreibman y Hanlon (2010), en su relevamiento sobre la creación de herramientas en las humanidades digitales (DH), informan que los creadores de herramientas de las DH son conscientes de que su práctica «no se considera necesariamente investigación académica en sí misma. Esta percepción, sostenida por la gran mayoría de los comités de evaluación para la

titularidad (tenure), los comités de tesis y nuestros pares, puede constituir un obstáculo para el desarrollo del campo de las humanidades digitales» (Schreibman & Hanlon, 2010).

Retomemos aquí la dicotomía clásica de la gestión de proyectos entre trabajo de proyecto y trabajo de operaciones. El principal problema de la creación de herramientas en la academia no es el trabajo de proyecto; por ejemplo, construir un prototipo, porque ocurre una sola vez, constituye un acontecimiento y, como tal, puede valorizarse, normalmente mediante una publicación con revisión por pares. El principal problema es el trabajo de operaciones, como el mantenimiento, que debe hacerse una y otra vez. El carácter interminable de esa parte de la creación de herramientas es un problema tanto para la obtención de fondos como para la valorización académica.

Los organismos de financiamiento suelen buscar invertir en proyectos autosostenibles, lo que en esencia significa minimizar los costos operativos. Las herramientas tienen costos operativos, sobre todo cuando se vuelven populares. Estos costos no consisten sólo en el mantenimiento del código o en sostener la infraestructura necesaria para construir y servir la herramienta, sino que incluyen también la gestión de la comunidad de usuarios, el mantenimiento del sitio web y de la documentación, etc. Estas tareas son difíciles de financiar. Algunas, aunque no todas, pueden delegarse en la comunidad de usuarios; y la comunidad necesita un punto de contacto (un gestor de proyecto) para mantenerse comprometida.

Del mismo modo, las operaciones suelen considerarse fuera del alcance de la publicación académica esencialmente por dos razones: no aportan (o no parecen aportar) nada nuevo, y su producto no se considera conocimiento. La mayoría de los actos de mantenimiento se verán como conservadores: preservar el statu quo o volver a él. La parte ontológica del mantenimiento es invisible cuando el cambio se destila lentamente a lo largo de múltiples intervenciones rutinarias. La batalla académica por la novedad sólo puede librarse en el terreno de los cambios espectaculares. Pero incluso cuando el trabajo de los creadores de herramientas puede presentarse como innovador, con demasiada frecuencia será rechazado durante la revisión por pares por no consistir en un conocimiento abstracto y reutilizable. Describir un nuevo algoritmo merece una publicación, pero implementarlo en una nueva herramienta no. El código se ve en general como la producción material de un trabajador, no como la producción de conocimiento de un pensador. En la academia, la programación no suele concebirse como escritura, aunque la situación está cambiando, como veremos.

> En mi universidad, mi informe anual va a ser evaluado por un grupo de directores de departamento y colegas de humanidades. Me llevo bien con ellos, y si presentara una herramienta flamante ese año, la mayoría me daría reconocimiento por ella; pero no al año siguiente. Sería igual que con un libro: se obtiene reconocimiento ese año, no el siguiente. El hecho de mantenerla no cuenta. (Geoffrey Rockwell, Jacomy, 2023)

EL PROBLEMA DEL TRABAJO OCULTO EN LA CREACIÓN ACADÉMICA DE HERRAMIENTAS

En el corazón de la economía de la creación de herramientas en las DCSSH se encuentra la realidad de que implica trabajo oculto: una parte considerable del tiempo y el esfuerzo necesarios para dar existencia a las herramientas y mantenerlas permanecerá oculta. Sin financiamiento por ser invisible, ese esfuerzo suele correr a expensas de los creadores de herramientas.

Como escriben Derrick et al. (2024) acerca del Festival of Hidden REF (por Research Excellence Framework) que organizan, «la recompensa académica es ciega a mucho trabajo que resulta crucial para la investigación, y a roles que hacen posible esa investigación». Los llaman «roles ocultos», y no se trata sólo de los creadores de herramientas. «Algunos, como los bibliotecarios y los técnicos, han existido desde que existe la investigación misma, mientras que roles como los de gestores y administradores de software de investigación, ingenieros de software de investigación y gestores de datos han surgido más recientemente (Kerridge et al., 2023). Estos "roles ocultos" han sido pasados por alto por una cultura que no logra mirar más allá de las publicaciones como medio para identificar y recompensar el valor en la cultura de la investigación» (Derrick et al., 2024).

Smithies et al. (2019), que mantienen proyectos de DH en el King's Data Lab (KDL, un laboratorio del King's College London), defienden la necesidad de hacer visible el problema. «Equipos como el KDL lidian con los problemas que plantean innumerables presiones: no es justo ni productivo que a los investigadores principales (PI), a los organismos de financiamiento y a la comunidad en general se les oculte la realidad de esas presiones. Significativamente, en lugar de ver estos problemas como motivo de vergüenza que debe ocultarse a administradores, financiadores y colegas dentro y fuera de nuestra institución, el laboratorio los reconoce como oportunidades de investigación para desarrollar metodologías mejoradas» (ibid).

El trabajo oculto del creador de herramientas se da en dos situaciones distintas: la creación y el sostenimiento de la herramienta. La creación de herramientas es a menudo una actividad no solicitada. Las herramientas demuestran su valor a lo largo de su ciclo de vida, cuando se hace visible que son científicamente productivas porque conducen a hallazgos significativos, son citadas en artículos fundamentales o porque suficientes investigadores las utilizan en sus investigaciones. Las herramientas nacen sin que el público las desee; se las aprecia a posteriori. La creación de herramientas corre necesariamente a expensas de sus creadores porque no hay más deseo de ella que el suyo propio. Algunos pueden crear herramientas como pasatiempo en su tiempo libre; muchos creadores de herramientas son investigadores que lo hacen como parte de su investigación, por ejemplo, en el marco de una subvención. En ese sentido, la creación de herramientas no carece necesariamente de financiamiento, pero aun así puede permanecer oculta en la medida en que la institución científica no la diferencia de otras actividades científicas como realizar trabajo de campo, extraer datos, ejecutar pruebas comparativas o escribir

artículos. Este camuflaje puede beneficiar inicialmente al creador de herramientas porque protege su libertad; pero si la herramienta adquiere una vida pública, surgen problemas.

Sostener una herramienta existente implica un tipo muy distinto de trabajo oculto. Las instituciones académicas suelen enfrentar tres dificultades a la hora de reconocer el sostenimiento de herramientas: ignoran en qué consiste, no esperan que sea así y no encaja en su perspectiva sobre la ciencia.

En primer lugar, las universidades suelen ignorar qué tipo de trabajo es necesario para mantener una herramienta en funcionamiento. Está, por supuesto, el mantenimiento, invisible porque no constituye un acontecimiento. Pero la herramienta no es sólo el código fuente; es también una serie de artefactos necesarios para su comunidad de usuarios. La vida de la herramienta requiere gestión del proyecto y del producto, vigilancia tecnológica, redacción de documentación, diseño web y gestión de la comunidad. Cuanto más ajenas resultan esas tareas a la alta dirección del creador de herramientas, más pueden considerarse innecesarias o incluso ilegítimas.

En segundo lugar, la creación de herramientas es ajena a la cultura universitaria. Incluso cuando la dirección apoya el trabajo del creador de herramientas, surgen muchos problemas prácticos. A partir de su experiencia en el mantenimiento de proyectos de DH en el KDL, Smithies et al. señalan que a menudo «la cultura circundante ofrece poco apoyo. Los organismos de financiamiento pueden exigir planes de gestión de datos para garantizar que el contenido se maneje adecuadamente, según el país de origen, pero parecen incapaces de abordar los complejos problemas asociados a los sistemas que generan y almacenan esos datos. Los colaboradores a menudo se incomodan ante el uso de marcos de la "industria" y del lenguaje "empresarial", y sugieren (comprensiblemente) que ello va en detrimento de la cultura de la investigación académica. Mientras tanto, algunos críticos de las humanidades digitales parecen ignorar que existe un universo de investigación digital de altísima calidad, hecha a medida pero en riesgo, muy alejada de los valores y los imperativos comerciales de la ideología de Silicon Valley» (Smithies et al., 2019). Esta resistencia cultural genera una fricción adicional cuando el trabajo de los creadores de herramientas se hace visible. El problema del trabajo oculto no es coyuntural sino sistémico.

El principal punto de fricción sistémica es la expectativa de la universidad de que los productos de las SSH no tengan costos operativos. Se supone que la difusión cuesta algo una sola vez, como los cargos por procesamiento de artículos, pero no durante todo el año. Las estructuras de financiamiento académico no son adecuadas para los costos fijos, que suelen estar a cargo de las funciones de apoyo. Como informan Smithies et al. sobre la situación del KDL, «los propios financiadores indicaron que no esperaban (o no estaban dispuestos a apoyar) el desarrollo de recursos digitales a largo plazo o permanentes: sin la buena voluntad de los colegas y de la institución anfitriona, la mayoría de ellos se habría cerrado hace años. Su futuro a menudo sólo se discutía de manera tangencial, eludido en las conversaciones entre los equipos técnicos y los PI durante el proceso de desarrollo, con la optimista esperanza de que "algo"

sucediera finalmente … Los PI trasladaban la responsabilidad emocional a los equipos técnicos, y viceversa: la responsabilidad contractual real normalmente quedaba sin definir» (Smithies et al., 2019). Una institución académica puede no tener ningún interés en apoyar activamente las herramientas. Y cuando lo tiene, siempre significa una de dos cosas: tiempo de trabajo dedicado del personal de apoyo, o subvenciones únicas; nunca significa tiempo remunerado para el investigador-creador de herramientas. Las universidades simplemente no saben cómo evaluar el mantenimiento de herramientas realizado por un investigador o, a la inversa, el trabajo académico realizado por un ingeniero informático. Lo que nos lleva a nuestra última dificultad.

En tercer lugar, la creación de herramientas no encaja en la perspectiva de las universidades sobre la ciencia, porque tiende un puente entre las funciones de apoyo y las académicas (como analizamos en el capítulo 21). Esto supone un riesgo para el creador de herramientas. En una situación en la que la dirección de la universidad llega a comprender lo que implica la creación de herramientas (dificultad 1) y reconoce las fricciones culturales (dificultad 2), bien podría concluir que la creación de herramientas no es una práctica científica legítima. Las universidades tienden a concebir las herramientas como infraestructuras, a menudo inspirándose en famosos proyectos de las ciencias naturales como el Gran Colisionador de Hadrones: iniciativas organizadas de arriba hacia abajo, mantenidas por un personal de apoyo dedicado, pero utilizadas por un grupo separado de académicos y dirigidas por un consejo científico. Es comprensible que, cuando una universidad paga para sostener una infraestructura de investigación, conserve a cambio la supervisión. Desde el punto de vista de la universidad, sostener la herramienta de un investigador también presenta riesgos: puede sentar un precedente (¿y si otros investigadores exigen costos operativos para sus propios productos científicos?), y compromete el nombre de la universidad sin una supervisión suficiente, por ejemplo, si la herramienta se ve involucrada en una filtración de datos o en actividades ilegales (crisis de relaciones públicas). La libertad académica que protege al investigador-creador de herramientas también lo aísla institucionalmente y tiende a que su trabajo no se contabilice.

La creación de herramientas es una paradoja para las universidades. No quieren que el personal de apoyo tenga su propia agenda de investigación, porque ese tipo de agencia es privilegio de los académicos. No permitirán que un ingeniero de software de investigación (RSE) cree nuevas herramientas de investigación por su cuenta, pero sí permitirán que lo hagan los investigadores en la medida en que cuente como producto científico. Las universidades quieren que la creación de herramientas se ubique del lado de la investigación. Sin embargo, también quieren que el sostenimiento de las herramientas se ubique del lado del apoyo. Quieren supervisión y control sobre cualquier proyecto con costos operativos, al tiempo que piden a los investigadores que enseñen, publiquen y aporten nuevo valor. Esas demandas contradictorias dificultan la creación de herramientas. Superar esa mentalidad es posible, pero exige cuestionar una perspectiva profundamente arraigada en la cultura de nuestras instituciones científicas.

Dejamos la última palabra sobre este tema a Smithies et al. (2019). «Con toda probabilidad, la sostenibilidad del software seguirá siendo un problema apremiante en el futuro previsible en

todas las disciplinas de investigación. … Las estrategias de financiamiento sostenibles deben basarse en una estimación de costos transparente que incluya los costos de infraestructura y mantenimiento, y volverse más simples y fiables gracias a buenas prácticas establecidas. Para que esto sea eficaz, es necesario desarrollar métodos realistas de estimación de costos, compartirlos entre los socios del producto e incorporarlos a la política de financiamiento» (ibid, p. 13).

READAPTAR LA PUBLICACIÓN PARA LA CREACIÓN DE HERRAMIENTAS

> En ese momento, [Stéfan Sinclar y yo] teníamos el problema académico de que había que publicar. Así que, hasta cierto punto, nuestra solución fue esta praxis. Planificábamos un proyecto y lo llevábamos a cabo. Por un lado, actualizábamos Voyant y, por otro, escribíamos un artículo que presentábamos en un congreso. Los artículos se convirtieron en un libro, y se lanzó Voyant 2.0. Nuestro libro salió en 2016: Hermeneutica. (Geoffrey Rockwell, en Jacomy, 2023)

Aunque el desarrollo de software no pasa naturalmente por trabajo académico, los creadores de herramientas disponen de múltiples estrategias para valorizarlo académicamente de todos modos. Los investigadores-creadores de herramientas en particular, es decir, quienes ocupan un cargo académico, tienen la necesidad existencial de hacerlo. Esto exige superar un problema de formato. Como señalan Jay et al. (2021), «si bien el software es hoy central para la producción de investigación, es difícil —posiblemente imposible— representarlo adecuadamente en las publicaciones académicas estándar». Pero los creadores de herramientas han encontrado estrategias. De la más encubierta a la más abierta, pueden (1) capitalizar la ciencia que posibilita la herramienta manteniendo en secreto la herramienta misma; (2) liberar públicamente la herramienta pero sólo publicar la ciencia que posibilita; (3) publicar un artículo sustituto; (4) publicar la herramienta misma como trabajo científico. No es ningún misterio que la comunidad de investigación está evolucionando en esa cuestión, y que las estrategias abiertas se han vuelto más viables en los últimos años. Recorramos esas estrategias.

La primera estrategia es la del creador de herramientas encubierto: un investigador que crea herramientas pero no quiere que se sepa. En cierto sentido, podríamos discutir si su tecnología merece llamarse herramienta (y el propio investigador, creador de herramientas) si no está destinada a compartirse; pero esas demarcaciones carecen de sentido. Lo importante es que la estrategia del creador de herramientas encubierto consiste en capitalizar la ciencia que posibilita su herramienta. Su herramienta es un activo para competir en el mercado de las publicaciones, lo que no lo incentiva a compartirla públicamente.

Una variante de esta situación la constituyen las herramientas cuya existencia se conoce, pero que son tan arcanas de dominar que sólo un pequeño grupo de personas, normalmente en torno al laboratorio del creador de herramientas, tiene las competencias para hacerlo. Las herramientas desarrolladas por Francis Chateauraynaud y Jean-Pierre Charriau, Prospéro y Marlowe, podrían ser de ese tipo (Chateauraynaud, 2003; Chateauraynaud, 2011; para una visión general, véase Angermuller, 2006). La notoriedad de estas herramientas se concentra sin duda en Francia, pero incluso dentro de la investigación francesa, pocos han podido utilizarlas para sus propias

investigaciones. Quizá sería más justo ver Prospéro y Marlowe como experimentos en curso, encarnaciones de un programa de investigación más que herramientas destinadas a ser plenamente operativas en manos de otras personas. Como tales, estas piezas de tecnología han tenido éxito a nivel nacional, si no más allá.

Una segunda estrategia consiste en liberar una herramienta pero sólo publicar la investigación que posibilita. Esto podría ser lo más sensato si la herramienta se desarrolló efectivamente con el único propósito de respaldar un estudio de caso. En el momento de la publicación, la herramienta podría verse como un apéndice de la publicación. Sin embargo, si la herramienta se reutiliza para otros estudios en manos de otras personas, puede adquirir autonomía. Aun así, el creador de herramientas puede no publicar nunca la herramienta misma como un objeto citable. El estudio de caso original a menudo se convertirá en lo que otros investigadores citan para saldar su deuda con la herramienta. Bien puede tratarse de una estrategia consciente del creador de herramientas. La herramienta Contropedia, por ejemplo, se desarrolló primero para un estudio de caso (Weltevrede and Borra, 2016), que se ha convertido en la forma en que se la cita principalmente. En particular, para un investigador-creador de herramientas de las DCSSH, suele ser más fácil publicar un estudio de caso que un artículo sobre una herramienta, porque los espacios de publicación para el primero son más accesibles que para el segundo, como veremos a continuación.

Una tercera estrategia es el artículo sustituto: una publicación que se cita en lugar de la herramienta misma. Gephi es un ejemplo típico. Se lo cita casi universalmente como Bastian et al. (2009), un simple póster presentado en el congreso ICWSM 2009. Esa referencia ha recibido unas 12 800 citas al momento de escribir estas líneas (Google Scholar, agosto de 2024) y recibió el ICWSM 2019 Test of Time Award, aunque el contenido del póster en sí es apenas pertinente, aunque más no sea porque Gephi ha cambiado considerablemente en una década. Obtener una materialización de la herramienta revisada por pares para que recibiera citas fue uno de los objetivos de escribir ese póster. El título de un artículo sustituto suele comenzar con el nombre de la herramienta, por ejemplo, «Scikit-learn: Machine Learning in Python» (Pedregosa et al., 2011). Esto facilita notablemente que el artículo llegue a los investigadores que buscan la herramienta en Google Scholar u otra base de datos. Los creadores de herramientas suelen indicar cómo preferirían que se citara la herramienta. Los creadores de Scikit-Learn remiten explícitamente al artículo mencionado en su sitio web (SciKit-Learn team, 2024).

Un artículo de estudio de caso también puede convertirse en el artículo sustituto, como informan Jay et al.: «La mayoría de los espacios de publicación carecen de directrices que alienten a citar el software directamente … Una solución alternativa habitual es citar en su lugar un artículo relacionado. Este podría ser un artículo que describa un estudio más amplio, en el que el software fue parte integral de esa investigación y se describe en la sección de métodos» (Jay et al., 2021).

Es importante señalar que el artículo sustituto no se revisa por pares como si fuera la herramienta, sino por sus propios méritos. Por ejemplo, si el artículo sustituto es un estudio de

caso, lo que se evaluó es ese estudio de caso, no la herramienta. Para el creador de herramientas, esto tiene dos caras. Por un lado, es posible superar la revisión por pares con un buen artículo pero una mala herramienta, por ejemplo, con errores o problemas metodológicos, siempre que no se note en el artículo. Por otro lado, el creador de herramientas puede tener una herramienta sólida pero tener dificultades para superar la revisión por pares, ya sea porque no es tan bueno escribiendo artículos o porque los criterios de la revista no están adaptados a una herramienta. Un buen ejemplo de ello es el popular algoritmo de disposición espacial de Gephi: ForceAtlas2. Una primera versión de ese algoritmo llevaba algunos años en Gephi, y los usuarios pedían una referencia para citar. Los diseñadores del algoritmo también consideraron valioso documentar el algoritmo y su método. Escribieron un artículo que comparaba el algoritmo con el estado del arte, exponía las ecuaciones en juego y daba cuenta cuantitativamente de los distintos parámetros disponibles, y lo enviaron a la revista de acceso abierto PLoS One. El revisor consideró que el envío no era lo suficientemente novedoso. Las diferencias cualitativas con otros algoritmos, en particular, no pasaron por novedad. De hecho, estaba bastante claro lo que los revisores tenían en mente: pruebas cuantitativas de que el algoritmo era más eficiente. Resultó que los creadores de herramientas podían proporcionar esa prueba comparativa. La agregaron al manuscrito, y fue aceptado (Jacomy et al., 2014). Pero es bastante evidente que esta prueba comparativa es irrelevante para los investigadores que citan el artículo.

El artículo sustituto tiene un defecto evidente: la herramienta evoluciona mientras el artículo sustituto permanece igual. Si el objetivo es acumular capital académico, que la herramienta pase a otras manos plantea un problema, porque la nueva autoría de la herramienta ya no se refleja en el artículo sustituto. Peor aún, los artículos sustitutos tienden a reforzar su estatus con el tiempo debido a su elevado número de citas. Si la herramienta es popular, existe una demanda de citarla, y si los creadores de herramientas no satisfacen esa demanda, puede que otra persona lo haga en su lugar. Quizá de manera menos dramática pero más frecuente, algunas herramientas quedan atadas a un artículo sustituto que no han elegido del todo. Por ejemplo, el artículo más citado sobre Jupyter Notebooks es Kluyver et al. (2016), con unas 3200 citas (Google Scholar, agosto de 2024). Es, de facto, un artículo sustituto. Sin embargo, en 2016, cuando alguien preguntó en el GitHub de la herramienta cómo citarla, los creadores respondieron: «Estamos trabajando en un artículo de referencia citable como corresponde para Jupyter, pero eso está llevando algo de tiempo; si necesita algo ya, creo que este artículo en las actas de la conferencia ElPub de este año es la cita más general que tenemos por el momento: [enlace a Kluyver et al., 2016]» (Jupyter, 2016). Esta issue de GitHub (y la discusión) sigue abierta en el momento de escribir estas líneas (agosto de 2024). El artículo de 2016 surgió como sustituto de la herramienta bajo la presión de la necesidad de los investigadores de saldar su deuda académica con ella.

Cuarta, la última y más reciente estrategia: citar la herramienta directamente, como un artefacto académico. Comencemos por aclarar que citar una herramienta es absolutamente posible, más allá de dificultades técnicas como determinar quiénes son los autores o cuándo se publicó la versión. Es posible dar formato, según las normas APA, a una referencia para la

versión actual de Gephi en el momento de escribir estas líneas: «Ramos Ibáñez, E., Bastian, M., & Jacomy, M. (2023). Gephi (0.10.1) [Software]». La versión citada aquí no está técnicamente revisada por pares, mientras que el artículo sustituto sí lo está, pero eso no impide citarla, aunque algunas revistas tienen la política de que las referencias a publicaciones no revisadas por pares deben señalarse como tales. Los creadores de herramientas pueden facilitar el proceso asociando la herramienta a un identificador de objeto digital (DOI), pero un DOI tampoco es necesario para que algo sea citable (Boettiger, 2013). Para aliviar aún más la dificultad técnica y/u obtener revisión por pares, los creadores de herramientas tienen ahora la oportunidad de publicar lo que Jay et al. llaman un «artículo de software». Este se diferencia de un artículo sustituto en que la revisión por pares evalúa la herramienta en sí, y no otra cosa, como un estudio de caso. El artículo de software «existe únicamente para describir el software, en un medio como SoftwareX, el Journal of Open Research Software (JORS), el Journal of Open Source Software (JOSS) o F1000 Research» (Jay et al., 2021). Aunque no resuelve todos los problemas, tiene la ventaja de ser más fiel a la práctica de la creación de herramientas y más accesible para los creadores de herramientas a quienes les cuesta escribir artículos académicos.

El investigador-creador de herramientas que sabe desenvolverse en la academia tiene acceso a todas esas estrategias y puede elegir la más adecuada en cada situación. Una herramienta muy popular también puede aportar una cantidad de citas muy superior a la que suelen obtener los investigadores de las DCSSH. La creación de herramientas también puede tener algunas ventajas para las carreras académicas en las DCSSH si el creador de herramientas tiene suficiente éxito y las estrategias adecuadas; pero esas estrategias requieren las competencias necesarias para escribir y superar la revisión por pares con suficiente eficacia. No están al alcance de perfiles menos inclinados a la investigación, como el ingeniero de software de investigación (RSE), que debe recurrir en cambio a otras vías, como asociarse con un investigador, para ser valorado en la academia.

> Esa es, creo, una de las cosas geniales que hicimos: al desarrollar esta praxis de experimentos, logramos que el desarrollo fuera académico. Logramos asegurarnos de obtener reconocimiento académico por la herramienta porque siempre la desarrollábamos mientras escribíamos artículos, presentábamos ponencias en congresos y, finalmente, el libro, MIT press … Siempre nos asegurábamos de tener dos o tres ponencias en congresos cada verano. Las ponencias informaban sobre las herramientas y a menudo tenían un componente académico. Pero luego decíamos que también estábamos adaptando esta herramienta. … Creo que todos en las humanidades digitales tienen que encontrar una manera [de hacer que el desarrollo sea académico]. Mis colegas en ciencias de la computación tienen algunos de los mismos problemas. Nadie les va a dar la titularidad (tenure) por escribir código. Por otro lado, tampoco se la van a dar sólo por escribir teoría. (Geoffrey Rockwell, en Jacomy, 2023)

READAPTAR LAS SUBVENCIONES PARA LA CREACIÓN DE HERRAMIENTAS

> Una de las principales cosas que hago [para conseguir financiamiento] funciona porque Voyant tiene suficiente éxito. Digamos que usted está escribiendo una solicitud de subvención. Es una subvención grande, y hay un lugar para Voyant, quizá para otra herramienta y Voyant. Hacemos una especie de trato. Usted incluye Voyant en su solicitud y yo escribo una carta de apoyo; o me

> incluye como coinvestigador, pero a cambio incorpora allí una partida presupuestaria. Así, durante cinco años, hay una cierta cantidad de financiamiento que me llega para dar soporte a Voyant, para hacer lo que usted quiere hacer, por ejemplo, agregar etiquetado de categorías gramaticales. (Geoffrey Rockwell, en Jacomy, 2023)

Obtener subvenciones es una estrategia válida para financiar herramientas DC para las SSH, en la medida en que se cumplan algunas condiciones. La primera condición es que la herramienta debe ser lo bastante popular y reconocida como para que un organismo de financiamiento confíe en su validez académica. Debe quedar claro que hacer avanzar la herramienta contribuye al proyecto académico defendido en la solicitud. La segunda condición es que la solicitud debe presentar el soporte de la herramienta como algo que crea valor académico. En pocas palabras, se supone que las subvenciones no financian el mantenimiento; la solicitud debe ofrecer la implementación de nuevas funcionalidades. La tercera condición es que debe haber alguien que escriba la solicitud. Esto suena trivial, pero podría ser el mayor problema. Es complicado que una sola persona tenga el tiempo y las competencias necesarias para hacerlo todo: programar la herramienta, escribir artículos (y enseñar) y redactar solicitudes de subvención. De manera más realista, el creador de herramientas forma parte de una red de colaboración que lo ayuda con eso, ya sea porque desarrolladores más competentes dan soporte a la herramienta mientras él gestiona el proyecto (en particular, redactando solicitudes de subvención), o porque algunos investigadores más competentes lo incluyen en sus solicitudes para que pueda concentrarse en el soporte de la herramienta. En ambos casos, el creador de herramientas necesita una red de apoyo.

Digámoslo con todas las letras: mientras el mantenimiento de las herramientas DC para las SSH sea un tabú, la creación de herramientas es insostenible en estas disciplinas. A menos que se financie explícitamente, las dos únicas maneras de que el mantenimiento se lleve a cabo son o bien en el tiempo personal de los creadores de herramientas, o bien camuflado como otra cosa, como agregar funcionalidades o usar herramientas para estudios. En realidad, es bastante fácil para el creador de herramientas sobrevaluar el costo de implementar una funcionalidad en una solicitud de subvención para obtener algo de tiempo para hacer mantenimiento. Hasta cierto punto, todas las partes involucradas pueden incluso ser conscientes de ello y estar implícitamente de acuerdo con la práctica. Esto, desde un punto de vista práctico, puede parecer una situación en la que todos ganan; pero no resuelve el problema de sostener la herramienta a largo plazo. De hecho, agrava el problema porque refuerza el tabú. Hacer pasar de contrabando la creación de herramientas como si fuera otra cosa ha sido la norma durante mucho tiempo, pero la situación está cambiando y están surgiendo nuevas maneras de hacerla sostenible.

LA SITUACIÓN DE LA CREACIÓN DE HERRAMIENTAS EN LAS DCSSH ESTÁ CAMBIANDO

A medida que la práctica de programar se normaliza, se acepta mejor como producción académica. La masificación de los medios computacionales, por ejemplo mediante el uso de

Jupyter Notebooks, favorece un gradiente más matizado de investigadores híbridos. Cada vez es menos necesario comprometerse con uno de dos lados, el desarrollo o la investigación. Muchos investigadores recurren a métodos computacionales sin definirse como científicos digitales o computacionales. La parte de programación es sólo un momento en el recorrido metodológico impulsado por sus datos y sus preguntas de investigación. A medida que todos llegan a programar un poco, la programación se reconoce con mayor frecuencia como una forma de escritura.

Se ha vuelto fácil publicar código, por ejemplo a través del Open Science Framework (Foster & Deardorff, 2017) o mediante un DOI de Zenodo (European Organization for Nuclear Research & OpenAIRE, 2013). Zenodo incluso permite versionar software para representar una nueva versión y actualiza automáticamente el DOI ante un commit de GitHub. Además, el Journal of Open Research Software (JORS) «publica Metaartículos de Software revisados por pares que describen software de investigación con un alto potencial de reutilización. Estamos trabajando con varios repositorios especializados e institucionales para garantizar que el software asociado se archive y preserve profesionalmente y esté disponible en acceso abierto. Igualmente importante, el software y los artículos serán citables, y se hará un seguimiento de su reutilización» (Software Sustainability Institute, 2024).

En algunos países, como los Países Bajos (Quality and Relevance in Humanities, 2024), los indicadores de calidad y relevancia en las humanidades ahora también incluyen el software (aunque a menudo sin especificar suficientemente cómo cuenta y sin mencionar el mantenimiento del software). En el Reino Unido, el KDL ha venido formalizando una Checklist for Digital Outputs Assessment (Ciula, 2019) y el Software Sustainability Institute ha propuesto una guía de buenas prácticas para la presentación de productos de software (Jay et al., n.d.). En Francia, Gomez-Diaz y Recio (2019) han propuesto el protocolo «CDUR» para la evaluación del software de investigación (RS), que consiste en las etapas cronológicas de «Citación, Difusión, Uso e Investigación» (ibid), y que aborda una serie de problemas identificados: la producción de software no se registra como producción de investigación por derecho propio (sólo como un medio para un artículo); el software no tiene un estatus claro (nombre, autores, fechas, versión …); se difunde de manera caótica (en blogs personales, forjas, redes sociales … pero no en la página del laboratorio); no tiene licencia; su estatus jurídico es problemático porque no tiene licencia ni autores; el laboratorio no tiene una política declarada sobre la producción de software; y la producción de software no se evalúa. Por supuesto, las herramientas de investigación en general no tienen todos esos problemas. El protocolo CDUR fue diseñado para abordar estas cuestiones en orden y ayudar no sólo al creador de herramientas, sino también a los comités de evaluación y a quienes toman decisiones, a dar cuenta del valor de la producción de software de investigación.

Hemos visto que abrir el código fuente de una herramienta no basta para hacerla sostenible. Tampoco basta un artículo sustituto, una entrada en Zenodo o una solicitud de subvención. Sin embargo, muchos pequeños cambios en las prácticas de investigación pueden conducir a

transformaciones en las culturas académicas y transformar las instituciones. La presencia de ingenieros de software de investigación ayuda a preservar las competencias técnicas y a normalizar las prácticas de software en la academia. Aliviar el tabú del mantenimiento sigue requiriendo la aparición de un financiamiento estructural semipermanente, pero este podría estar en camino. En Europa, los Países Bajos son particularmente activos en ese frente, por ejemplo con la efímera subvención de infraestructura PDI-SSH que financió el proyecto CAT4SMR (DMI-TCAT, YouTube Data Tools, 4CAT). También cabe destacar la Amsterdam Declaration on Funding Research Software Sustainability, que busca cambiar la manera en que los financiadores abordan el software de investigación. Además, la convergencia y consolidación de los movimientos de código abierto y de ciencia abierta pueden dar lugar a la voluntad institucional, a nivel europeo, de construir bienes comunes digitales sostenibles. Por ejemplo, la fundación neerlandesa NLnet ha tenido un éxito notable en la estabilización de proyectos de código abierto de pequeña y mediana escala.

Las herramientas y los creadores de herramientas son de muchos tipos y se encuentran en situaciones diversas, y ningún modelo único funcionará para todos, pero aun si no podemos prever dónde podrían producirse futuros avances que mejoren la condición del creador de herramientas, las tendencias parecen darnos motivos para ser optimistas, tanto para el creador de herramientas como para la comunidad de investigación en general. Como concluyen Jay et al. en su casi manifiesto, «el reconocimiento oficial del software como producto de investigación será, en última instancia, transformador, al mejorar la calidad, la reproducibilidad y la escalabilidad de nuestra producción de conocimiento, así como al reconocer el papel a menudo oculto del creciente número de investigadores académicos que dedican la mayor parte de su tiempo a escribir código» (Jay et al., 2021).

REFERENCIAS

Angermuller, J. (2006). Francis Chateauraynaud, Prospéro. Une technologie littéraire pour les sciences humaines. Mots. Les langages du politique, (81), 119–121.

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. In Proceedings of the International Aaai Conference on Web and Social Media, 3(1), 361–362.

Boettiger, C. (2013). Doi != Citable. Lab Notebook, June 3. https://www.carlboettiger.info/2013/06/03/DOI-citable.html (accessed 19/06/2024).

Chateauraynaud, F. (2003). Marlowe. Vers un générateur d'expériences de pensée sur des dossiers complexes. Bulletin de méthodologie sociologique/Bulletin of sociological methodology, (79), 6–32.

Chateauraynaud, F. (2011). Argumenter dans un champ de forces. Essai de balistique sociologique. Sociologie Du Travail, 55(2), 270–271. Éditions Petra.

Ciula, A. (2019). KDL Checklist for Digital Outputs Assessment. https://doi.org/10.5281/zenodo.3361580

Derrick, G., Hettrick, S., Baker, J., Karoune, E., Kerridge, S., Fletcher, G., Chue Hong, N., Ballantyne, L., Fransmann, J., & Roche, T. (2024). Shaping the future of research evaluation: Insights from The Festival of Hidden REF. https://doi.org/10.1108/EGP-02-2024-0001

European Organization For Nuclear Research, & OpenAIRE (2013). Zenodo. https://doi.org/10.25495/7GXK-RD71

Foster, E. D., & Deardorff, A. (2017). Open science framework (OSF). Journal of the Medical Library Association, 105(2), 203.

Gomez-Diaz, T., & Recio, T. (2019). On the evaluation of research software: the CDUR procedure. F1000Research, 8, 1353.

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. Reticular. https://reticular.hypotheses.org/8039

Jacomy, M., Venturini, T., Heymann, S., & Bastian, M. (2014). ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. PloS one, 9(6), e98679.

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. International Journal of Digital Curation, 16(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Jay, C., Hettrick, S., Chue Hong, N., Haines, R., & Grieve, S. (n.d.). Good Practice in Submitting Software Outputs to REF2021. Software Sustainability Institute. https://www.software.ac.uk/guide/good-practice-submitting-software-outputs-ref2021

Software Sustainability Institute (2024). Journal of Open Research Software. https://openresearchsoftware.metajnl.com/ (accessed 03/09/2024)

Jupyter (2016). Citing Jupyter · Issue #190 · jupyter/jupyter. GitHub. https://github.com/jupyter/jupyter/issues/190 (accessed 02/09/2024)

Kerridge, S., Poli, S., & Yang-Yoshihara, M. (Eds.) (2023). The Emerald handbook of research management and administration around the world. Emerald Publishing Limited.

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter Notebooks–a publishing format for reproducible computational workflows. In B. Schmidt & F. Loizides (Eds.), Positioning and power in academic publishing: Players, agents and agendas (pp. 87–90). IOS Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Quality and Relevance in Humanities (2024). Software. https://www.qrih.nl/en/case-studies-and-indicators/qrih-indicators/research-products-for-peers/software (accessed 03/09/2024).

Ramos Ibáñez, E., Bastian, M., & Jacomy, M. (2023). Gephi (0.10.1) [Software].

Schreibman, S., & Hanlon, A. M. (2010). Determining value for digital humanities tools: Report on a survey of tool developers. Digital Humanities Quarterly, 4(2). https://www.proquest.com/docview/2555208512/abstract/8B9E2E7ABEE94C7EPQ/1 (accessed 01/09/2024).

SciKit-Learn team (2024). About us. Scikit-learn. https://scikit-learn.org/stable/about (accessed 02/09/2024).

Smithies, J., Westling, C., Sichani, A. M., Mellen, P., & Ciula, A. (2019). Managing 100 digital humanities projects: Digital scholarship and archiving in King's Digital Lab. Digital Humanities Quarterly, 13(1).

Weltevrede, E., & Borra, E. (2016). Platform affordances and data practices: The value of dispute on Wikipedia. Big Data & Society, 3(1). https://doi.org/10.1177/2053951716653418

25. En defensa de la figura del investigador-creador de herramientas

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

A modo de conclusión, quisiéramos volver al creador de herramientas como figura dentro del grupo de investigación. En el capítulo 20 sostuvimos que las herramientas son portadoras de teoría. Son instrumentos de poder y contribuyen a la política del método (Adams et al., 2005). Este punto también pone de relieve la diferencia entre el ingeniero de software de investigación (RSE) y el investigador-creador de herramientas en lo que respecta a la autoría, pero de otra manera. La propiedad de la herramienta conlleva responsabilidad metodológica. El supuesto agnosticismo del RSE lo incentiva a renunciar a la autoría, replegarse en el campo del soporte tecnometodológico y eludir la responsabilidad. El RSE está en una posición débil para defender o discutir el impacto científico de la herramienta, porque esa es tarea de los académicos. La posición del investigador-creador de herramientas es más sólida en ese sentido, no sólo porque está más dispuesto a asumir la autoría y la responsabilidad, sino también porque tiene un interés directo en defender la herramienta como encarnación de su propia investigación. Aunque, como hemos visto, la herramienta es siempre más que la intención de su creador y puede empoderar a los usuarios de maneras imprevistas.

Hemos visto en los capítulos 21 y 22 que la creación de herramientas en las DCSSH abarca un espectro de perfiles y situaciones, algunos más inclinados hacia el mantenimiento de infraestructuras, otros hacia la invención de experimentos científicos. El creador de herramientas en general está orgulloso de su herramienta y se ocupa de ella, incluso cuando le causa problemas. Este apego es un contrapeso necesario frente a muchas cargas: mantenimiento y reparación, recaudación de fondos, gestión de proyectos … Que la herramienta sea suya o no marca una diferencia para el creador de herramientas, lo que a su vez incide en el diseño, la presentación y la difusión de la herramienta dentro de la comunidad de las DCSSH.

Este es un primer punto en el que nos resulta útil distinguir entre el RSE y su contraparte en el espectro de la creación de herramientas, el investigador-creador de herramientas (o «investigador-desarrollador» para Baxter et al., 2012). El investigador-creador de herramientas ocupa un puesto académico y la herramienta que crea es una extensión de su investigación, una expresión de su agenda científica. En cambio, el RSE ocupa un puesto de apoyo, definido por la institución como un recurso para investigadores cuya agenda científica prevalece sobre la suya (si es que la institución siquiera advierte que un RSE puede tener una). La lógica de esta división del trabajo exige que las herramientas sean inventadas por investigadores-creadores de herramientas y luego mantenidas por el RSE. Pero esto omite que el primero está apegado a su

herramienta y no quiere renunciar al control ni a la autoría, mientras que el segundo no lo está y puede no querer asumir el control ni la autoría. El perfil del RSE tiene que ver con el cuidado de las infraestructuras, pero se queda corto en lo que respecta a herramientas que no se parecen a infraestructuras, sino más bien a productos.

En el capítulo 23 repasamos el ciclo de vida de las herramientas y vimos que muchas herramientas DC para las SSH nacen de la práctica de un investigador-creador de herramientas, pero también que el mantenimiento es una carga difícil de afrontar para el creador de herramientas. En este sentido, el investigador-creador de herramientas está en la posición más débil. El mantenimiento es recurrente pero invisible y, como desarrollamos en el capítulo 24, pertenece al régimen de las operaciones (consideradas costos fijos), por oposición a los proyectos (considerados inversiones). En la división tradicional del trabajo en las universidades, se supone que las operaciones se asignan a los RSE, mientras que la dirección de proyectos corresponde a los investigadores. Aquí, de nuevo, resulta útil distinguir entre el RSE y el investigador-creador de herramientas. Es mucho más fácil para un RSE-creador de herramientas justificar el trabajo de reparación y mantenimiento, pero, como se supone que el RSE da apoyo a otros, también existe el riesgo de que sus superiores le asignen tareas distintas del mantenimiento de su herramienta. El investigador-creador de herramientas tiene el problema opuesto: es libre de mantener su herramienta, pero debe hacerlo durante un tiempo de investigación que ningún organismo de financiamiento está dispuesto a costear.

De manera similar, ocuparse de una herramienta conlleva trabajo de gestión de proyectos, como redactar solicitudes de subvención o comunicarse con colaboradores y usuarios, que es mejor que asuma un investigador-creador de herramientas que un RSE.

Nuestro último punto en este texto será defender la figura del investigador-creador de herramientas como un personaje importante para repensar las condiciones de producción del aparato de las DCSSH. En pocas palabras, existe el riesgo de que la figura emergente del RSE conduzca a reforzar una división institucional que es antinómica respecto de la creación de herramientas como práctica. Pero comencemos nuestro argumento reconociendo que vemos la aparición de redes de RSE como un avance fantástico para la causa de la creación de herramientas en la academia. También reconocemos que el RSE no es necesariamente un creador de herramientas, y que cuidar las infraestructuras académicas es contribuir a la ciencia por derecho propio, independientemente de que quien las cuida entienda o no su práctica como creación de herramientas.

Nuestras observaciones nos llevan a creer que, en realidad, existe un continuo de perfiles mucho más rico de lo que sugiere la división entre apoyo y academia. Algunos RSE publican y afirman una agenda científica como si fueran académicos; y algunos investigadores-creadores de herramientas han encontrado estrategias para que el trabajo de mantenimiento se lleve a cabo. Pero, con mayor frecuencia, la clave de una creación de herramientas sostenible es la colaboración a través de la división entre apoyo y academia. Hemos visto este patrón en la Digital Methods Initiative de Ámsterdam, en el médialab de Sciences Po en París y en el King's

Data Lab de Londres (entre otros lugares). La colaboración ideal es entre un RSE-creador de herramientas y un investigador-creador de herramientas, es decir, entre un técnico inclinado a la investigación y un académico inclinado al desarrollo. Al estar presentes a ambos lados de la división institucional, pero siendo lo bastante intercambiables, pueden encontrar las mejores estrategias para que su herramienta tenga éxito a largo plazo. Esa fue la forma de la mítica colaboración entre Stéfan Sinclair y Geoffrey Rockwell en Voyant.

Defendemos la figura del investigador-creador de herramientas porque la preeminencia del RSE no debería ocultar que muchas de las herramientas que usamos en las DCSSH fueron inventadas por personas con una visión científica y un compromiso con la práctica técnica crítica (Agre, 1997). Como teoría infraestructurada, una herramienta nunca es la encarnación neutral de un método preexistente, aunque sólo sea porque la implementación remodela el método. Pone en acto algo nuevo y a menudo inesperado. Pero, de hecho, el creador de herramientas nunca inventa el método antes que la herramienta; más bien, desarrolla el método en tanto herramienta. Los métodos del creador de herramientas son nativamente computacionales y sólo secundariamente material publicado, independientemente de la posición del creador de herramientas.

La ilusión gerencial de que nuestro aparato científico puede ser imaginado por cerebros académicos y mantenido por manos de apoyo va en contra de todo lo que sabemos sobre la creación de herramientas. Esperamos que los próximos cambios institucionales reconozcan esta realidad para dar apoyo a formas más híbridas y flexibles de compromiso científico con la tecnología, a ambos lados de la división entre apoyo y academia.

REFERENCIAS

Adams, J., Keane, W., & Dutton, M. (2005). The politics of method in the human sciences: Positivism and its epistemological others. Duke University Press.

Agre, P. E. (1997). Toward a critical technical practice: Lessons learned in trying to reform AI. In G. Bowker, S. L. Star, L. Gasser, & W. Turner (Eds.), Social science, technical systems, and cooperative work: Beyond the Great Divide (pp. 131–157). Psychology Press.

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. Digital Research 2012 (pp. 1–3). University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf